

Advanced Inquiry Project: CESD 850

Growing Greener:
*Integrating Technology with Hydroponics to Mitigate Climate
Change Impacts on Agriculture*

Jared Buckman

April 2025

Table of Contents

Table of Contents.....	2
Abstract.....	4
Define.....	5
Overview.....	5
Problem Definition.....	5
Literature Review.....	10
Stakeholder Requirements.....	12
Feasibility Study Report.....	14
Success Criteria.....	21
Planning.....	23
Overview.....	23
Gantt Chart.....	23
Initial Brainstorming.....	24
Flow chart.....	25
Skills Development Table.....	26
Real-World Impact.....	27
Communication and Feedback.....	28
Research.....	31
Overview.....	31
Essential Findings.....	31
Solution.....	37
Design.....	38
Overview.....	38
Architecture & Design.....	38
Sensor and Hardware Design.....	40
Mobile Application Design.....	42
Physical Structure Design.....	43
Testing Table.....	52
Data Dictionary.....	54
Development.....	56
Overview.....	56
Sensors and Hardware.....	57
Developing The App for User Interaction and Data Monitoring.....	82
Physical Structure of the System and 3D-Printed Sensor Housing.....	90
Testing.....	97
Overview.....	97
Testing Table.....	97
Process.....	99
Evaluation.....	104
Conclusion.....	105
Future Development.....	106
Appendix.....	107
Images and Graphs.....	107

Codebase 9.1 (Complete Python Code).....	151
List of Abbreviations.....	155
References.....	156

Abstract

Hydroponics is an innovative soil-free farming technique that enables efficient water and nutrient management, making it a sustainable solution for modern agriculture. The goal of this project is to exemplify the benefits of cultivating plants hydroponically, whether the client is a large-scale food company or a home chef. In either case, the benefits of hydroponics are profound.

This project aims to develop an automated hydroponic system that integrates sensors for data collection, and a mobile application for real-time monitoring and control. The system consists of a DS18B20 water temperature sensor, a DF Robot Gravity analog pH sensor, and a CQ Robot non-contact liquid level sensor connected to an Arduino Uno and Raspberry Pi 5. In addition, a 2x16 LCD is connected to the Raspberry Pi to display the sensor data without the mobile app. The mobile app provides users with real-time data and remote control capabilities, while a 3D-printed housing protects the sensors within the physical hydroponic setup.

This automated hydroponic system promotes sustainability by optimizing water and nutrient usage, reducing waste, and minimizing environmental impact. Unlike traditional soil-based farming, hydroponics requires significantly less water, as it recirculates nutrients without any loss due to soil, preventing runoff and contamination. This also saves users money and time tending to the soil. Hydroponics also saves plants from the constraints of poor soil conditions and environmental pollutants.

This project highlights the potential of automation in hydroponics for improved efficiency and sustainability. If more time and resources are provided, the ideal product would include AI-driven automation and feedback, additional sensors for more precise monitoring, and greater environmental control. This would include auto adjusting the water temperature and pH automatically based on user input, adding an EC sensor for more water data, and using temperature/humidity, light, oxygen, and carbon dioxide sensors to study the environmental conditions. Lastly, a more efficient NFT system would be constructed, offering a more stable structure and uniform light distribution, which would support the growth of larger plants and allow for better control of various variables.

Define

Overview

In this section, I establish the key problem my hydroponic system aims to solve and outline the project's objectives, constraints, and requirements. The goal is to create an efficient, automated hydroponic system that monitors and maintains optimal growing conditions using sensor technology, cloud-based data storage, and a user-friendly mobile application. Throughout this report, phases will be divided into three sections, reflecting the system's core components: sensors and automation, the mobile application, and the physical structure. In this section, I will introduce the problem to be studied, present my preliminary research on existing solutions, identify stakeholders and clients impacted by this project, evaluate its feasibility, and define success criteria—measurable goals used to track progress and effectiveness.

Problem Definition

Context and Background

Traditional agriculture is both a contributor to and a victim of climate change. Conventional farming practices significantly contribute to greenhouse gas emissions, deforestation, and excessive water consumption, leading to soil erosion, loss of biodiversity, and environmental degradation. At the same time, climate change disrupts agricultural productivity by altering weather patterns, reducing water availability, and increasing extreme weather events. These challenges threaten global food security, necessitating a shift toward sustainable agricultural practices that minimize environmental impact while ensuring food production remains stable.

Impact of Climate Change on Crop Cultivation

1. **Temperature Extremes:** Rising temperatures cause heat stress, reducing crop yields, altering growing seasons, and inhibiting seed germination. Some regions may become unsuitable for traditional farming.
2. **Water Scarcity and Droughts:** Changing precipitation patterns and higher evaporation rates lead to water shortages, making irrigation more difficult—especially in arid and semi-arid regions.
3. **Unpredictable Weather Events:** Increased storms, floods, and frosts disrupt planting and harvesting cycles, damage crops, and degrade soil quality.
4. **Pests and Diseases:** Warmer temperatures and increased humidity create favorable conditions for pests and plant diseases, increasing crop losses and reliance on pesticides.
5. **Soil Degradation:** Rising temperatures, erosion, and salinization decrease soil fertility, forcing farmers to use more fertilizers or seek alternative cultivation methods.

Impact of Agriculture on the Environment

1. **Deforestation** – Clearing land for farming reduces biodiversity and contributes to carbon emissions.
2. **Soil Degradation** – Overuse of land leads to erosion, nutrient depletion, and desertification.
3. **Water Use & Pollution** – Irrigation depletes freshwater sources, and runoff from fertilizers and pesticides contaminates water bodies.
4. **Greenhouse Gas Emissions** – Livestock farming produces methane, and fertilizers release nitrous oxide, both potent greenhouse gases.
5. **Loss of Biodiversity** – Some practices, such as monoculture farming, reduce habitat diversity, endangering various species.
6. **Pesticide & Herbicide Impact** – Chemicals harm pollinators (e.g., bees), disrupt ecosystems, and may affect human health.

Problem Statement

Current agricultural systems are heavily reliant on arable land, extensive water use, and synthetic fertilizers, contributing to climate change while struggling to cope with its impacts. The growing global population only increases the demand for food, further stressing an already vulnerable system. As traditional agriculture faces challenges like water scarcity, soil degradation, and unpredictable weather patterns, the need for a more resilient, resource-efficient, and sustainable farming approach is becoming critical. Hydroponic systems, while offering a great alternative, often lack sufficient automation, making it difficult for users to effectively monitor and adjust key variables like water temperature, pH, and lighting. This leads to inconsistent growing conditions, inefficient resource use, and lower crop yields. Additionally, many existing hydroponic systems do not integrate seamlessly with mobile applications, limiting their accessibility and ease of use. The need for an intuitive, automated, and user-friendly solution is clear—one that enables users to efficiently manage their hydroponic gardens, optimize plant growth, and reduce resource consumption, therefore addressing both agricultural and technological challenges.

Hydroponics as a Solution

Hydroponics, a soilless farming method that delivers nutrients through water-based solutions, offers a viable alternative to traditional agriculture. By eliminating the need for soil, hydroponic systems can be implemented in controlled environments such as greenhouses and vertical farms, optimizing resource efficiency and reducing dependence on climate conditions.

Objectives of Hydroponics

1. **Mitigate Environmental Impact:** Reduce water consumption, minimize land use, and eliminate the need for chemical fertilizers, thereby lowering agriculture's carbon footprint.
2. **Enhance Food Security:** Ensure a stable and resilient food supply by cultivating crops in controlled environments less susceptible to climate variability.
3. **Promote Sustainable Practices:** Transition from traditional farming to hydroponic methods that can be implemented in urban settings, reducing the strain on natural ecosystems and conserving biodiversity.

Key Benefits of Hydroponics

1. **Water Efficiency:** Uses up to 90% less water than traditional farming by recirculating nutrient solutions, making it ideal for regions facing water scarcity.
2. **Reduces Dependence on Major Food Suppliers:** Growing plants at home decreases reliance on store-bought produce, alleviates strain on the food supply, and in turn reduces environmental impact.
3. **Climate Resilience:** Controlled indoor environments shield crops from extreme weather, temperature fluctuations, and droughts, ensuring year-round production.
4. **Reduced Land Dependence:** Vertical and urban farming solutions maximize yield per square foot, reducing pressure on arable land that is increasingly degraded due to climate change.
5. **Higher Yields and Faster Growth:** Optimized nutrient delivery and environmental conditions promote faster growth and greater crop productivity.
6. **Pest and Disease Reduction:** Indoor hydroponic systems limit exposure to pests and soil-borne diseases, reducing the need for chemical pesticides and promoting cleaner food production.

Challenges of Hydroponics

Despite its advantages, hydroponic farming presents challenges that must be addressed for widespread adoption:

1. **High Initial Costs:** Setting up hydroponic infrastructure requires significant investment in technology and controlled environments.
2. **Technical Expertise:** Managing nutrient solutions, monitoring environmental conditions, and troubleshooting system failures require specialized knowledge.
3. **Energy Consumption:** Climate-controlled hydroponic farms rely on artificial lighting, pumps, and environmental controls, increasing energy use. Renewable energy integration can mitigate this impact.

Conclusion

Climate change threatens global food security by increasing temperature extremes, water scarcity, and soil degradation. Hydroponics presents a sustainable solution by improving water efficiency, reducing reliance on fertile land, and providing climate-resilient food production. While challenges exist, advancements in technology and sustainable energy solutions can enhance hydroponic systems, making them a key strategy for future food security in a changing world.

Literature Review

Why Hydroponics Are The Best Farming Alternative

Sources: [Infarm- Berlin. Germany.](#), [AppHarvest- Kentucky. USA.](#), [AeroFarms- New Jersey. USA](#)

Companies such as Infarm, based in Berlin, Germany, have developed an innovative vertical farming approach to address critical environmental challenges posed by traditional agriculture. By focusing on sustainability and efficiency, Infarm aims to reduce the negative impacts of global food production, including greenhouse gas emissions, land use changes, water scarcity, pesticide use, biodiversity loss, and climate vulnerability.

Greenhouse Gas Emissions and Climate Impact:

The global food system is responsible for a third of human-induced greenhouse gas (GHG) emissions, primarily due to land clearance and transportation. Infarm reduces transportation emissions by growing food in urban areas, minimizing food miles. Additionally, their approach minimizes land conversion, reducing carbon emissions from deforestation. Infarm also tackles food waste by focusing on reducing methane emissions through more efficient packaging and consumption.

Land Use and Agricultural Efficiency:

With half of the world's habitable land already dedicated to agriculture, Infarm's vertical farming method offers a more space-efficient solution, claiming up to 100 times greater efficiency than conventional farming. This allows the company to conserve land, having saved over 200,000 square meters of space and reduces the need for further deforestation, helping preserve natural habitats.

Water Scarcity and Conservation:

Agriculture accounts for 70% of global freshwater use, a significant issue given the growing water scarcity worldwide. Infarm's closed-loop system uses 95% less water compared to traditional farming. By recycling and recapturing water, the system drastically reduces water consumption, addressing the global challenge of water scarcity, especially in regions prone to drought.

Chemical Pesticides and Environmental Impact:

Conventional farming heavily relies on chemical pesticides, leading to environmental pollution and harm to aquatic ecosystems. Infarm avoids chemical pesticides entirely, offering an eco-friendly alternative that reduces water contamination and protects biodiversity. This pesticide-free approach contributes to healthier ecosystems and reduces the risk of toxic algal blooms in freshwater systems.

Biodiversity Loss and Conservation:

Agriculture is a major driver of biodiversity loss, with large-scale land use changes threatening wildlife and pollinators. Infarm's efficient farming system reduces the amount of land used, preserving natural habitats and preventing further ecosystem destruction. By avoiding chemical inputs, Infarm helps protect local wildlife and supports biodiversity conservation, with the potential for land to be repurposed for rewilding or conservation.

Climate Resilience and Food Security:

Climate change has already impacted agricultural productivity, and global food systems remain vulnerable to environmental shocks. Infarm builds climate resilience by localizing food production within urban areas, reducing reliance on external environmental factors. By controlling growing conditions in vertical farms, Infarm ensures stable crop production, even in the face of unpredictable weather events, and reduces the vulnerability of supply chains to climate-related disruptions.

Conclusion:

Infarm's urban farming technology presents a sustainable and efficient solution to the many environmental challenges of modern agriculture. Through vertical farming, water conservation, and pesticide-free practices, Infarm addresses greenhouse gas emissions, land use, water scarcity, and biodiversity loss while building resilience against climate change. Infarm's model offers a promising pathway toward a more sustainable and resilient food system, especially as the world faces the dual challenges of population growth and climate change.

Stakeholder Requirements

Clients:

1. **Urban Farmers and Agricultural Entrepreneurs:** These clients are seeking efficient, sustainable farming methods to cope with limited space and water resources in urban environments. Hydroponics offers them a viable solution to grow fresh produce year-round, reducing dependency on traditional farming methods.
2. **Rural Farmers Affected by Climate Change:** With increasingly unpredictable weather patterns, these farmers are looking for resilient agricultural methods. Hydroponics provides an alternative that can thrive despite adverse climatic conditions, ensuring a stable food supply.
3. **Small Families and Home Chefs (Mrs. Borken):** These families are interested in hydroponics as a way to produce fresh, healthy food at home. With limited space and resources, they seek an easy-to-maintain, cost-effective system that allows them to grow vegetables and herbs for daily use, reducing grocery bills and ensuring food security.

Stakeholders:

1. **Government and Policymakers:** They are interested in promoting sustainable agricultural practices to achieve climate goals and enhance national food security. Policies supporting hydroponics could reduce the agricultural sector's carbon footprint and water usage.
2. **Environmental Organizations:** Focused on reducing the environmental impact of agriculture, these stakeholders are advocates for methods that conserve water, reduce land use, and minimize chemical runoff. Hydroponics aligns with their mission to protect ecosystems and promote sustainable practices.
3. **Investors and Financial Institutions:** They see hydroponics as a promising investment opportunity, driven by the growing demand for sustainable food production technologies. These stakeholders are interested in funding scalable projects that offer both environmental and financial returns.
4. **Academic and Research Institutions:** These institutions are engaged in studying the technological, economic, and social implications of hydroponics. They play a critical role in innovating and optimizing systems for broader adoption and effectiveness.

Client Background

My client, Mrs. Borken, is an Environmental Science teacher at Greens Farms Academy and a passionate home chef with a strong background in sustainability and education. She integrates environmental awareness into her teaching and daily life, making her an ideal candidate for incorporating a hydroponic system into both her professional and personal environments. As an educator, she seeks to use hydroponics to demonstrate sustainable agriculture methods to her students. At home, she values fresh, locally grown ingredients and wants a system that provides a reliable source of herbs and vegetables year-round. Currently, she may rely on traditional gardening or store-bought produce, which can be impacted by seasonal availability and food miles. A hydroponic system would offer a more controlled, sustainable way to grow fresh ingredients, reducing her dependence on external sources. This project would not only support her teaching goals but also align with her commitment to sustainability, allowing her to model eco-friendly practices in both the classroom and her home kitchen.

Conclusion:

The integration of hydroponics into agricultural practices is essential to address the dual challenges of climate change and food security. By reducing the environmental impact of food production and providing a sustainable alternative to traditional farming, hydroponics offers a pathway to a more resilient and sustainable agricultural future. The adoption of this technology, supported by policy incentives, research, and education, can play a crucial role in mitigating the effects of climate change on agriculture and ensuring global food security.

Feasibility Study Report

Executive Summary:

1. From this feasibility study, I realized that it will be much cheaper to buy and assemble the frame from a company—at least \$140 cheaper. However, this loses some of the originality aspects of my project and it won't be as “handmade.” It is still the exact same thing as I would be doing and I can still add the pump, lights, and heater, but I just need to assemble the structure instead of cut and create it myself.
2. It will be difficult to get parts of specific dimensions and assemble them together
3. There are so many types of structures out there: some that grow a lot more vertically, but have fewer yields and no space for the heat lamps, and then the PVC pipes that take up a little more space, wider and still provide height optimization.
4. Scaling this project into a company is not as difficult on the design front as it is on the business front, and cutting expenditures and making profits
5. There's a lot of competition out there, each with a different mission statement. The larger and more successful companies have bigger farms and more automation technology that makes it easy to monitor their plants and control their environment.

Introduction:

Goals:

1. Optimize space by going vertical, while also allowing plants to grow tall and big.
Mechanisms that will allow you to move the beds/shelves of other plants up or out can be very useful.
 - a. I want to bring the space optimization techniques large hydroponic companies successfully incorporate into smaller farms that are not designated to generate revenue or don't have much surplus to spend on fancy technology, such as that allowing vertical hydroponics to be as successful as they are.
2. Have proper water flow and a warm, nutrient-enriched solution. Lights that mimic a warm environment on the bottom of the shelves, providing heat and light to the plants below, is important.
3. Research digs deep into why this is important, how my project solves issues posed by climate change, and how my hydroponics are different
4. Research addresses how my project can be scaled to be applied in the global economy
5. By the end of this project, I hope to have a fully functioning hydroponic system that accurately addresses my client's needs

Data Collection:

1. This is a list of the 24 top-performing companies in hydroponic farming with links to their company pages. Their websites include so much information about their company, process, values, sustainability reports, information on why they do what they do, how they combat climate change, and more. It is a super helpful resource and has almost all the information I need on the "why" of my project
 - a. [24 Top-Performing Companies in Hydroponic Farming](#)
2. This link answers the "how," or how am I going to build my project, and explains how it relates to the why
 - a. <https://www.youtube.com/watch?v=x4eKmNx6l8>

Market Analysis and Product Potential:

- As large companies make more profits and can spend more on technology, they have great facilities with dozens of workers and scientists working together to cultivate crops as efficiently as possible. From the smaller-scale home designs I've seen, they are always lacking something. Most of the time it is inefficient space optimization.
 - I want to incorporate the space optimization techniques large hydroponic companies use into smaller farms that are not designated to generate revenue or don't have much surplus to spend on fancy technology such as that allows vertical hydroponics to be as successful as they are.
- Similarities with other products I've seen:
 - Has a pump and a container for water that sources the water for the system.
(most I've seen) Controls the PH level of the water, (only some) have grow lights above the plants
- What sets my project apart:
 - I'll include a water temperature controller (water heater) and make sure the PH level of the water is managed correctly
 - My system includes growing lights placed above the plants
 - The main difference is that my project has an app where the user can control and monitor the heat and lights
- Customer demands:
 - My system optimizes space to grow as much as she can
 - She can grow plants all year round
 - Plants are safe from natural events that can damage crops
 - The project saves water and is sustainable

Technical Feasibility:

- To build the system, I am relying on materials that I should have easy access to. It might be more difficult to get PVC pipes of specific dimensions or to create the water heater, but it should not pose a big problem. I will use Amazon to get as many materials as I can, and for products I can not get on Amazon, I will look at gardening stores and online stores.
- What might pose a challenge is having the machinery to create the frame and my knowledge of how to use it, but I will work closely with my advisors and watch videos to teach myself what I need to do and how.

- Also, it will be very challenging to create an app that can control the light and heat of the system when you are not actively watching it, but this is something that I would like to try to do as it will differentiate my project from others.

Financial Feasibility:

- Given a budget of \$500, I can easily build my prototype. The most expensive individual parts will be the lights and the structure as their technologies are more expensive. But then I also need to buy the PVC pipes, tubes, plants, and possibly drill bits.
- If this project were to be scaled to a company, there would be a lot more factors and costs that I need to consider such as property (buying, taxes, maintenance, etc), paying employees, capital expenditures, and also balancing expenses and profits to name a few.
- Project resource documentation:
 - <https://docs.google.com/document/d/16CZPMwYgPCZF477YQeIFeIA8kaXh8zBe6Z65hBf7wGO/edit?usp=sharing>
- I estimate that my project will cost roughly \$350 to \$500 depending on which products I get off Amazon, with the bulk of this cost being the physical frame (can either be \$100 or up to \$250).

Organizational and Operational Feasibility:

- For my project itself, my hydroponics will need to be semi-maintained by me. Meaning, I will not need other people or any help like that as I can easily maintain the system myself, but the goal is to have the system automated.
 - The pump will turn on and off on itself depending on the program I write and how long I set its active duration to. Similarly, the lights will be programmed to turn on and off, providing 14 hours of light, then off for the rest of the 24-hour cycle. Lastly, the heater will automatically turn on if the water temperature falls below a certain temperature.
 - I will need to manage the PH level, water level, and cultivation of my plants.
- In a scaled environment, I would have employees assisting with the cultivation and maintenance of the crops and system. I would also have an executive board and people who help with the business, marketing, financials, etc... behind the scenes.

Risk Assessment:

- Areas of risk include the financial feasibility aspect and how well this project can be scaled to compete with competitors.
 - Given only \$500, I'm sure I can create a very great and successful hydroponic system that accurately portrays the findings from my research, however, to stay under this cost limit, I might need to order parts that I might be able to make myself in order to save money.
 - One challenging aspect of this project is making it as original as possible while making sure it is functional and cost-effective.
- There is not much that sets companies apart from each other in the hydroponics industry other than the crops they grow and how they grow these crops.

Resource Requirements:

Essential:

- PVC Pipes for plants and frame
 - Plastic tubes for water
- Hydroponic Seedling pots
- Water and nutrients
- Water pump
- PH reader
 - PH up and down

What my project adds:

- Heat lamps
- Thermometer (made with Raspberry Pi)
- Water heater

What I hope to include in my project:

- An app that allows you to control the temperature, and the lighting from your device

Conclusion and Recommendations:

Important information and things to remember:

- It will be \$140 cheaper to buy the frame off Amazon rather than to build it from scratch. It's still exactly what I would do, but saves me the money I can allocate elsewhere
- Try to create an app that can allow the user to control and monitor the status of the hydroponics and its environment, and allow the user to add data logs. This is definitely something different than anything I have seen or researched so far
- It's hard to think of this project from a marketing perspective because it is in its early stages as I have not researched the hydroponic market from a value perspective, as it is not very relevant to my research (only to see what is out there).
- Goals:
 - To be able to scale this to a company perspective, think more about how the design would work and worry less about how the company would function from a business perspective
 - I want to bring space optimization technique,s large hydroponic companies successfully incorporate into smaller farms that don't have much surplus to spend on fancy technology
 - To have proper water flow, a warm, nutrient-enriched solution, and lights that mimic a warm environment on the bottom of the shelves provide heat and light to the plants below

Reflection:

After pushing this study off for quite some time as it was a lot to digest at the time, it has been really helpful to think about my project from all these different angles such as a financial, technical, and market perspective. This study really helped me organize my ideas and what I wanted to do with my project, and as I was thinking about how I could make my project different from other hydroponics I've seen, I was inspired to design an app that the user can connect the hydroponics to. I'm more confident in my work and I am certain that my hydroponic system will fully function. I know I need to give myself enough time for things and mechanics to go wrong and adjust.

I learned that once I get started on something I am deeply invested in, it is very hard to stop. When I am working, my thoughts always spark ideas I then try to chase, and then I actually go on physical tangents researching 3 different things at once. This is why it has been really helpful to document my progress so I can always go back and see where I left off or what I have already done.

I can use this skill in the future for any big research projects that have many components to them, so I stay organized and don't get lost in my resources and research. I can also use this study to remind myself there are multiple areas to think of when you are starting a company, or pitching an idea to others.

Success Criteria

Project Focus:

1. **Technological Innovation Over Plant Cultivation:** The project emphasizes the development and optimization of the hydroponic system's mechanical and technological components. Key areas include precise water circulation, efficient nutrient delivery, and control over the growing environment that create ideal growing conditions, ensuring the system's functionality and sustainability is prioritized over traditional plant-growing methods.
2. **Climate-Resilient Hydroponic Solutions:** The hydroponic system addresses specific climate change challenges, such as managing temperature fluctuations, conserving water in drought-prone areas, and withstanding extreme weather. The system's design ensures stable crop production regardless of external environmental conditions.

Technical Components:

1. **Water Temperature Control:** The system consistently maintains water temperature between 68°F and 72°F to ensure optimal plant growth and system efficiency.
2. **pH Level Stability:** The water's pH level remains stable within the range of 6.0 to 6.5, suitable for most hydroponic crops and essential for nutrient absorption.
3. **Light and Rest Cycle:** Plants receive 14-16 hours of artificial light per day, followed by 8-10 hours of darkness, mimicking natural light cycles to promote healthy growth.
4. **Synchronized Water Pump:** The water pump operates in sync with the lighting schedule to ensure efficient water circulation and nutrient distribution during active growth periods.

Technological Integration:

1. **Headless Raspberry Pi Operation:** The Raspberry Pi operates in a headless setup, without the need for a monitor or keyboard, enhancing the system's efficiency and reducing hardware requirements.
2. **Scalable Prototype:** The prototype is designed to be a clear, functional representation of a scalable hydroponic system, demonstrating the potential for expansion to larger, commercial-scale projects.

User Interaction and Data Accessibility:

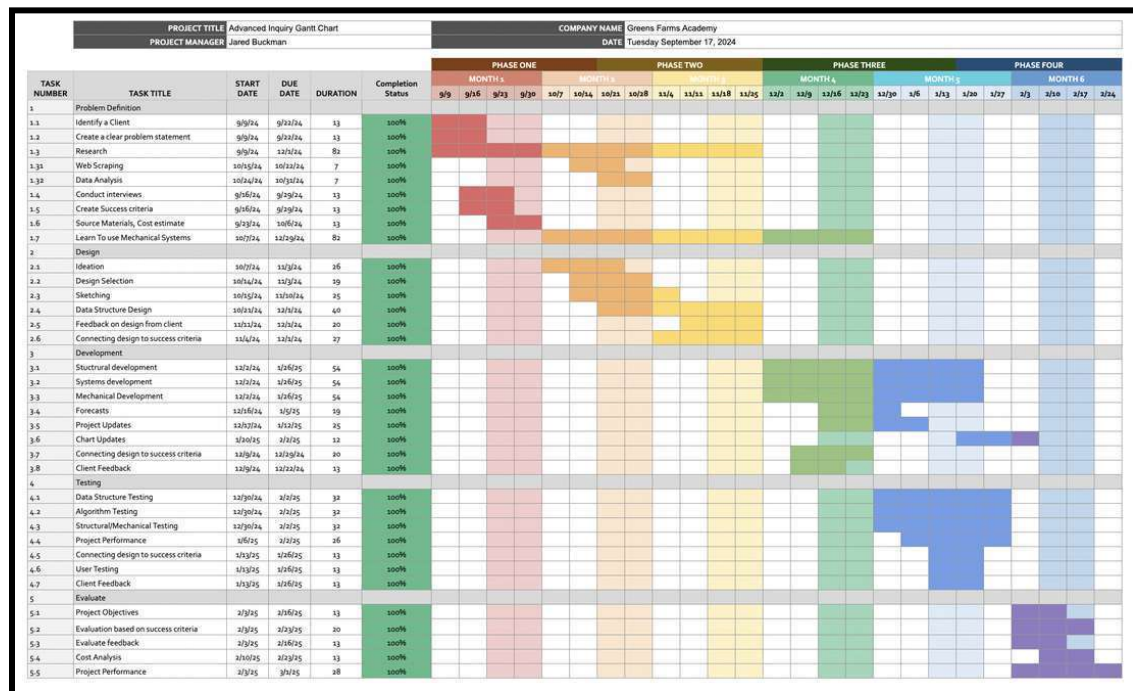
1. **User-Friendly Interface:** The system includes an intuitive user interface that allows users to monitor and control the hydroponic setup easily, enhancing usability and engagement.
2. **Real-Time Data Display:** A dedicated app provides real-time data on key parameters such as water temperature, pH levels, and light cycles, enabling users to make informed decisions and monitor system performance effectively.

Planning

Overview

In this section, I will detail the project timeline, resources, and specific tasks required to implement the hydroponic system successfully. This includes determining key milestones and breaking down the process into manageable phases, including the initial brainstorm. I will assess the skills needed to work on this project and identify areas for growth. A focus will be placed on prioritizing tasks and setting clear deadlines to ensure the system's components—sensors, automation, mobile application, and physical structure—are developed before adding nonessential components. The Planning phase serves as a roadmap, ensuring the project stays on track and meets all established success criteria.

Gantt Chart



See Appendix pg 107.

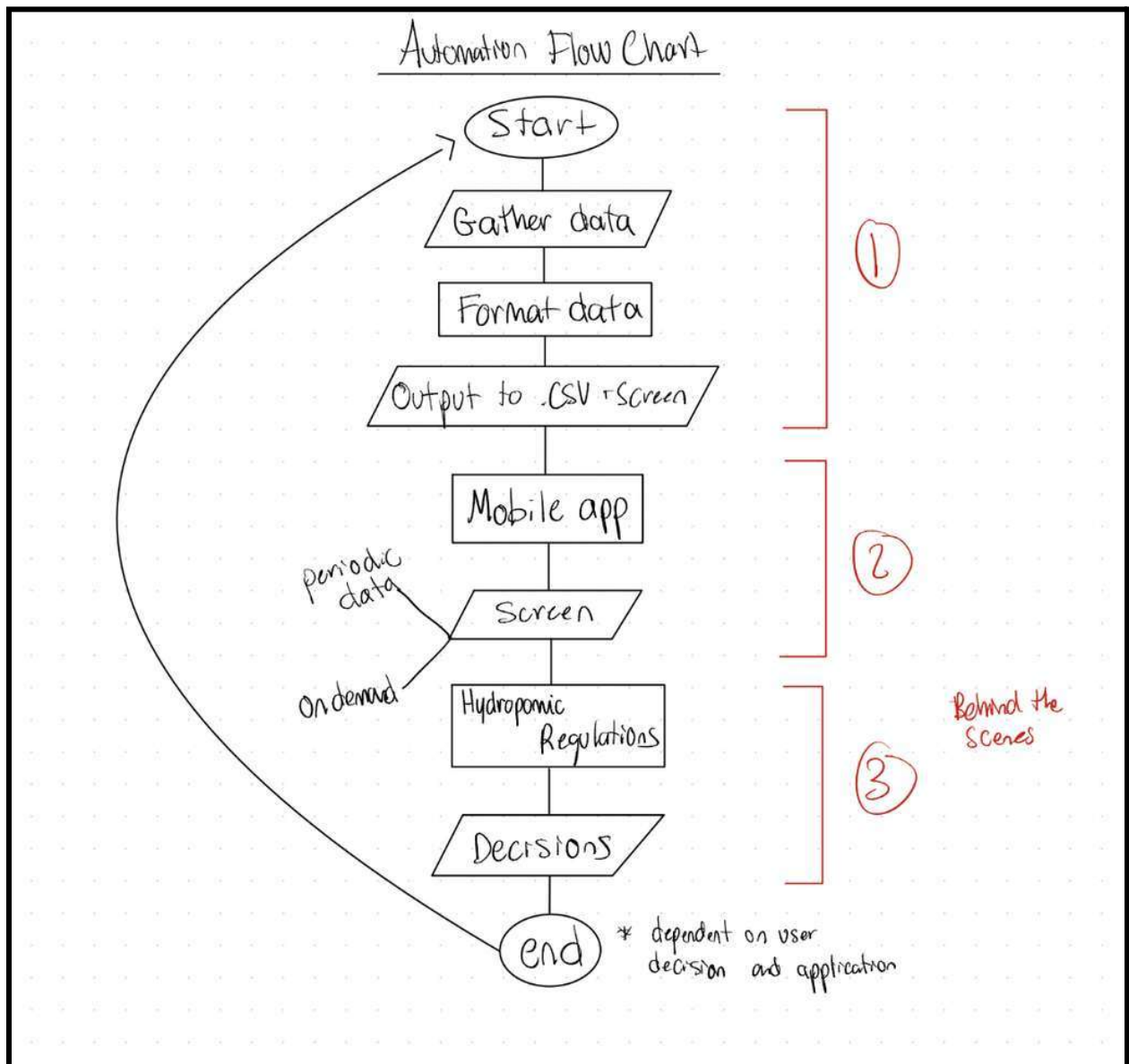
A Gantt chart is a visual tool that helps me track the timeline of my project. It displays tasks along a timeline, showing their start and end dates, and helps me identify which tasks depend on each other. A Gantt chart is essential for organizing and scheduling key milestones, like designing the system, assembling components, testing sensors, and implementing automation.

By visualizing these tasks, I can ensure that everything is completed on time, stay on track with my deadlines, and easily spot potential delays. This is especially important as the project follows the engineering life cycle, from ideation and design to testing and evaluation. By updating the chart regularly, I can see what elements are completed, and what I need to finalize.

Initial Brainstorming

Before stepping into my first class, I knew I wanted to design a system that would help people cultivate plants despite the challenges posed by climate change and difficult growing conditions—whether for large-scale companies or families planting in their gardens. Initially, I proposed a robotic sprinkler irrigation system to my client, Mrs. Borken. However, after discussing how plants absorb nutrients, we identified a major flaw: as plants grow leaves, they can block the soil, preventing the roots from receiving adequate water. This inefficiency leads to significant water waste, so we pivoted to a system that delivers water directly to the roots while controlling the nutrient supply. We also took environmental factors into account. High tunnels, which extend the growing season by allowing sunlight to warm and humidify the air to create a controlled environment, provide an effective alternative to traditional greenhouses at a lower cost. By placing my hydroponic system inside a greenhouse, I could regulate not only water and nutrient flow but also environmental conditions, ensuring optimal plant growth.

Flow chart



See Appendix pg 128.

This Automation Flow Chart outlines the process of collecting, formatting, and utilizing sensor data within the hydroponic system. The process starts with gathering data from sensors, which is then formatted and outputted as a CSV file and displayed on the user's device. The system provides both periodic data updates and on-demand data retrieval, allowing users to access real-time hydroponic conditions.. Finally, decisions are made based on user input or automation rules before the process loops back, ensuring continuous monitoring and regulation.

Skills Development Table

Skill	Details	Focus/Purpose	Progress	Reflection
Web Scraping	I used Raspberry Pi on the desktop to scrape information off a website of our choice and create a new file with that information so we could save it.	Learn how to web scrape and continue our research. Provides us with another way to conduct our research	I completed this activity, and my information and code can be found here. Web Scraping	The most important skill I learned was how to get the class information by clicking on the text I wanted to scrape and inspecting the page. It was just as helpful learning the code that makes this process function. I think this can definitely help me with my research. If there is a lot of information on a page that is hard to access with selecting all and copying the information. Lastly, it would be nice to know exactly why some websites don't permit us to web scrape and how they block it.
Data analysis	I used Google Colab, a Google Python program, and followed a video to learn about the Panda library and how to use it for data analysis. I created a data table that easily stores and organizes information	Learn how to use the Panda library and create a data frame that stores and displays data in a 2D array.	After having trouble with the program not running, it finally worked after using the CSV file format. We did not fully finish this first class but I am confident we will next class.	The most important skill I learned was how to use certain commands from the Panda library to create this data frame and that Google Colab is a great program for this. I think this could be a very helpful tool to organize my information as my current process of organizing my research is beginning to get a little unorganized. I think I need to learn more about the Panda library and other commands that could simplify the process of importing data to the colab folder.
Research	Conducted research by looking at web pages, videos, company blogs, and charts. Created a Google doc to organize all of my information and hyperlinks so I can easily access the sites for more information.	The purpose of this is to do background research on why my project is important and how I can make the physical prototype.	I have found at least 20 sources of information ranging from videos on examples of types of hydroponic systems I can make and web pages about the impacts of climate change on cultivation. Source Docum...	The most important skill I learned is how to organize all the information I am acquiring. This allowed me to easily access all my links and pages where there is more information than I cited in my source document. My next steps are to keep doing what I'm doing and get more information.
Data Analysis using spreadsheets	I used Google Sheets to conduct data analysis and order data according to a conditional. In my case, each numerical input was assigned a color based on positive or negative, (red or green), and the	Learn to use Google Sheets to conduct data analysis and organize data scraped in previous exercises.	I finished this document and also created a chart by using the data. This is a link to the data analysis sheet: Climate Chang...	This exercise was actually pretty helpful and gave me something to visualize my data with. I can also use the chart and take screenshots of the color-coded data to use as graphs for my presentation and write-up.

	shade of that color was determined by how close it was to the max or minimum value.			
Using the Raspberry Pi	I used the Raspberry Pi to connect sensors and display data in a format I could see and understand, then write this data into a file	This central part of my project allows me to automate my hydroponic system where the user can control and monitor the system from a distance	Very ongoing; this is going to take through January as it is the biggest and most time-consuming aspect of my project.	The biggest skill so far for me has been patience. There are many things that are confusing, difficult, and some things I just don't know. I need patience as I work through all of this and problem-solve issues.
Using the Arduino	Similar to the role Raspberry pi plays in my project, I used the Arduino to program my sensors, then plugged the board into the Raspberry Pi so all my sensors are controlled by one unit.	Arduino also lets me automate my hydroponic system, and it is very easy to work with.	Like the Raspberry Pi, I am still in progress with coding and programming my sensors with Arduino	I learned how to use Arduino circuit boards and sensors, and how to connect the Arduino to the computer, and Raspberry Pi each has the Arduino IDE. I also learned Arduino is a great source for finding code, sensors, and projects.

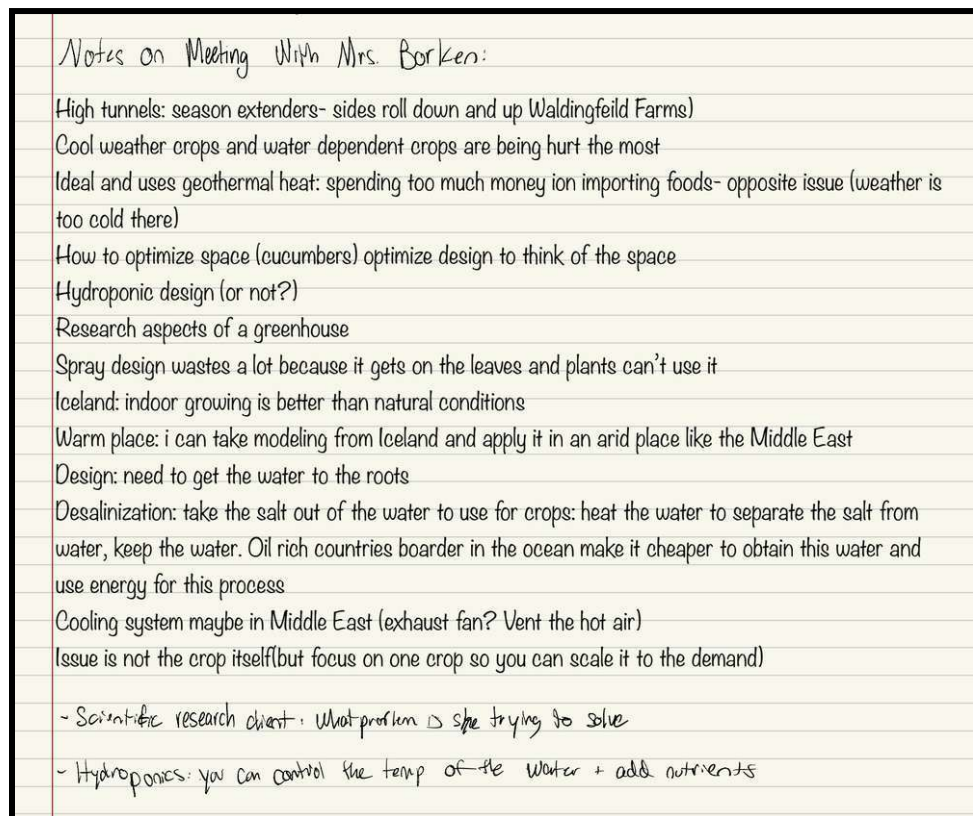
Real-World Impact

Other than the benefits of hydroponics as opposed to traditional farming, key impacts of growing hydroponically include:

1. **Sustainable Food Production:** Hydroponics allows for year-round cultivation, unaffected by seasonal changes or soil quality.
2. **Scalability & Accessibility:** The system can be scaled for home use, commercial farming, or community projects, making it accessible for households, schools, and urban areas with limited farming space. Hydroponics can also be used in resource-scarce regions, ensuring a reliable food supply.
3. **Integration with Smart Technology:** The incorporation of sensor-based automation ensures optimal conditions for plant growth while reducing labor and monitoring efforts. Integrating technology makes hydroponic farming more efficient and user-friendly.
4. **Economic & Social Benefits:** Encourages local food production, reduces dependency on imported produce, and lowers carbon footprints from transportation. Can be implemented in schools for educational purposes, teaching students about sustainable agriculture and STEAM concepts.

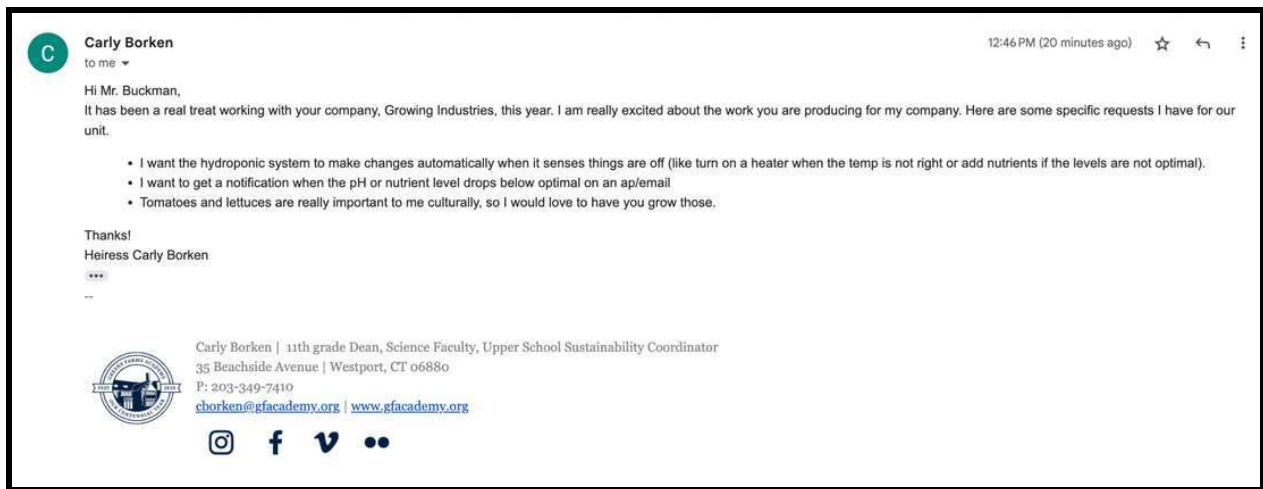
Communication and Feedback

Effective communication and feedback from the client are essential to ensuring the hydroponic system meets their needs and functions as intended. Ongoing input helps refine the design, optimize performance, and enhance usability. It also allows for the establishment of clear success criteria, ensuring the final product aligns with client expectations. By involving the client throughout the development process, potential issues can be identified and addressed early, leading to a more reliable and efficient system.



See Appendix pg 136.

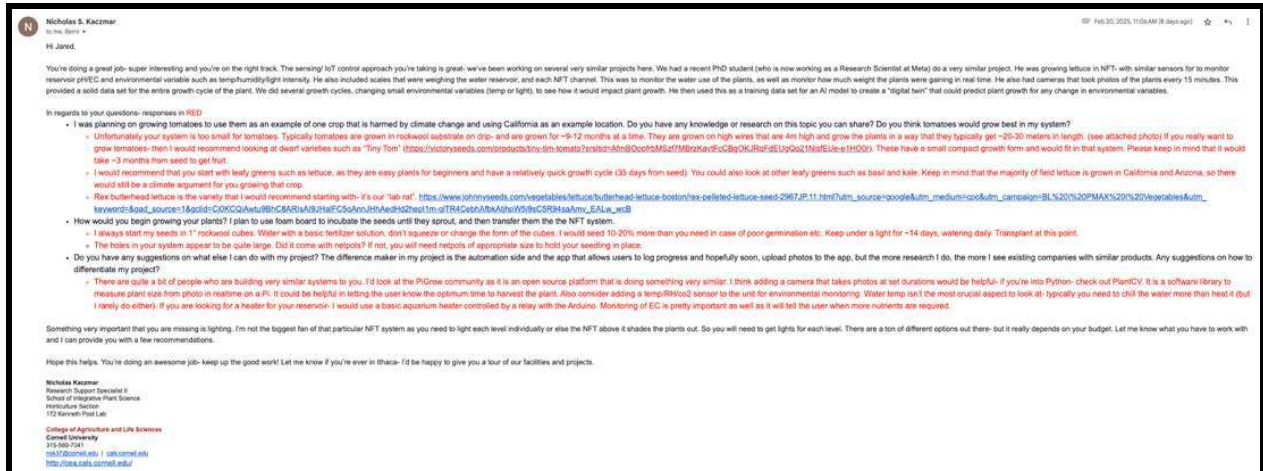
This image of my notes is from my first meeting with Mrs. Borken. Here, we discussed the idea of sprinkler irrigation, the benefits of high tunnels, and examples of places this would be beneficial. The initial goal of my project was to grow sustainably and maximize crop yield, and to do this, the system needed to maximize how much nutrients the roots received. As stated before, sprinkler irrigation had many flaws, so hydroponics became the solution.



See Appendix pg 137.

This communication with my client allowed me to add to my success criteria and give me goals for the automation of my system and functionality of the application.

I have also been in contact with professionals in the industry, such as Nicholas S. Kaczmar. Kaczmar is a Research Support Specialist at Cornell University's Controlled Environment Agriculture program, focusing on hydroponics and aquaponics within the Horticulture Section of the School of Integrative Plant Science at the College of Agriculture and Life Sciences. His research includes collaborations on hydroponic systems, such as the Kratky method, aiming to enhance sustainable food production. He has collaborated on research projects that explore nutrient management, water quality, and innovative greenhouse technologies. Additionally, Kaczmar contributes to publications and outreach efforts aimed at advancing controlled environment agriculture for both commercial and academic applications.



See Appendix pg 137.

This photo shows an email response from Kaczmar to me, giving me suggestions on how I can take my project to the next level, and tips on what tomato varieties have grown best hydroponically.

Research

Overview

In this section, I will review the research I have conducted in preparation of my project, including existing knowledge and technologies relevant to the development of my hydroponic system. By researching the importance of hydroponics and why they are used, I can better understand essential features that will make the system a success. This includes reviewing over 20 sources regarding existing hydroponic systems, available sensor technologies, and software solutions for automation and mobile applications. I will also investigate best practices in system integration and data management. By understanding what's already out there, I can make better decisions and ensure the project is set up for success.

Essential Findings

About Hydroponics (background)

Hydroponics is a method of growing plants without soil, utilizing a nutrient-rich water solution instead. In traditional farming, soil provides plants with water and essential nutrients, but in hydroponic systems, this role is fulfilled by alternative growing mediums such as Rockwool, vermiculite, perlite, or expanded clay. Some systems even allow plants to be grown directly in water or seedling pods without any solid medium. This method enables controlled nutrient delivery, improving plant growth efficiency and yield.

There are several types of hydroponic systems, each offering different approaches to water and nutrient distribution. The Nutrient Film Technique (NFT) circulates a continuous flow of nutrient-enriched water through sloped channels, allowing plant roots to absorb nutrients while remaining oxygenated. Wick systems rely on capillary action to transport water from a reservoir to the plant roots, making them a passive and low-maintenance option. Ebb and flow (or flood and drain) systems periodically flood the growing medium with nutrient-rich water before draining it back into the reservoir. Deep Water Culture (DWC) suspends plant roots directly in oxygenated water, while drip systems deliver precise amounts of nutrients via drip lines, which can be adjusted based on plant needs. Aeroponic systems suspend plants in the

air and mist their roots with nutrient solutions, maximizing oxygen exposure and promoting rapid growth.

Hydroponics offers numerous advantages, making it an attractive alternative to traditional farming. It allows for year-round cultivation, making it possible to grow plants in urban environments, greenhouses, and even space. The system uses significantly less water than soil-based farming, as water is recirculated rather than lost through runoff or evaporation. Plants can be grown closer together, maximizing yield in limited spaces. Additionally, hydroponics eliminates the need for weeding, reduces the risk of soil-borne diseases, and allows for better control over nutrient levels, resulting in healthier and faster-growing plants.

There are some challenges associated with hydroponic farming. The initial setup costs, especially for commercial operations, can be high, making it difficult for small-scale farmers to invest in the technology. Hydroponic systems require knowledge and expertise to maintain optimal growing conditions, and any disruptions, such as power outages or water loss, can quickly impact plant health. Additionally, while hydroponics reduces soil-borne diseases, it does not eliminate all risks—diseases can spread rapidly in a shared water system.

Most plants can adapt to hydroponic growth, but some varieties are less suited to this method. Plants that require extensive space to climb or sprawl, such as vines and large trees, are difficult to cultivate hydroponically. Root crops, like potatoes, carrots, and onions, also pose challenges because they require more support that soil provides and water does not. However, leafy greens, herbs, and many fruiting plants thrive in hydroponic systems.

Hydroponics is categorized into water-culture and medium-culture systems. Water-culture systems, such as NFT and aeroponics, do not use a solid medium, relying entirely on nutrient-rich water or mist. Medium-culture systems, like drip and ebb-and-flow setups, use solid substrates such as sand, perlite, or Rockwool to support root structures. These systems can also be classified as open or closed—open systems dispose of excess nutrient solution, while closed systems recycle and reuse it.

The nutritional quality of hydroponically grown produce compared to soil-grown crops is still under study. Research suggests that hydroponically grown strawberries and raspberries may contain higher levels of vitamins and polyphenolic compounds, although they may have lower sugar content. Since hydroponics allows for precise control over growing conditions, there is

potential for optimizing nutrient content, making hydroponically grown food an appealing option for sustainable agriculture.

Looking ahead, hydroponics has significant potential to address food security challenges in arid and urban environments. Advancements in desalination technology may allow coastal and desert regions to implement hydroponic farming with treated seawater. Additionally, hydroponics has already been integrated into space exploration, with astronauts growing lettuce aboard the International Space Station. Ongoing research into hydroponic cultivation methods for space travel could pave the way for sustainable food production during long-duration missions, further demonstrating the versatility and promise of this innovative agricultural approach.

Current Initiatives

Infarm, based in Berlin, Germany, addresses several major environmental challenges associated with traditional agriculture by using vertical farming. The global food system contributes significantly to greenhouse gas emissions, mainly through land clearance, transportation, and food waste. Infarm reduces its carbon footprint by growing food locally in urban areas, minimizing transportation emissions, and using sustainable packaging. The company also tackles land use inefficiencies by implementing vertical farming, which is up to 100 times more space-efficient than conventional methods, preserving land that would otherwise be cleared for agriculture. Water scarcity is another concern, as traditional farming consumes 70% of global freshwater withdrawals. Infarm's closed-loop system conserves 95% more water than traditional farming by recycling and recapturing evaporated water. Additionally, Infarm eliminates the use of chemical pesticides, helping to prevent pollution in oceans and freshwater ecosystems. By reducing deforestation, water consumption, and pollution, Infarm also mitigates biodiversity loss. Lastly, the company strengthens food security by creating a climate-resilient food system that operates independently of external factors like extreme weather and supply chain disruptions.

AppHarvest, located in Kentucky, USA, focuses on sustainable farming solutions to address the increasing global food demand as the population approaches 10 billion by 2050. Climate change, soil degradation, and resource depletion pose significant threats to traditional agriculture. Soil erosion, for example, occurs at a rate 100 times faster than soil formation due to conventional farming practices. To combat these challenges, AppHarvest employs hybrid

lighting technology, which blends natural sunlight with energy-efficient LED lights that are 40% more efficient than conventional lighting. This system ensures optimal plant growth while conserving energy and reducing reliance on artificial light sources. By integrating sustainable solutions, AppHarvest works to maintain food production efficiency while minimizing its environmental impact.

AeroFarms, based in New Jersey, USA, is tackling some of the most pressing agricultural issues, including population growth, water scarcity, loss of arable land, excessive pesticide use, and supply chain emissions. By adopting advanced vertical farming techniques, AeroFarms conserves land and water while eliminating the need for harmful pesticides. Their controlled indoor farming environment ensures year-round food production with significantly lower resource consumption than traditional methods. Additionally, their localized production model reduces transportation-related emissions and strengthens the resilience of the food supply chain. Through these innovations, AeroFarms is addressing both environmental and logistical challenges in modern agriculture.

Optimal Environment for hydroponics

Unlike traditional soil-based cultivation, hydroponics requires careful monitoring of pH levels and electrical conductivity (EC) to ensure that plants receive the optimal balance of nutrients. The ideal pH range for hydroponic nutrient solutions is between 5.0 and 6.0, with the root environment ideally maintained between 6.0 and 6.5 for efficient nutrient absorption. Since soil naturally buffers pH fluctuations, hydroponic systems require manual adjustments using pH Up or pH Down solutions to maintain stability. Proper EC levels must also be regulated to prevent nutrient deficiencies or toxic buildup, which can negatively impact plant growth.

Water temperature plays a crucial role in hydroponic systems, affecting processes such as photosynthesis, respiration, root growth, and nutrient absorption. The optimal water temperature range is 20–25°C (68–77°F). If water temperatures fall below 20°C (68°F), plant metabolism slows down, potentially stunting growth or even killing plants. Conversely, temperatures above 25°C (77°F) increase the risk of root rot and deplete dissolved oxygen levels, which are vital for healthy root development. To regulate water temperature, growers can use water chillers, cooling fans, or ice packs. Aeration is another essential factor, as oxygen-rich water promotes faster growth and reduces the risk of disease. This can be

achieved by placing an air pump and air stones in the reservoir to continuously supply oxygen, typically operating for 15 minutes every hour.

Lighting is another critical aspect of hydroponic plant growth. Since hydroponic plants do not receive natural sunlight, artificial lighting must mimic natural daylight cycles. Most plants require 14–16 hours of light followed by 8–10 hours of darkness for proper growth and development. The type of grow light used depends on the plant's growth stage. LED grow lights are the most energy-efficient and versatile, offering full-spectrum lighting suitable for all plant stages. Fluorescent lights are ideal for seedlings and leafy greens due to their low heat output and efficiency. Metal Halide (MH) lights emit a blue/white spectrum, which is beneficial during vegetative growth, while High-Pressure Sodium (HPS) lights produce warm red light, ideal for flowering and fruiting plants.

By maintaining the correct pH balance, water temperature, oxygen levels, and lighting conditions, hydroponic systems create an optimal environment for plant growth. This method not only conserves water and space but also eliminates the need for chemical pesticides, making it a sustainable and efficient solution for modern agriculture.

Climate change impacts on agriculture

Climate change significantly impacts agriculture and food supply by altering growing seasons, increasing irrigation needs, and exacerbating air pollution, which affects crop health. Ozone pollution further reduces photosynthesis and plant growth, increasing susceptibility to diseases and pests, which disrupts ecosystems. These factors contribute to economic instability for farmers, especially as cash crop yields decline.

Farmers are adapting to these challenges by seeking controlled environments for cultivation to mitigate unpredictable climate effects. Rising temperatures and shifting precipitation patterns affect water availability, soil quality, and the spread of pests and diseases, leading to lower crop yields and quality. This, in turn, threatens food security, particularly in vulnerable communities.

Certain staple crops are especially endangered by climate change. Maize production faces sharp declines due to erratic rainfall, particularly affecting small-holder farmers. Wheat yields may increase in temperate regions but decline in hotter areas like India and Africa. Rice

production is highly vulnerable to temperature rises, with significant risks for millions dependent on it. Soybean farming, while benefiting in some regions, contributes to deforestation and is ultimately threatened by climate-induced water and heat stress. Potatoes, bananas, cocoa, and coffee also face production challenges due to changing weather patterns, water shortages, and pest expansion.

Climate disasters disproportionately affect countries already experiencing political and economic instability. Nations like Somalia, Syria, the Democratic Republic of Congo, and Yemen are struggling with climate-induced food insecurity, severe droughts, and extreme weather, exacerbating humanitarian crises. In Africa, droughts, floods, and conflicts further destabilize agriculture, leaving millions at risk of famine.

One example of a high risk nation is Saudi Arabia, whose agriculture sector relies heavily on irrigation, is particularly vulnerable to climate change. Rising temperatures and water scarcity are reducing yields of key crops like wheat and dates, leading to increased dependence on food imports and rising food prices. Additionally, pest populations are expected to grow, causing further crop losses. Addressing these challenges requires sustainable agricultural practices, improved water management, and climate-resilient crop varieties.

How hydroponics and growing smart are beneficial

Hydroponics, when integrated with smart farming technologies, presents a sustainable and efficient method of crop production. This system eliminates soil use and significantly reduces water consumption by delivering nutrients directly to plant roots. Smart farming utilizes IoT, sensors, and automation to continuously monitor and optimize growing conditions, ensuring year-round productivity regardless of environmental factors. These advancements lead to higher crop yields, faster growth, and minimal reliance on chemical inputs.

Furthermore, smart hydroponic farming supports regional food production, enhances food security, and promotes eco-friendly pest management while reducing agricultural waste. Emerging trends include AI-driven automation, remote cultivation, and data-driven decision-making, which contribute to greater efficiency and resilience in farming. With climate change threatening global food security, hydroponics and smart agriculture offer viable solutions to mitigate shortages, such as those currently affecting staple crops like onions, tomatoes, and potatoes.

Solution

Hydroponics offers a stable and sustainable solution for agriculture in the face of global warming by providing a controlled environment that is independent of climate fluctuations. Unlike traditional farming, which is vulnerable to droughts, extreme temperatures, and soil degradation, hydroponic systems optimize water usage and deliver nutrients directly to plant roots, ensuring consistent growth regardless of external conditions. Large-scale companies are already leveraging this technology to produce high-yield, pesticide-free crops with minimal resource waste, reducing their carbon footprint and increasing food security. At the same time, home chefs and small-scale growers can use compact hydroponic systems to cultivate fresh herbs and vegetables year-round, promoting local food production and reducing reliance on supply chains disrupted by climate change. By integrating hydroponic farming at all levels, from industrial agriculture to household kitchens, we can create a resilient, climate-smart food system that meets the demands of a changing world.

Design

Overview

In this phase, I will outline the design considerations for my Hydroponic System, which includes three main components: the sensors and hardware, the mobile application, and the physical structure. The design outlines the architecture and how each component interacts, ensuring a seamless and functional system. The aim is to establish a clear framework for how the system will operate and interact, focusing on its theoretical structure before the actual implementation begins.

Architecture & Design

System Architecture:

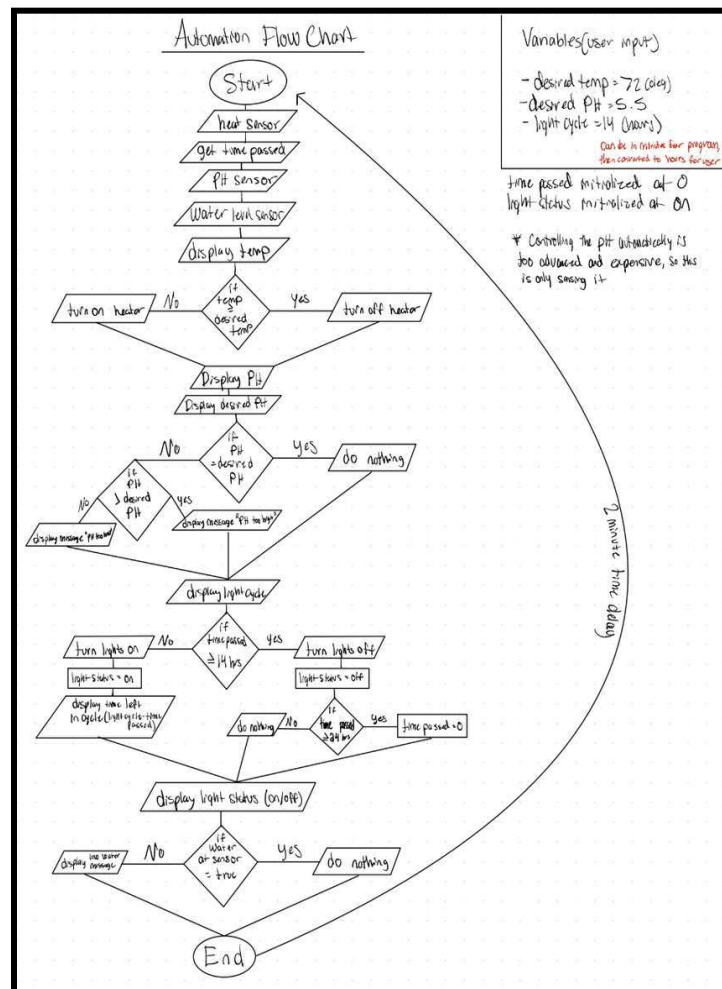
- The sensors collect data and send it to the Raspberry Pi and Arduino
- The microcontroller processes the data and sends it to a backend server or cloud platform.
- The mobile app pulls data from the server and displays real-time sensor information.
- The automation system uses sensor data to adjust the water flow and pH, providing real-time feedback to the user via the app.

Database Schema:

- Tables for storing sensor readings (water temp, pH, water level)
- Control logs for automation actions (pump activated/deactivated, lights on/off)

User Interface:

- Dashboard displaying current sensor readings (values)
- Control buttons to trigger automation (adjust water temp, start pump, turn lights on/off)
- Notes application to track plant growth
- Resources page to connect users to troubleshooting sources
- Alerts for abnormal conditions (water temp too hot/cold, low water level, incorrect pH)



See Appendix pg 129.

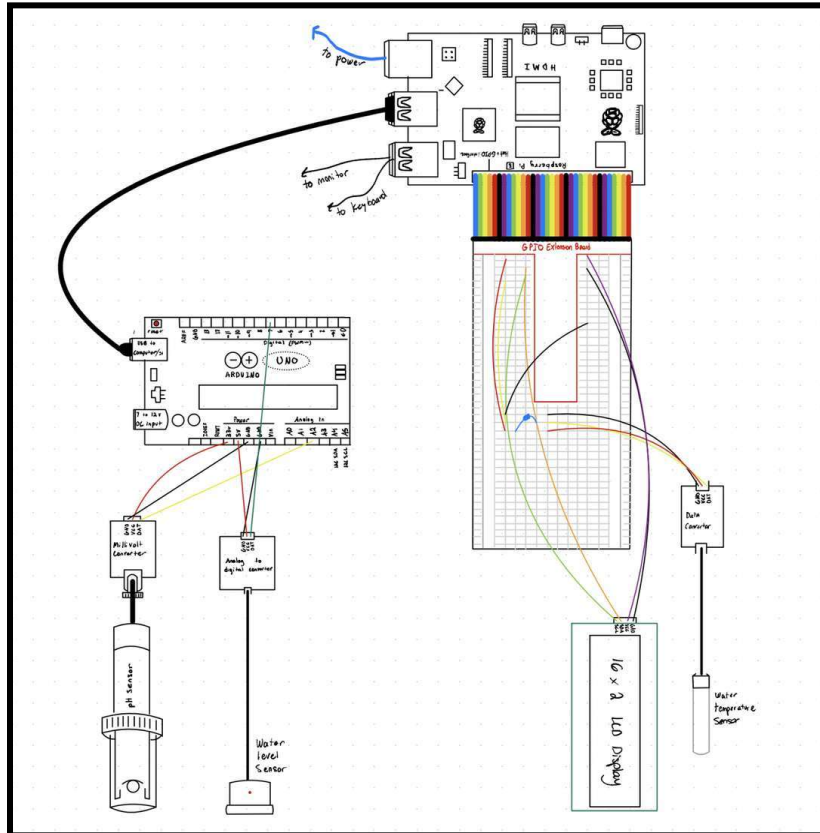
This Automation Flow Chart outlines the decision-making process for monitoring and regulating the hydroponic system. The system starts by collecting data from the temperature sensor, pH sensor, and water level sensor before displaying the recorded data. If the temperature is above the user-defined threshold, the heater turns off. Next, the system displays the pH value and compares it to the desired pH level. If the pH varies from the desired level, a message prompts the user to add up or down. The system then manages the light cycle, checking whether the preset light period has passed. If so, the lights turn off, and the cycle resets. The system also checks whether the water level sensor detects water, displaying a warning message if necessary. The process repeats periodically with a 2-minute delay, ensuring continuous monitoring.

Sensor and Hardware Design

The hydroponic system uses three key sensors: a water temperature sensor, a contactless water level sensor, and a pH sensor, in addition to a 2x16 LCD. The water temperature sensor ensures that the water remains within the ideal range for plant growth, typically between 68°F and 74°F. The water level sensor helps track the amount of water in the system, ensuring that the system never runs out of water. The pH sensor measures the acidity of the water, ensuring it remains within the optimal range of 6.0 to 6.5 (depending on what plants are being grown) for nutrient absorption. The LCD receives the data from sensors and outputs the data on 2 rows for the user to read when they are monitoring their plants.

The processors, such as a Raspberry Pi or Arduino, serve as the brain of the system. They receive the data from the sensors through GPIO(general purpose input/output) pins, send it to the LCD, and write the data to a stored file. I used an Arduino for the water level and pH sensors because these sensors typically output analog signals, and the Arduino has built-in analog input pins that can easily read and process these signals, making it ideal for real-time sensor data handling. The Raspberry Pi would require additional components (like an analog-to-digital converter[ADC]) to read the analog data. Together, this technology is responsible for the data collection and organization of the hydroponic system.

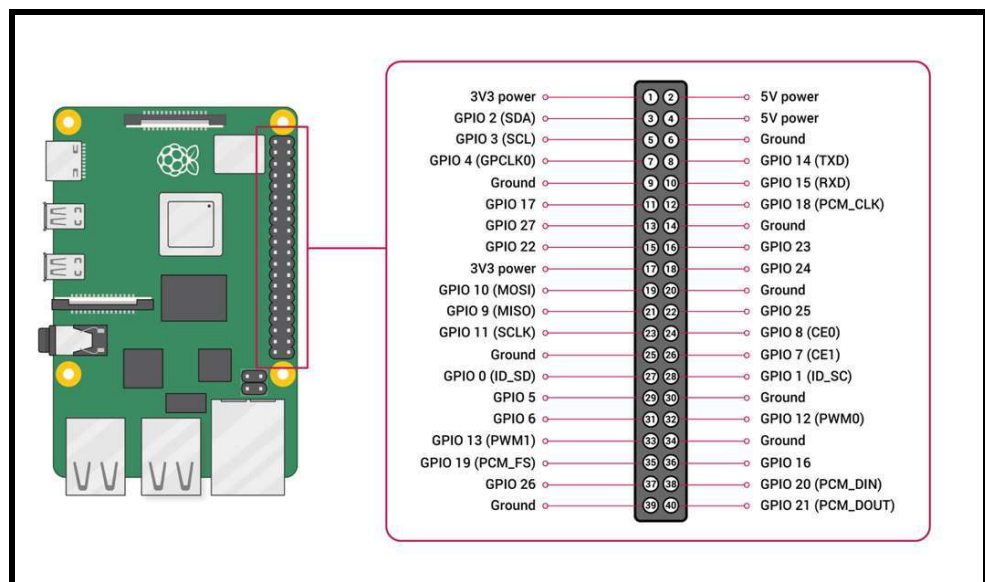
Wiring Diagrams:



See Appendix pg 130.

This Wiring Diagram shows the ports each sensor is connected to, in addition to their wires: ground, data, and power. I used the GPIO extension board so I could include the resistor that limits the current passing through the wires. The extension board attaches to the pins on the Raspberry Pi and allows the user to attach more wires.

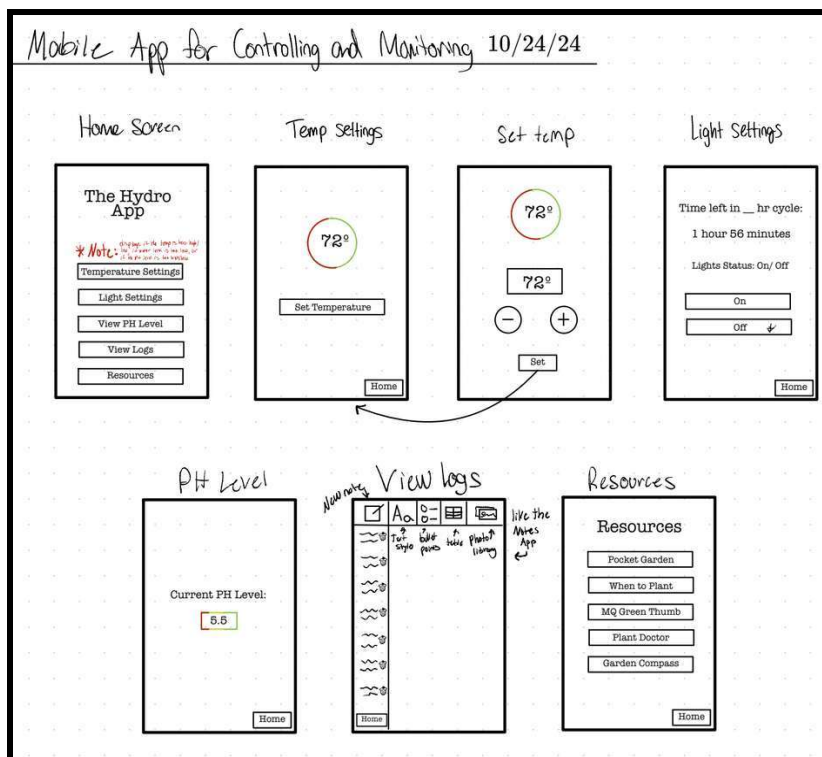
This image shows the functions of all the ports of the Raspberry Pi and their placements. I used this when first learning about the processor to understand where to connect the sensor wires.



See Appendix pg 107.

Mobile Application Design

The Mobile Application Design focuses on creating a functional interface for monitoring and controlling the hydroponic system. The user interface (UI) is designed with simple navigation in mind, incorporating wireframes that outline the layout of key elements such as real-time data displays, control buttons, and system status indicators. The app includes user control elements that allow direct interaction with the system, such as buttons to activate pumps or adjust water flow. User experience (UX) considerations ensure the app is accessible and straightforward, prioritizing simplicity for seamless interaction. A flowchart (found under the architecture and design tab) maps out user interactions and app functionality, ensuring efficient data retrieval and system responses. Firebase, a cloud-based solution for data storage, is implemented to store sensor readings and user inputs such as notes to track plant growth and links for troubleshooting resources, allowing for real-time synchronization and user customization



See Appendix pg 131.

This is my initial sketch of the app design which includes separate pages dedicated to the monitoring of the sensors and control of hydroponic conditions. Initially, I had all warnings appear on the home screen, and a separate screen for the control of the water temperature, however, due to the 5-screen restriction on Thunkable (the app builder I am using), I had to combine all the water maintenance features on one page.

Physical Structure Design

The physical structure of the hydroponic system includes key components purchased from Amazon: an NFT (Nutrient Film Technique) system, a 20-gallon water tank, and grow lights. The NFT system consists of PVC pipes and connectors to create a 3-level structure with 4 rows of cut-out pipes to house the plant sites. In total, the structure contains 108 plant sites suitable for a variety of plants. The 20-gallon tank holds and circulates water for the system, while the grow lights provide the necessary light spectrum for plant growth. To house the sensors and LCD, a custom 3D-printed sensor housing was designed. The housing is specifically crafted for water resistance and convenience, ensuring the sensors remain protected from exposure to splashes and are easily accessible.

Before I constructed the structure of the system, I researched different types of hydroponic systems and their feasibility for a project of this scale. I refined my options to two systems: NFTs and DWCs, and weighed the pros and cons to decide on constructing an NFT system. I also had to consider the type of NFT system, such as an A-frame or a vertical system. One reason to choose hydroponics as a farming technique is for its ability to optimize space by growing vertically, and vertical NFTs do this better than A-frame. This is a central reason why I chose to build a vertical NFT system.

NFT System (Nutrient Film Technique):

Pros:

- Efficient use of water and nutrients due to continuous flow.
- Lightweight and space-saving design, making it ideal for smaller setups.
- Faster plant growth as the roots are exposed to oxygen while absorbing nutrients.

Cons:

- Vulnerable to pump failure or clogging, which can cause water flow issues.
- Not suitable for larger plants with deep root systems.

Deep Water Culture (DWC) System:

Pros:

- Provides plants with an abundant supply of water and nutrients, leading to rapid growth.
- Roots are submerged in nutrient solution with oxygen pumped from below, ensuring plants get all necessary nutrients.

- Less prone to pump failure issues compared to NFT.

Cons:

- Requires larger containers and more space, making it less efficient for small spaces.
- Risk of root diseases due to the constant presence of water.
- Potentially higher maintenance as the nutrient solution needs to be changed regularly.

A-Frame NFT System:

Pros:

- The A-frame structure provides a sturdy base that can withstand heavy nutrient solution flows and is less likely to tip over.
- It can be scaled by adding more tiers to the frame, offering flexibility in scaling the system.

Cons:

- The frame's angled design can be more challenging to build or modify compared to simpler vertical setups.
- The A-frame's design can limit the amount of light some plants receive and plant height.
- Ensuring even water flow across all tiers can be more difficult, especially if the frame has many levels.

Vertical NFT System:

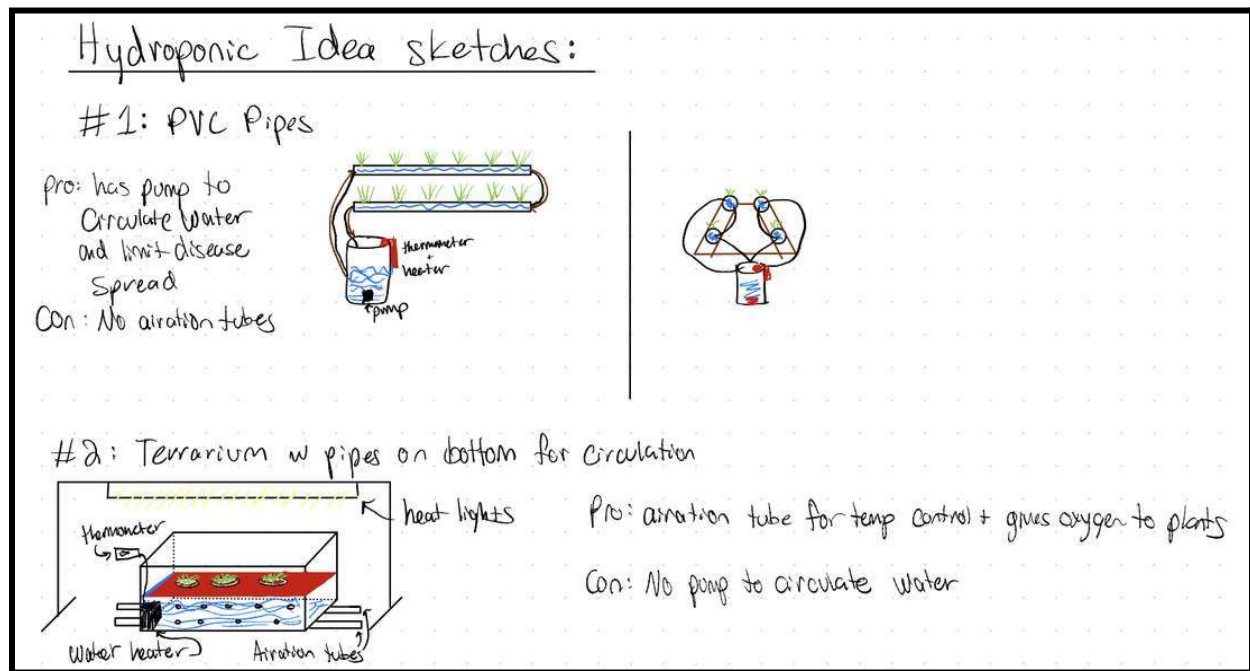
Pros:

- Vertical NFT systems use minimal floor space, making them ideal for high-density urban farming or indoor growing.
- Layers of tubes allow for multiple rows of plants in a small footprint, increasing overall plant capacity.
- You can adjust the number of layers or tiers to suit the space and plant needs.
- Because of its simplicity, vertical NFT systems are often easier to automate (for nutrient delivery and water recirculation).

Cons:

- Reaching the higher levels can be difficult, making plant care, pruning, and harvesting more labor-intensive.
- Ensuring that the nutrient solution flows evenly to all plants in multiple vertical layers can be difficult.
- Taller systems can experience stress from the weight of water and nutrient solution, so structural support needs to be robust.

The following sketches outline the possible designs for the structure of the system, and my initial comments on each design.

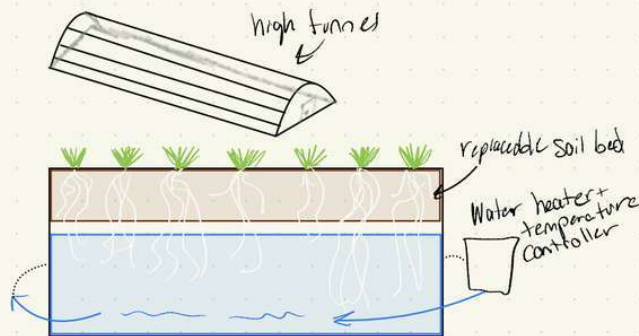


See Appendix pg 132.

These are the first 2 sketches I drew when deciding between the NFT system and a DWC system. A DWC system would have been pretty difficult to construct given the limited space available to me, and when thinking about area optimization, NFT is the better choice.

Terrarium sketch/idea #2: Hydroponics

9/20/24



Features:

- High tunnels to protect plants against rising heats
- Ventilation
- Lined with hydroponics
 - ↳ Water can be heated/cooled
 - ↳ filtration pumps water + nutrients
- Thermometer
- LED light (to mimic sun)
- Solar panels (power the mechanics)
- Soil bed can be replaced

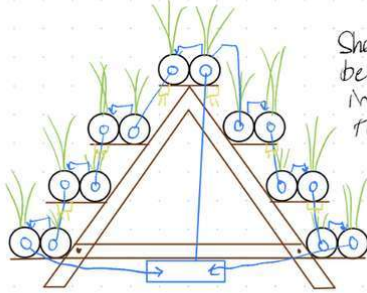
See Appendix pg 132.

Back in September, when I was thinking more broadly, I thought of placing the hydroponic systems in high tunnels so the temperature and humidity could also be controlled. If I were experimenting with optimal growing conditions, I would consider this (or another way of controlling the air as well as the water).

Hydroponic structure sketch #3 (10/24/24)

I want to have 3 or 4 levels with 4 tubes of plants (2 on each side). Each tube will have 8-10 (probably 9) plant spots.

Side view:

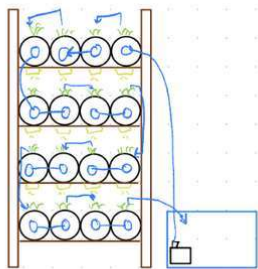


Shorter plants can be placed on the inside, larger on the outside

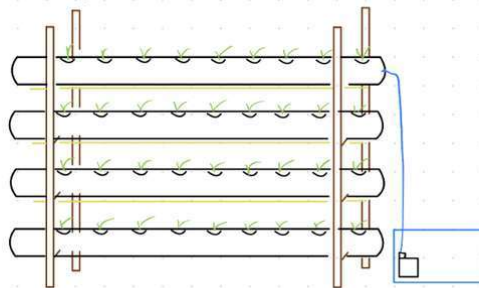
* I might not be able to incorporate lights into this design because plants are spread out. Unless I place lights only over smaller plants that are covered anyways

Structure Sketch #4 (10/24/24)

Side view:

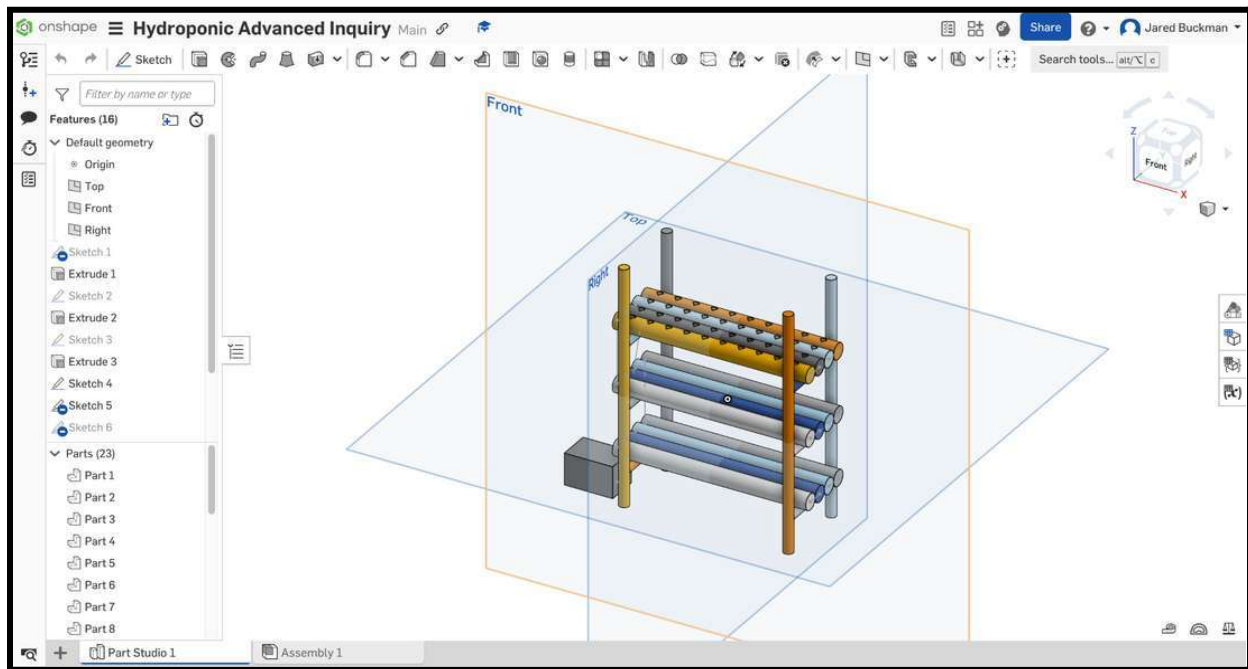


Front view:



See Appendix pg 133.

Sketches 3 and 4 visualize my thinking behind my decision to use a vertical NFT system instead of an A-frame NFT system. The idea behind the A-frame was to have taller plants on the outside, and shorter plants on the inside where there was another level above them with a grow light attached. This idea, however, wasn't feasible due to a lack of space optimization, the time it would take to gather materials and build this, and the lack of support tall plants would have on the outside.

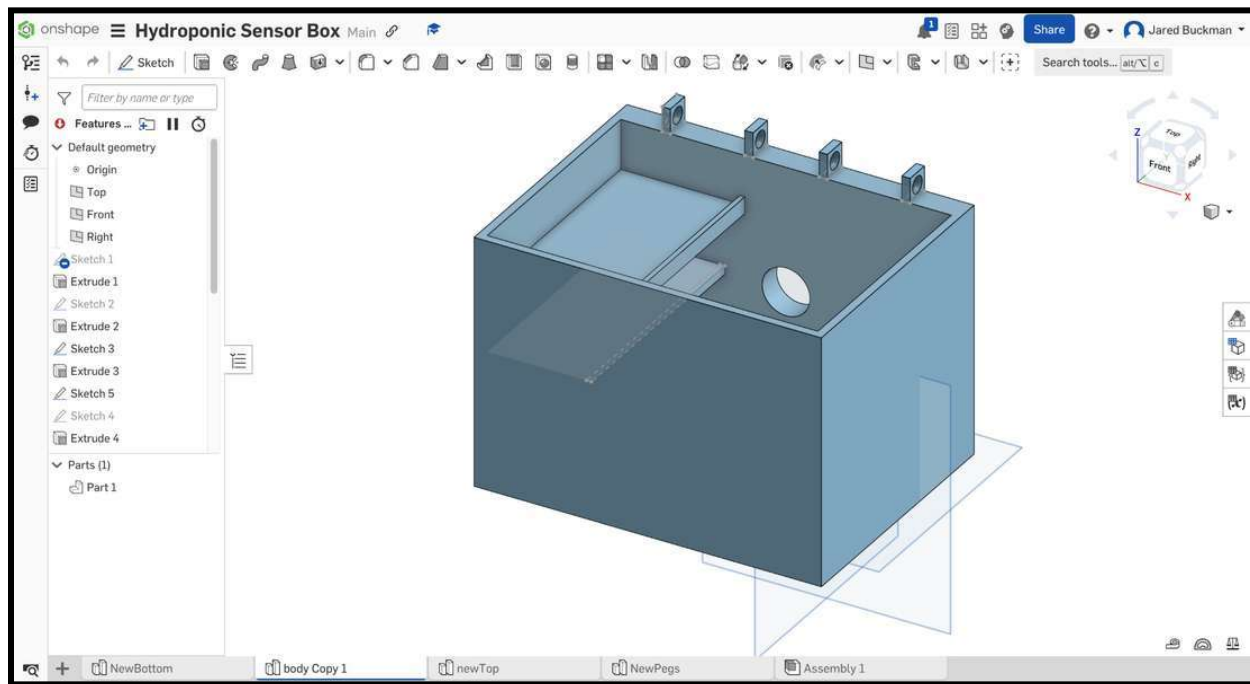


See Appendix pg 134.

Once I decided on my vertical NFT system design, I used OnShape, a CAD software, to sketch my PVC structure in case I were to build it from scratch.

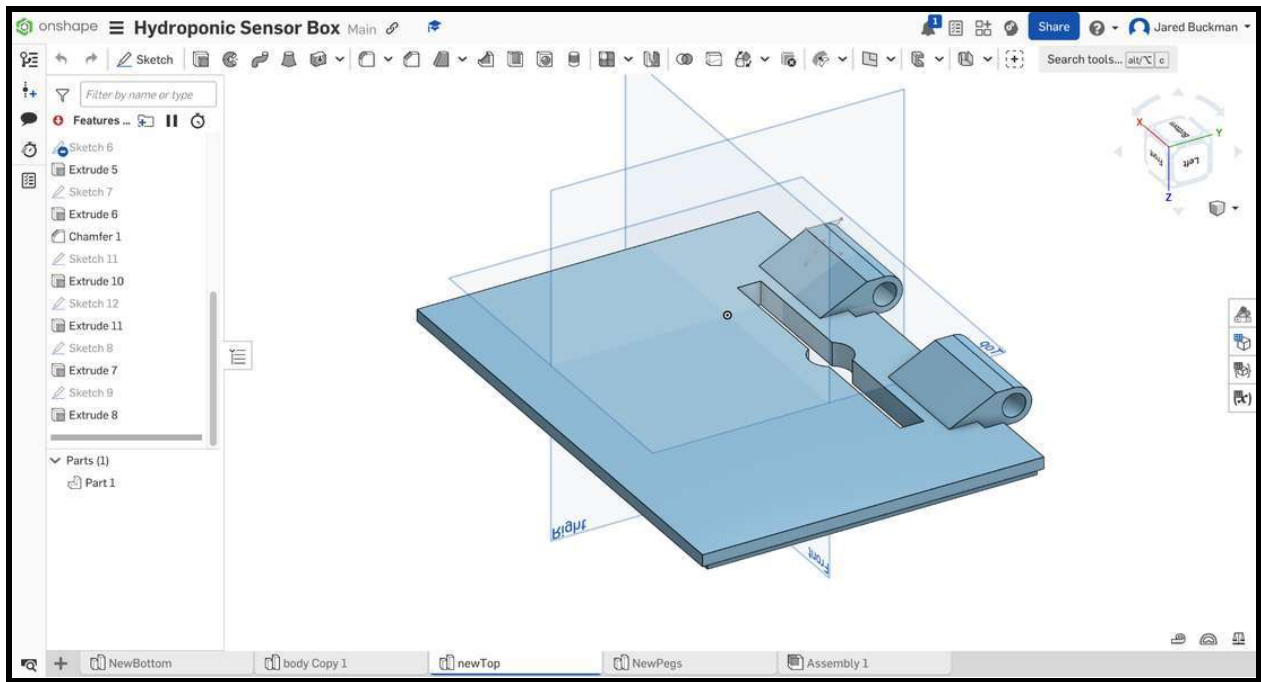
3D Printed Sensor Housing

Once I finished assembling the sensors and processors, I had to think about how to keep the technology safe, and how the sensors were supposed to gather data. Having used CAD and OnShape before, I made a 3D sketch of a hollow box and a lid to sit on top. From there, I created hinges (and pegs to insert) to allow the lid to attach to the body, and open and close. I cut a hole from the top for the sensors, and for the processors to receive air. This contraption is then attached to the container of water with the cut-out circle pressing against the container as this is where the water level sensor attaches.



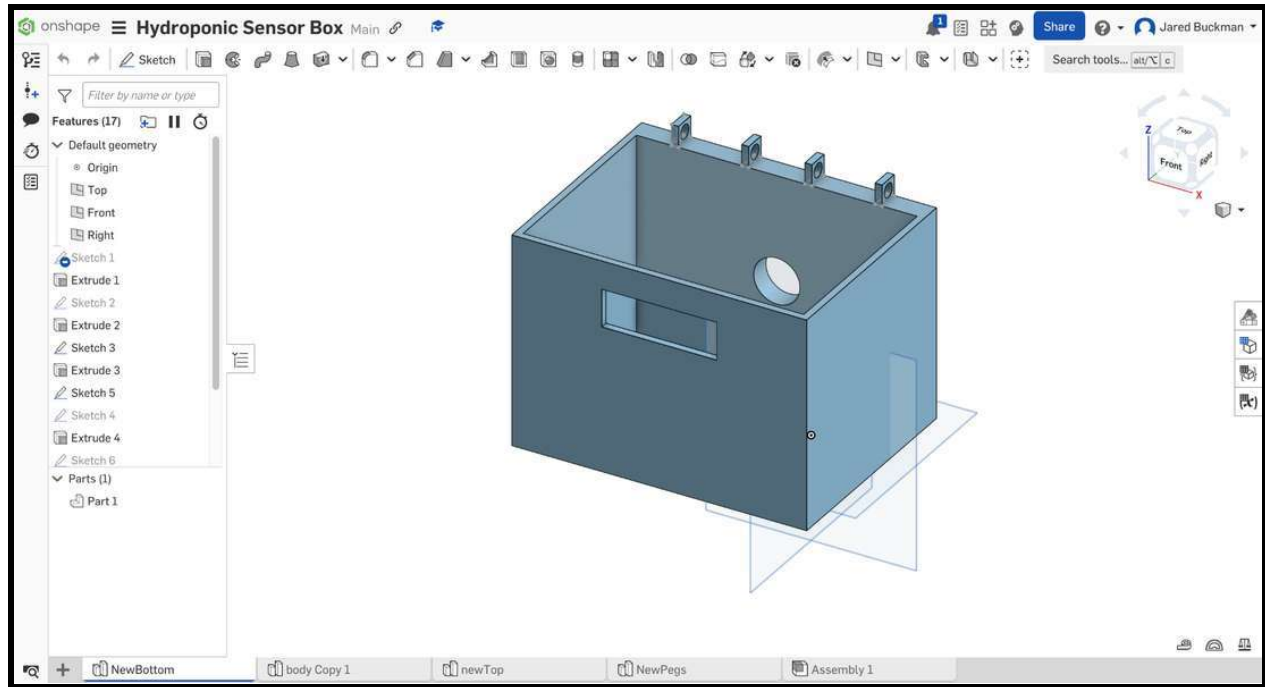
See Appendix pg 134.

This is my initial sketch of the body, after creating the basic shell. I added shelves for the Raspberry Pi and Arduino to sit inside the box. Theoretically, this would have been nice, but the 3D printer was unable to print supports for these shelves, so I had to print the shelves separately and place them in the box. This edition also didn't include the LCD as I had not assembled it yet.



See Appendix pg 135.

This is the final version of the lid, with a raised lip to ensure a secure connection with the body.



See Appendix pg 135.

This is the final version of the body, with a circular cut-out for the water level sensor, and a rectangular cut-out fit for the LCD.

Testing Table

Test No.	Test Description	Test Data (input)	Expected Outcome
1	Water temp sensor	70.8°F	Temperature in Celsius and Fahrenheit as a float
2	pH Sensor	pH: 7	pH and voltage values as a float
3	Water Level Sensor	"Water level low"	Boolean outcome displayed as a string
4	Combining Arduino file with raspberry pi file	2025-01-08 09:23:49= 22.5C 72.85F Voltage: 0.86V pH: 7.85 Water level low. Add more water.	Prints the time, then temperature data, then arduino data in the output
5	Write sensor data to a .csv file	2025-01-08 09:23:49= 22.5C 72.85F Voltage: 0.86V pH: 7.85 Water level low. Add more water.	Prints the time, then temperature data, then arduino data in a new .csv file
6	Display sensor data on LCD	22.5C 72.85F Voltage: 0.86V pH: 7.85 Water level low. Add more water.	Prints the temp data line 1, and scrolling arduino data on line 2
7	Allows the raspberry to run on a schedule using crontab	0 6 * * * /home/pi/start_displaySensorData.sh 0 21 * * * /home/pi/stop_displaySensorData.sh	Raspberry pi runs from 6AM to 9PM every day
8	Headless technology	@reboot /bin/sleep 30; /usr/bin/python /home/pi/displaySensorData.py	Makes the raspberry pi run without wired connection to the monitor
9	User interface for app	Clicking buttons	User can toggle screens and use control buttons
10	Local storage for notes app	Testing 123	App temporarily stores user-inputted notes
11	Local storage for resources	Testing title Testing link	App temporarily stores user-inputted links and titles

12	Cloud storage for app	Enters sample notes, closes app, and comes back	App permanently stores data and user input until deleted
13	3D Structure print	(30x20x20) cm	A 3D box has enough room to house the sensors and the lid fits the body tightly
14	Construction of frame	Water pumped through	The frame pieces fit securely and no water leaks
15	Water pump	Pump turned on for 5 minutes, rest for 30 minutes	When activated, water is pumped out at constant rate with no leaking
16	Grow lights	Lights turned on for 3 hours	When connected, lights shine uniformly on the plants below

Data Dictionary

Field Name	Data Type	Description
base_dir	String	Path to the directory containing 1-Wire sensor devices
device_path	String	Path to the specific water temperature sensor
rom	String	ROM name of the connected temperature senso
I2C_ADDR	Integer	I2C address of the LCD display
LCD_WIDTH	Integer	Maximum number of characters per line on the LCD display
LCD_CHR	Integer	LCD mode for sending character data
LCD_CMD	Integer	LCD mode for sending command data
LCD_LINE_1	Integer	Address of the first line on the LCD display
LCD_LINE_2	Integer	Address of the second line on the LCD display
LCD_BACKLIGHT	Integer	Backlight control bit for the LCD
ENABLE	Integer	Enable bit for toggling LCD commands
E_DELAY	Float	Delay time in seconds for LCD operations
bus	SMBus Object	SMBus object used for I2C communication
arduino_port	String	Serial port address where the Arduino is connected
baud_rate	Integer	Baud rate for serial communication with Arduino
output_file	String	Name of the CSV file used to store collected data (CombinedData.csv)
ser	Serial Object	Serial object for communicating with the Arduino
temp_c	Float	Measured water temperature in Celsius (rounded to 2 decimal places)
temp_f	Float	Measured water temperature in Fahrenheit (rounded to 2 decimal places)

timestamp	Datetime	Current date and time when the data is recorded
start_time	Datetime	Time of when the while loops begins again
arduino_data	String	Data received from the Arduino, including voltage, pH, and water level readings
file	CSV Writer Object	CSV writer object used for writing data to the CombinedData.csv file

Development

Overview

In this phase, I will discuss the development of my Hydroponic Project, which consists of three key components: the sensors and hardware, the mobile application, and the physical structure of the system. Each component plays a vital role in ensuring the system functions efficiently, providing users with real-time data and control over their hydroponic environment. The sensors and hardware monitor crucial factors such as water temperature, pH levels, and water levels, allowing the system to adjust conditions automatically. The mobile application enables users to access this data remotely, receive alerts, and make adjustments from their smartphones. Lastly, the physical structure, including the NFT system and sensor housing, ensures that all components are securely integrated and optimized for plant growth. Throughout this section, I will detail the technical process of creating of these components, challenges encountered, the solutions implemented, and the integration process to create a seamless, functional system.

Sensors and Hardware

In this section, I will explain the development and integration of the sensors that monitor the hydroponic system, such as water temperature, pH level, and water level. Additionally, I will show how I used Raspberry Pi and Arduino to program these sensors and provide pieces of code to show the background of what is happening. All of my code can be found in the appendix. The sensor and automation components are central to the project, providing real-time data and automated responses that ensure optimal plant growth conditions.

Introduction to Sensors and Automation

The sensors used in this project monitor the core aspects of the hydroponic system, while the automation system processes sensor data to manage water flow and other system activities. By integrating these technologies, I will create a smart, self-regulating hydroponic system that ensures accurate environmental control for plant growth. Accurate readings will be displayed on the LCD and set on a timer to optimize power consumption, in addition to a mobile device.

Essential Materials

- Raspberry Pi
- Arduino
- Water Temperature Sensor
- pH Level Sensor
- Water Level Sensor
- LCD Display

Stages

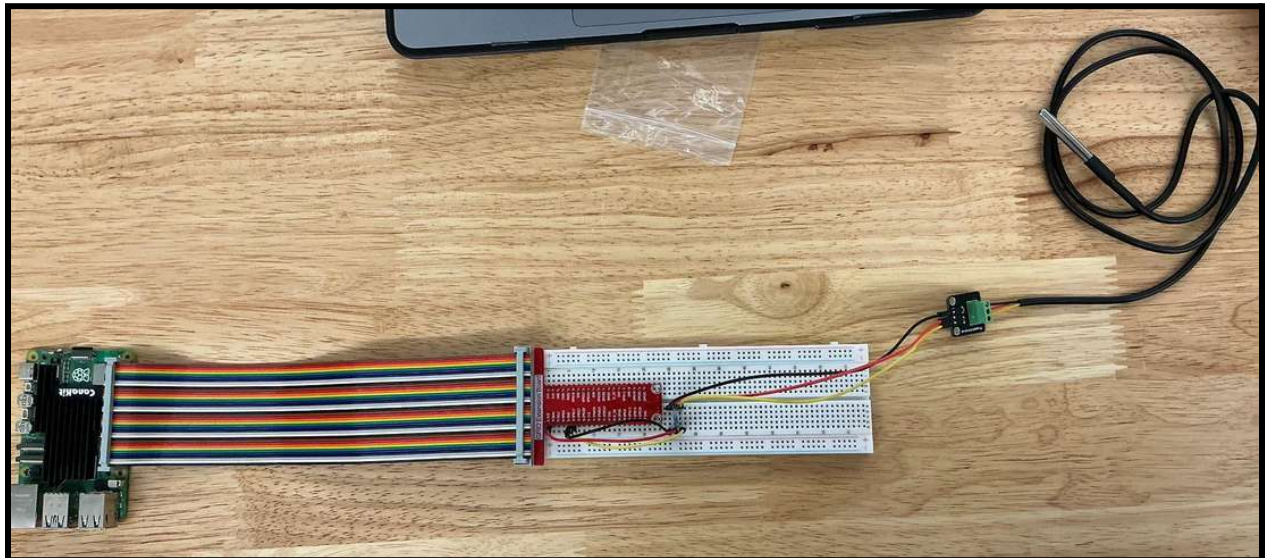
- Prepare: Research the sensors and display and sketch the wiring diagram
- Connect: Set up sensors with the Raspberry Pi and Arduino and test data transmission.
- Program: Code the controller platforms to format the data and output the sensor data to a combined file.

Water Temperature Sensor

The DS18B20 temperature sensor is used in the system to monitor water temperature. This sensor is chosen for its waterproof design, high accuracy, and wide temperature range, making it ideal for hydroponic applications. To integrate the sensor with the Raspberry Pi, a breadboard and a GPIO extension board are used. A $4.7k\Omega$ pull-up resistor is placed between the data wire and the 3.3V power supply to stabilize the communication between the sensor and the Raspberry Pi.

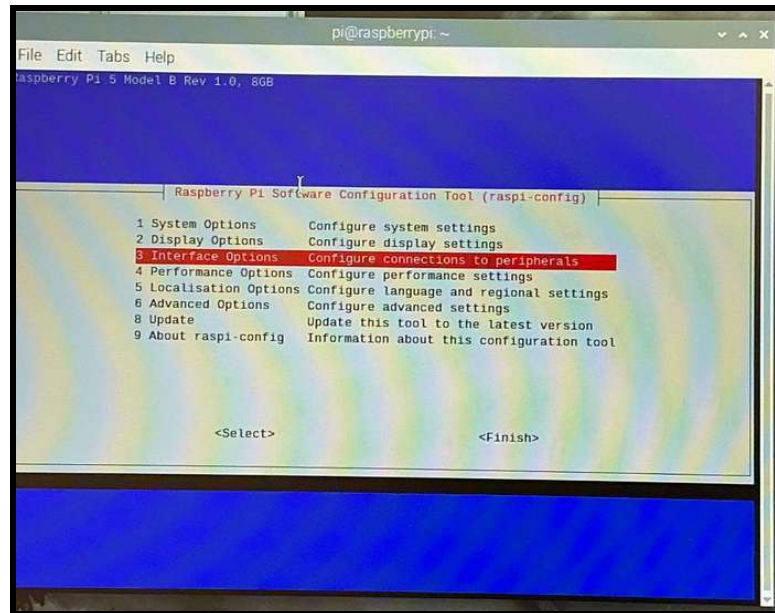
Python handles data collection, using the `w1-thermsensor` library to communicate with the sensor. Temperature readings are taken at regular intervals, processed, and formatted before being sent to the mobile app. This allows for real-time monitoring of water temperature, ensuring optimal growing conditions. The temperature data plays a crucial role in the system as water temperature is a key factor in plant growth. If readings indicate a deviation from the ideal range, users will receive an alert through the mobile app.

Some challenges included occasional sensor misreads and fluctuations. Debugging involved checking the wiring and ensuring a stable connection through the breadboard. The system now reliably captures and displays water temperature data.



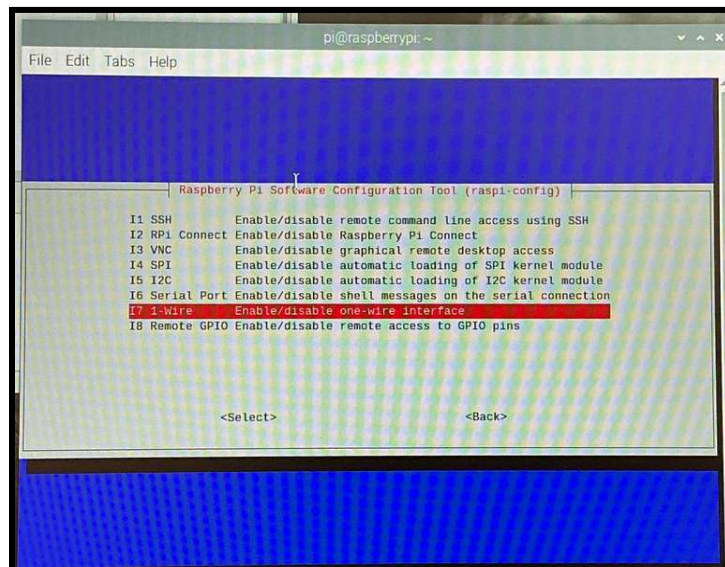
See Appendix pg 108.

This is an image of the wiring for the DS18B20 temperature sensor. It was fairly simple to assemble, especially because the GPIO extension board allowed me to include a resistor.



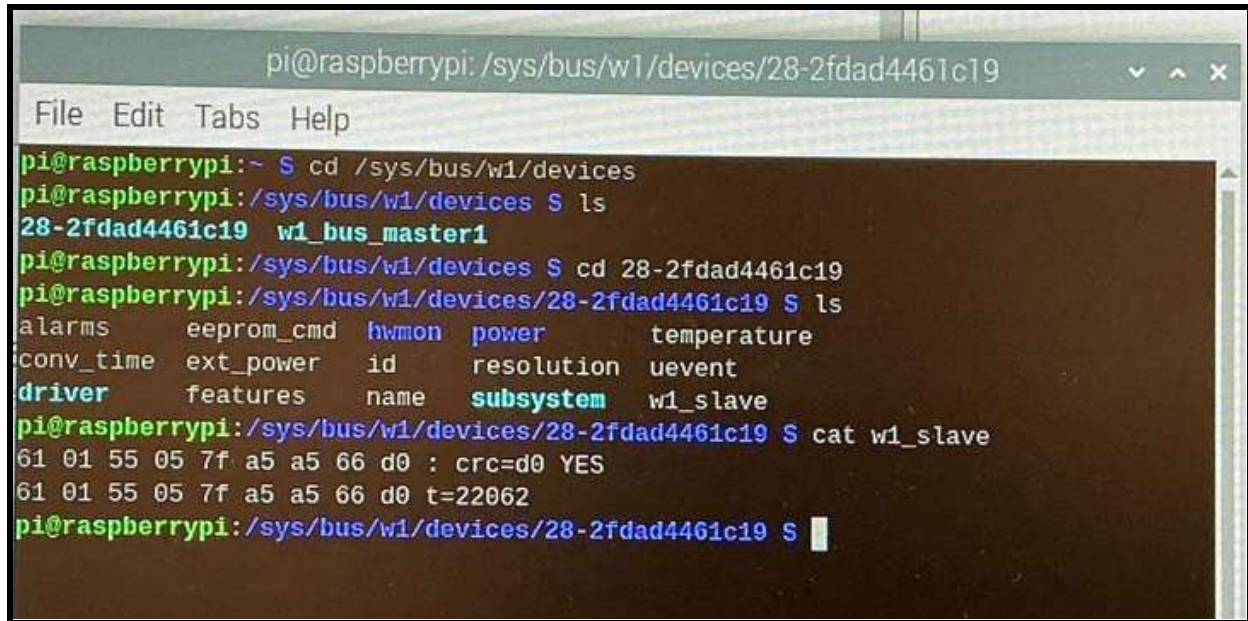
See Appendix pg 108.

1-Wire communication protocol allows the temperature sensor, and other sensors, to be connected to a single data line on the Raspberry Pi which reduces the number of pins needed on the Raspberry Pi. To set this up, I first needed to type **sudo raspi-config** directly into the the Raspberry Pi. This opens the configuration settings and allows me to adjust the Interface options.



See Appendix pg 109.

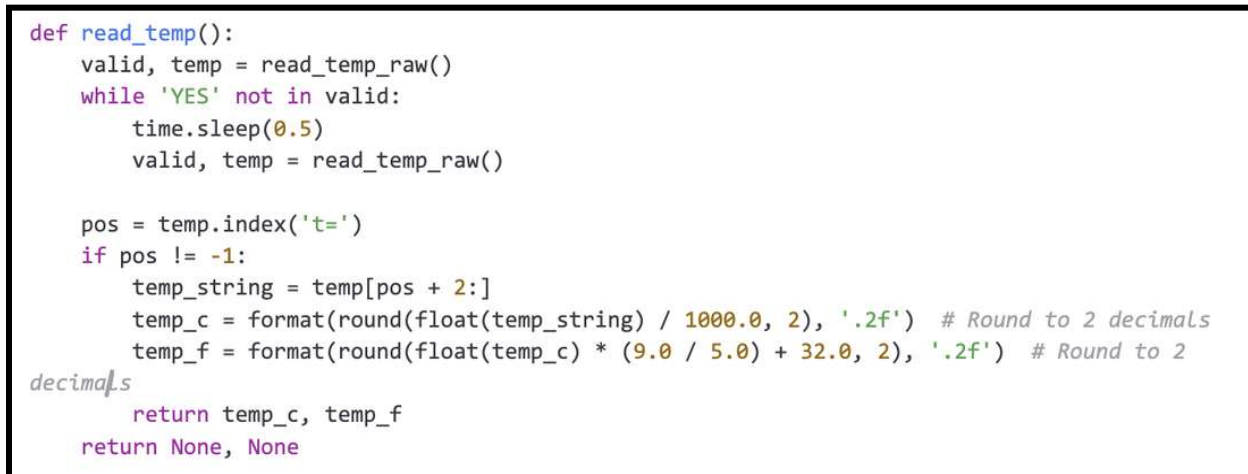
From there, I can enable 1-wire communication. Once this is selected, I rebooted the system for these changes to occur.



```
pi@raspberrypi: /sys/bus/w1/devices/28-2fdad4461c19
File Edit Tabs Help
pi@raspberrypi:~ $ cd /sys/bus/w1/devices
pi@raspberrypi:/sys/bus/w1/devices $ ls
28-2fdad4461c19 w1_bus_master1
pi@raspberrypi:/sys/bus/w1/devices $ cd 28-2fdad4461c19
pi@raspberrypi:/sys/bus/w1/devices/28-2fdad4461c19 $ ls
alarms      eeeprom_cmd hwmon       power       temperature
conv_time   ext_power   id          resolution  uevent
driver      features    name        subsystem   w1_slave
pi@raspberrypi:/sys/bus/w1/devices/28-2fdad4461c19 $ cat w1_slave
61 01 55 05 7f a5 a5 66 d0 : crc=d0 YES
61 01 55 05 7f a5 a5 66 d0 t=22062
pi@raspberrypi:/sys/bus/w1/devices/28-2fdad4461c19 $
```

See Appendix pg 109.

To ensure that my temperature sensor is connected, I checked the 1-wire devices directory to see what sensors are connected. `cd /sys/bus/w1/devices` navigates to the 1-wire devices directory. Typing `ls` lists all the files in the directory, which showed me my temperature sensor. Typing `cd 28-2fdad4461c19` and `cat w1_slave` get the sensor directory and displays its data. Here, `t=22062` shows that the sensor is reading a water temperature of 22.062°C.

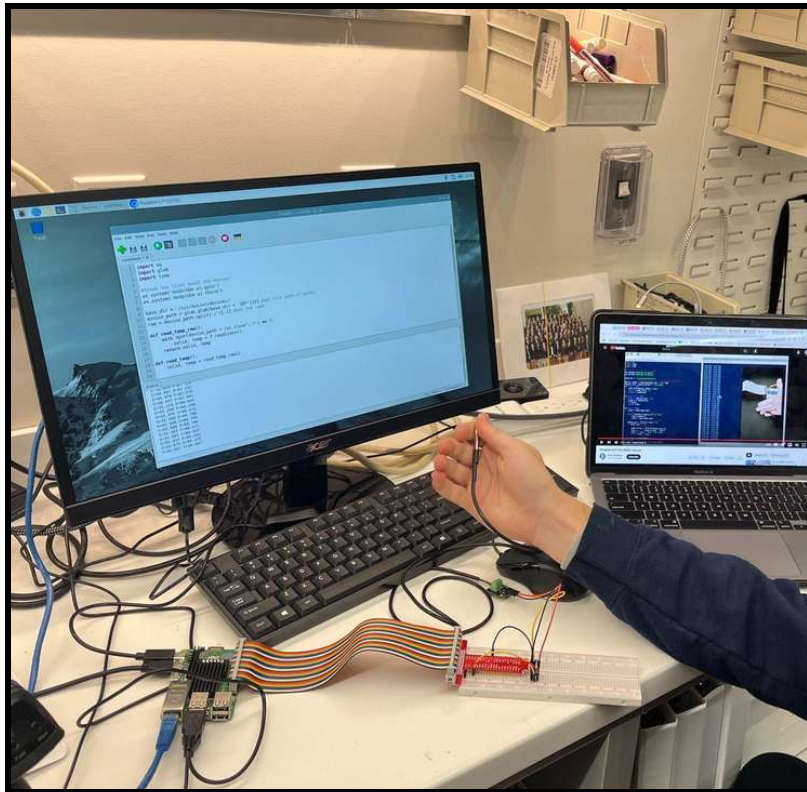


```
def read_temp():
    valid, temp = read_temp_raw()
    while 'YES' not in valid:
        time.sleep(0.5)
        valid, temp = read_temp_raw()

    pos = temp.index('t=')
    if pos != -1:
        temp_string = temp[pos + 2:]
        temp_c = format(round(float(temp_string) / 1000.0, 2), '.2f') # Round to 2 decimals
        temp_f = format(round(float(temp_c) * (9.0 / 5.0) + 32.0, 2), '.2f') # Round to 2
        decimals
    return temp_c, temp_f
    return None, None
```

See Appendix pg 151.

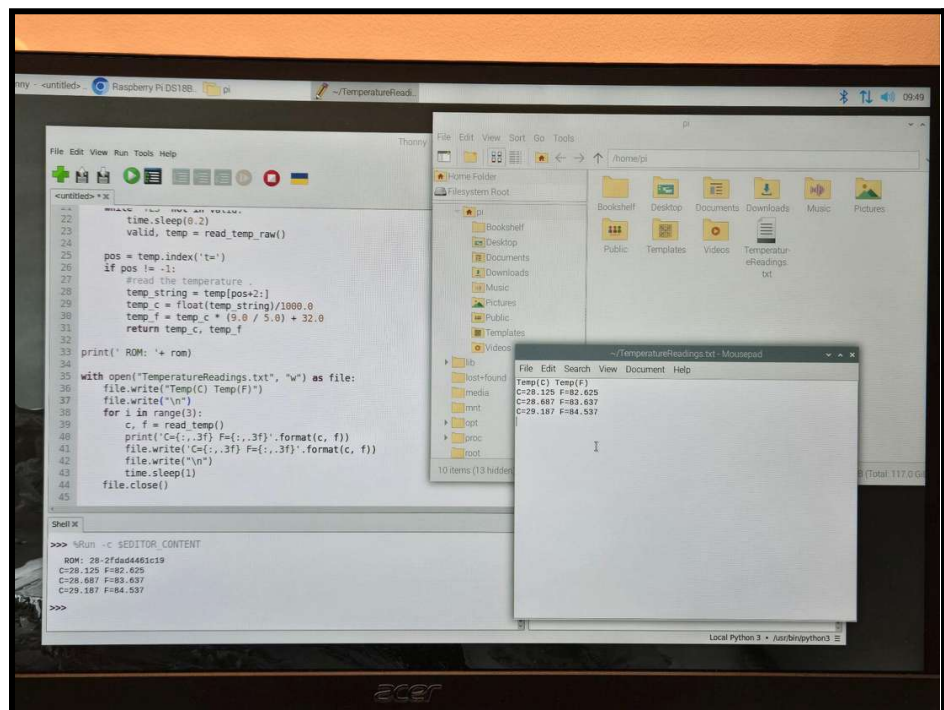
This is the function in my final python program that reads the temperature from the sensor, sorts the data, and returns the temperature in degrees celsius and fahrenheit.



In this image, you can see me testing the sensor after I set up the 1-wire interface and wrote the code in Thonny, a Python IDE. The data is sent to the output where the temperature is displayed in Celsius and Fahrenheit (the code includes the conversion).

See Appendix pg 110.

This is an image of my temperature data writing to a new .CSV file I created. Within the code, I created a loop that only took 3 readings to ensure the code worked. Here you can see that my trial was successful.



See Appendix pg 110.

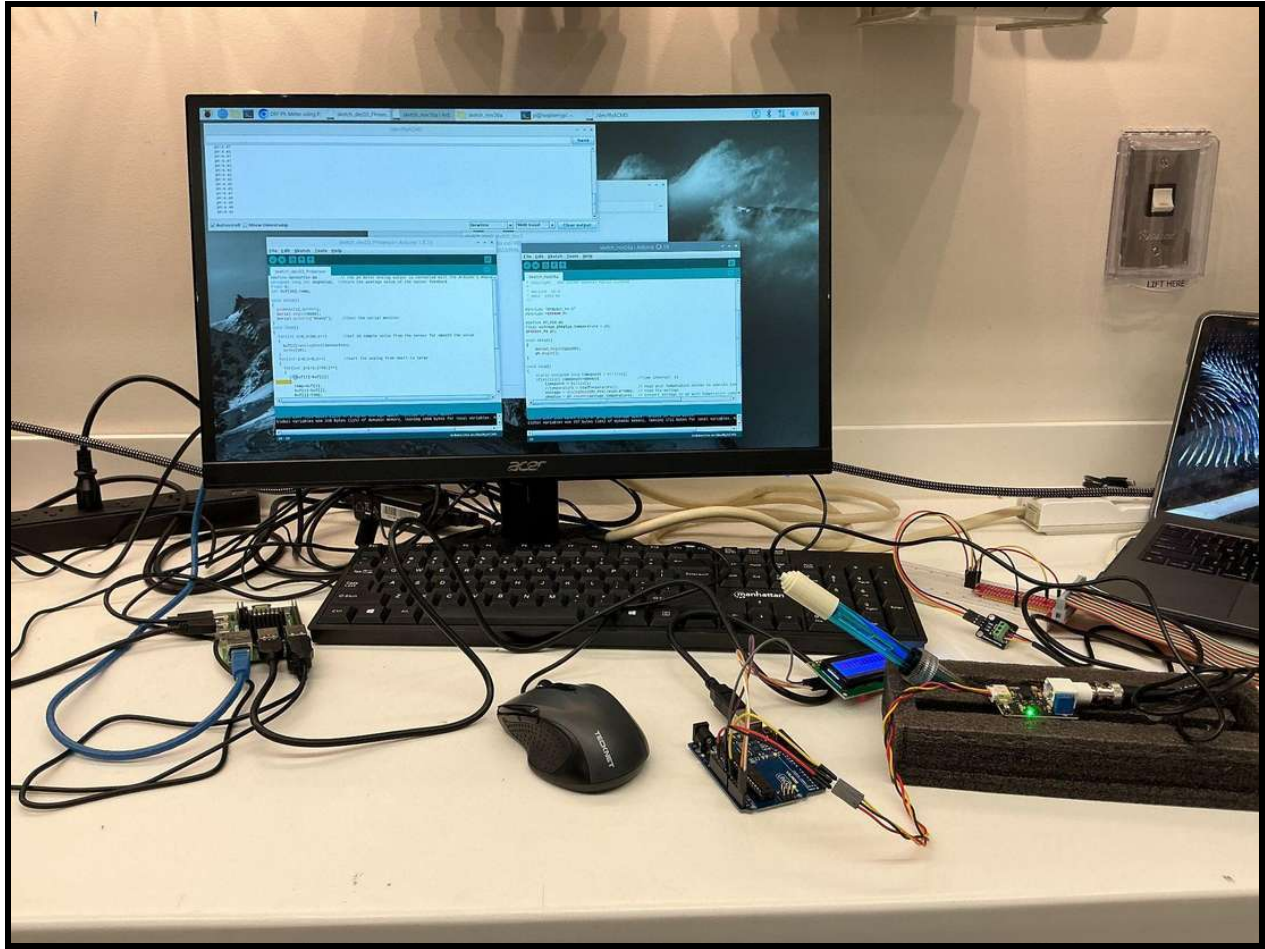
pH Level Sensor

The pH sensor used in the project is the DFRobot Gravity Analog pH Sensor. This sensor operates within a voltage range of 0 to 5V and is capable of accurately measuring the pH level in solutions, with an accuracy of ± 0.1 pH under ideal conditions. Calibration is essential for precise readings, and the sensor requires periodic calibration using pH buffer solutions to ensure that it provides accurate measurements over time. The pH sensor is connected to the Arduino through its millivolt converter, which translates the sensor's analog voltage output into a readable signal. The output from the converter is then sent to the Arduino analog input pin. The sensor typically requires 5V from the Arduino's power supply with no additional power regulation necessary.

The project uses the Arduino IDE to manage the data received from the pH sensor. The sensor readings are collected from the analog input pin and converted into a voltage value. This voltage is then mapped to a corresponding pH level using a calibration curve within the code. The calibration process involves immersing the sensor in pH buffer solutions (pH 4, 7, and 10) and applying linear regression to adjust the resulting values. This ensures that each reading corresponds to a specific pH level, rather than just the pH of the buffer solutions. The constants derived from this calibration are then incorporated into the code to ensure accurate pH measurements. Additionally, the code takes 10 readings, cuts off the two lowest and highest, then averages the 6 remaining values to get a more accurate measurement.

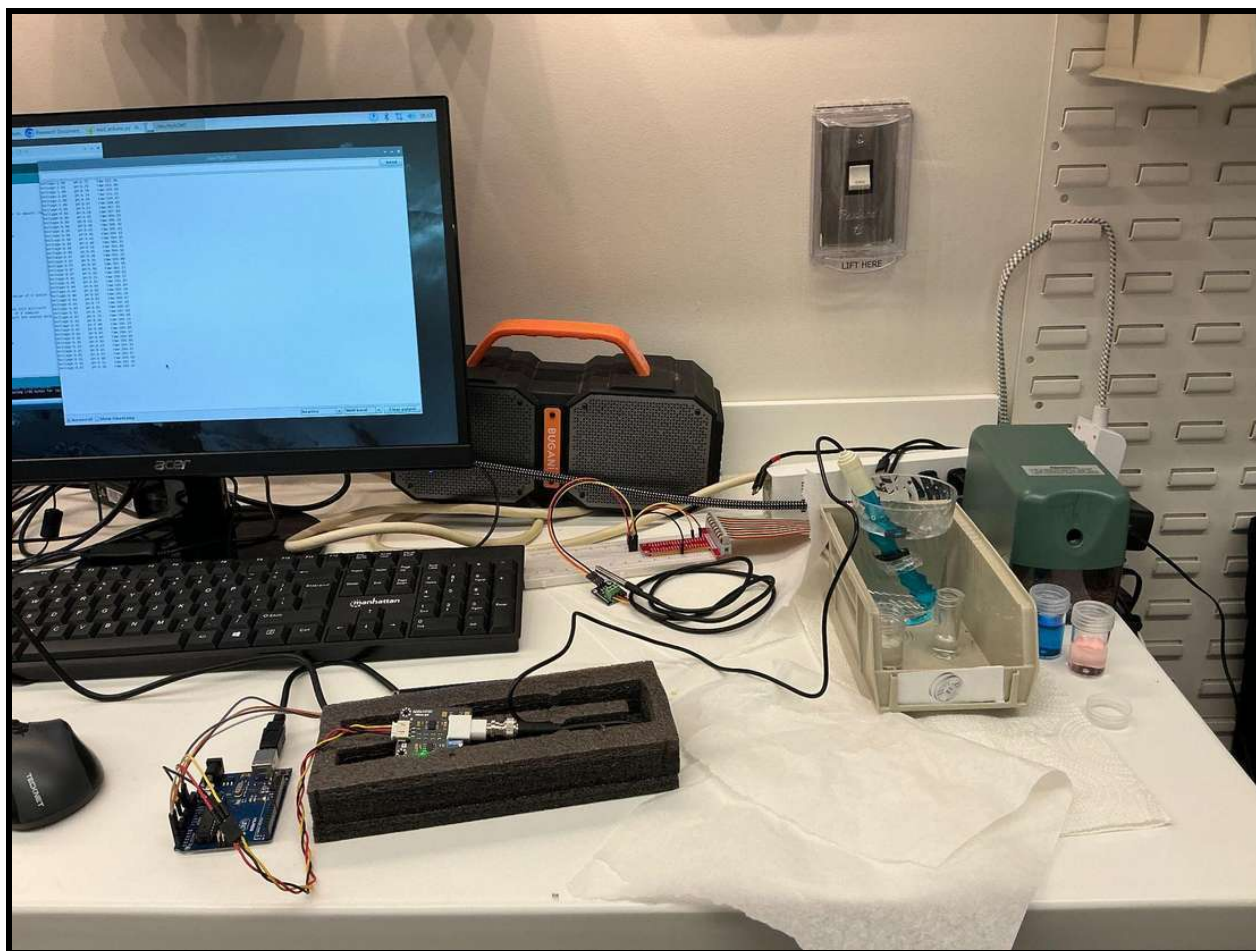
The pH data plays a crucial role in maintaining the acidity levels within the hydroponic system, as different plants grow better in different pH levels. If the pH value deviates by more than 0.5 pH units from the desired range, the system triggers an alert to the user via the mobile app. In the future, the system can be designed to automatically adjust the pH by triggering pumps to regulate the solution's acidity. The pH data is stored and displayed on the mobile app, where the user can view real-time pH values.

One of the main challenges encountered with the pH sensor was receiving incorrect readings resulting from poor calibration, especially if the buffer solutions were contaminated during the calibration process or if the sensor wasn't fully immersed. To address these issues, the system underwent a few recalibration trials to ensure accurate pH readings.



See Appendix pg 111.

This image displays the pH sensor setup, connected to an Arduino, which is linked to a Raspberry Pi and then to a monitor for data visualization. The monitor shows two Arduino files: on the left is a backup copy of the sensor code from the previous class, while the file on the right is actively running. At the top of the screen, the data viewer presents real-time readings from the pH sensor.



See Appendix pg 111.

This image shows the process of calibrating the pH sensor. Currently, the sensor is sitting in distilled water to clean, after being used for calibration with the pH 7 buffer solution. This was the second endpoint for my calibration: I used the buffer solution of 4 and 7 to create an equation, and tested my results with the pH 10 buffer solution.

The following images show the readings I received when the sensor was submersed in all 3 buffer solutions:

pH 4:

```
sketch_dec03_PHsensor | Arduino 1.8.19
File Edit Sketch Tools Help
sketch_dec03_PHsensor
void setup()
{
  pinMode(13,OUTPUT);
  Serial.begin(9600);
  Serial.println("Ready"); //Test the serial monitor
}

void loop()
{
  for(int i=0;i<10;i++) //Get 10 sample values from the sensor
  {
    buf[i]=analogRead(SensorPin);
    delay(20);
  }
  for(int i=0;i<10;i++) //sort the analog from small to large
  {
    for(int j=i+1;j<10;j++)
    {
      if(buf[i]>buf[j])
      {
        temp=buf[i];
        buf[i]=buf[j];
        buf[j]=temp;
      }
    }
  }

  avgvalue=0;
  for(int i=0;i<10;i++)
  {
    avgvalue+=buf[i];
  }
  float voltage=(float)avgvalue*0.0024/5; //convert the analog in
  float pHvalue=(float)avgvalue/5; //take average of 6
  float calibratedpH = m*pHvalue + b; //convert to pH

  Serial.print("Voltage:"); //print voltage heading
  Serial.print(voltage, 2); //print voltage data
  Serial.print(" pH:"); //print pH heading same line
  Serial.print(calibratedpH, 2); //print pH data
  Serial.print(" pH:"); //print pH heading same line
  Serial.print(pHvalue, 2); //print pH data
  Serial.println("");
  digitalWrite(13, HIGH);
  delay(800);
}

Global variables use 240 bytes (12% of dynamic memory, leaving 1800
Autoscroll Show timestamp
Newline 9600
```

See Appendix pg 112.

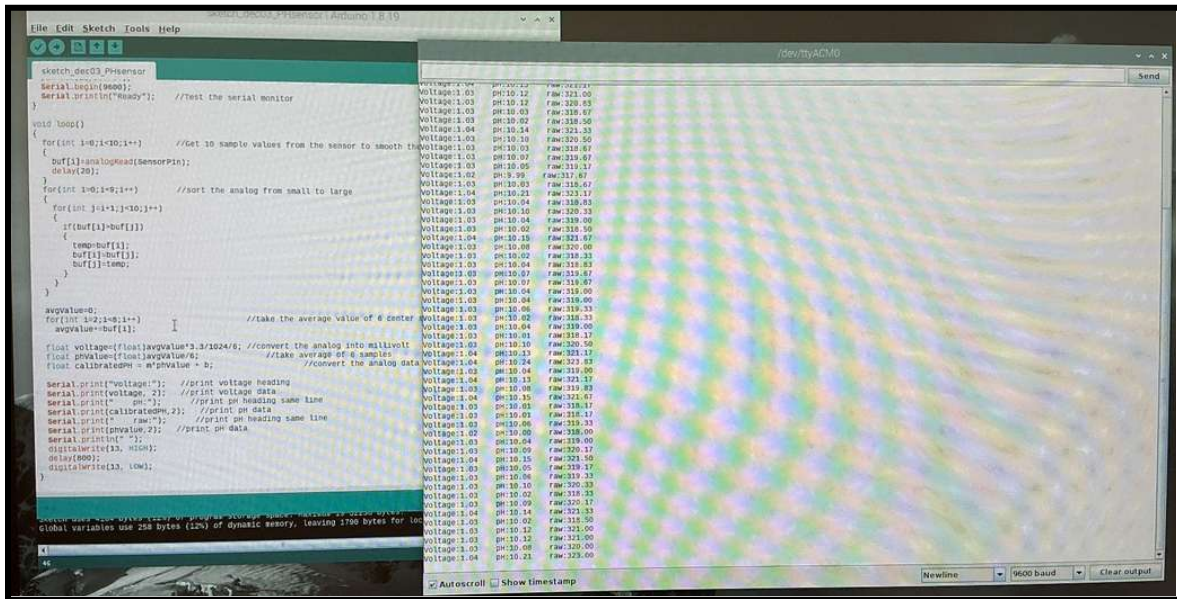
pH 7:

```
sketch_dec03_PHsensor | Arduino 1.8.19
Help
sketch_dec03_PHsensor
06:32:05.176 -> Voltage:1.20 pH:7.00 pH:248.77
06:32:05.178 -> Voltage:1.21 pH:7.17 pH:248.50
06:32:06.171 -> Voltage:1.22 pH:7.19 pH:248.83
06:32:07.183 -> Voltage:1.22 pH:7.19 pH:248.83
06:32:08.190 -> Voltage:1.20 pH:7.09 pH:248.33
06:32:09.183 -> Voltage:1.21 pH:7.15 pH:248.00
06:32:10.176 -> Voltage:1.20 pH:7.08 pH:246.17
06:32:11.202 -> Voltage:1.21 pH:7.12 pH:247.17
06:32:12.196 -> Voltage:1.20 pH:7.05 pH:245.33
06:32:13.189 -> Voltage:1.20 pH:7.09 pH:246.33
06:32:14.214 -> Voltage:1.20 pH:7.04 pH:245.17
06:32:15.206 -> Voltage:1.21 pH:7.15 pH:248.00
06:32:16.198 -> Voltage:1.20 pH:7.06 pH:245.67
06:32:17.224 -> Voltage:1.21 pH:7.11 pH:247.00
06:32:18.218 -> Voltage:1.21 pH:7.17 pH:248.33
06:32:19.210 -> Voltage:1.22 pH:7.20 pH:249.17
06:32:20.206 -> Voltage:1.21 pH:7.13 pH:247.50
06:32:21.232 -> Voltage:1.21 pH:7.11 pH:246.83
06:32:22.224 -> Voltage:1.20 pH:7.09 pH:246.50
06:32:23.217 -> Voltage:1.21 pH:7.13 pH:247.50
06:32:24.243 -> Voltage:1.20 pH:7.03 pH:245.00
06:32:25.235 -> Voltage:1.21 pH:7.15 pH:247.83
06:32:26.228 -> Voltage:1.21 pH:7.13 pH:247.50
06:32:27.254 -> Voltage:1.21 pH:7.11 pH:247.00
06:32:28.247 -> Voltage:1.21 pH:7.11 pH:247.00
06:32:29.273 -> Voltage:1.21 pH:7.11 pH:246.83
06:32:30.265 -> Voltage:1.20 pH:7.10 pH:246.67
06:32:31.258 -> Voltage:1.21 pH:7.12 pH:247.17
06:32:32.284 -> Voltage:1.20 pH:7.09 pH:246.33
06:32:33.277 -> Voltage:1.20 pH:7.07 pH:246.00
06:32:34.270 -> Voltage:1.20 pH:7.10 pH:246.67
06:32:35.263 -> Voltage:1.20 pH:7.05 pH:245.33
06:32:36.289 -> Voltage:1.21 pH:7.11 pH:247.00
06:32:37.281 -> Voltage:1.21 pH:7.15 pH:248.00
06:32:38.274 -> Voltage:1.21 pH:7.15 pH:247.83
06:32:39.300 -> Voltage:1.20 pH:7.07 pH:245.83
06:32:40.293 -> Voltage:1.21 pH:7.14 pH:247.67
06:32:41.286 -> Voltage:1.21 pH:7.13 pH:247.33
06:32:42.313 -> Voltage:1.21 pH:7.15 pH:248.00
06:32:43.305 -> Voltage:1.20 pH:7.10 pH:246.67
06:32:44.303 -> Voltage:1.20 pH:7.05 pH:245.50
06:32:45.328 -> Voltage:1.20 pH:7.05 pH:245.33
06:32:46.321 -> Voltage:1.20 pH:7.07 pH:245.83
06:32:47.313 -> Voltage:1.20 pH:7.05 pH:245.50
06:32:48.339 -> Voltage:1.19 pH:7.01 pH:244.50
06:32:49.332 -> Voltage:1.19 pH:7.00 pH:244.33
06:32:50.324 -> Voltage:1.20 pH:7.08 pH:246.17
06:32:51.350 -> Voltage:1.20 pH:7.03 pH:245.00
06:32:52.342 -> Voltage:1.20 pH:7.06 pH:246.17
06:32:53.335 -> Voltage:1.20 pH:7.06 pH:245.67
06:32:54.328 -> Voltage:1.21 pH:7.11 pH:247.00
06:32:55.354 -> Voltage:1.21 pH:7.16 pH:248.67
06:32:56.347 -> Voltage:1.21 pH:7.11 pH:246.83
06:32:57.339 -> Voltage:1.21 pH:7.12 pH:247.17

Global variables use 240 bytes (12% of dynamic memory, leaving 1800
Autoscroll Show timestamp
Newline
```

See Appendix pg 112.

pH 10:



See Appendix pg 113.

Calibrating the pH Sensor

12/3/24

PH Solution	PH Sensor Output 1 (pre-commenting out the existing conversion code)	PH Sensor Output 2	Sensor Calibrated (trial 2)
4.0	2.86	170.54	3.99
7.0	4.19	244.31	
10.0	7.17	323.58	10.229706

Trial 1:

$$y=mx+b, y=2.2556x - 6.1624$$

$$m: \Delta y / \Delta x = (7-4)/(4.19-2.86) = 3/1.33 = 2.2556$$

$$b: 2.86 - 2.2556(4) = -6.1624$$

Trial 2:

$$y=mx+b, y=2.2556x - 6.1624$$

$$m: \Delta y / \Delta x = (7-4)/(4.19-2.86)$$

$$b = y - mx: 2.86 - 2.2556(4) = -6.1624$$

(170.54,4) and (244.31,7)

$$y=0.0407x-2.94$$

(170.54,4) and (323.58,10)

See Appendix pg 113.

Initially, the equation converting the analog data to millivolt was multiplied by 5 which represents the voltage the sensor is using. However, my sensor was plugged into the 3.3V pin, so I changed the equation to be multiplied by a factor of 3.3 instead of 5, and I got readings closer to the buffer solution.

12/10/24: with the conversion from 3.3V (voltage=avgValue*3.3/1024/6)

Solution	PH Tester stick	PH Sensor Output (average of 60 values)	Sensor Calibrated (average)
4.0	4.01	170.14	3.995
7.0	7.01	246.90	6.99
10.0	9.98	319.21	9.91

y = pH solution, x = sensor readings

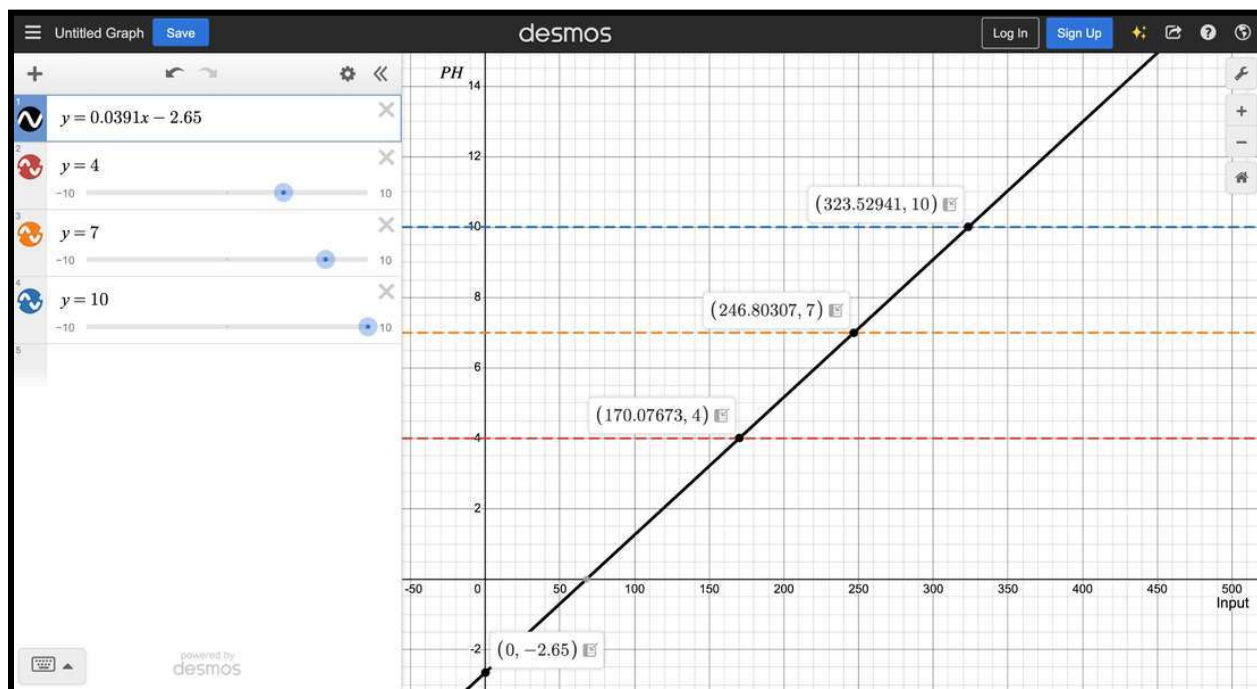
y = m*x + b: **y = 0.0391x - 2.65**

m: $\Delta y / \Delta x = (7-4)/(246.90-170.14) = 0.0391$

b = y-mx: $4-0.0391(170.14) = -2.65$

See Appendix pg 114.

On my third trial, after changing the multiplication factor and taking an average of multiple points for a more accurate representation, I received values very close to the buffer solutions I was testing.



See Appendix pg 114.

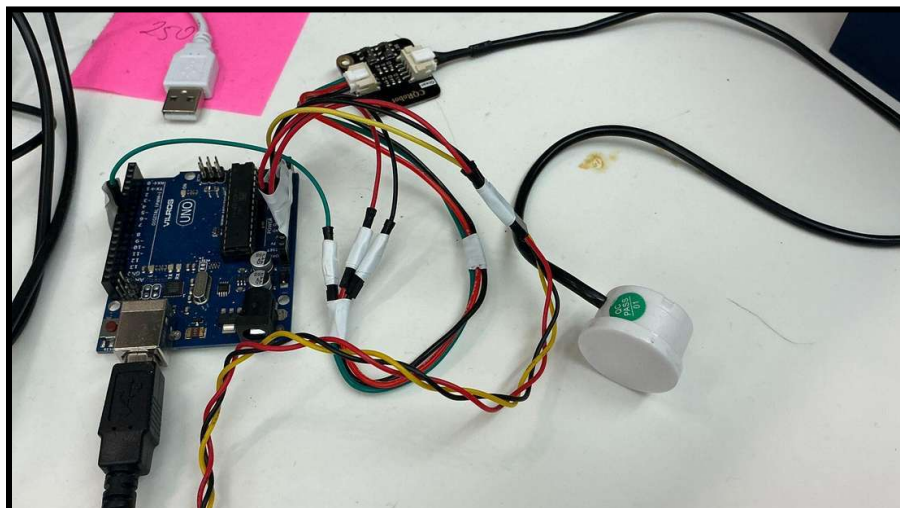
To illustrate the conversion of sensor data to a pH value, I graphed the equation I determined, where y is the pH solution and x is the sensor reading. When testing true a pH of 4, the sensor should output 170.07673, which would then be converted into a pH value 4. However, due to some variability in the sensor, it recorded 170.14, a pH of 3.995- almost perfect.

Water Level Sensor

The project uses the CQ Robot non-contact liquid level sensor to monitor water levels in the system. This sensor offers a wide measurement range, high accuracy, and a fast response time, making it ideal for detecting varying water levels in a reservoir. It works by emitting an ultrasonic signal and measuring the reflection to determine the presence of water. The sensor is connected to the Arduino through a digital input pin, and its analog output is processed after passing through an ADC converter. The sensor requires only a 3.3V power supply from the Arduino, and the ADC converts the analog signal into a digital value that the Arduino can interpret.

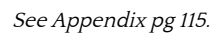
The sensor detects the water level by sensing the presence or absence of water, outputting a boolean value (HIGH or LOW). This value is then translated into a string value, "Water Detected" or "Water Level Low," for display purposes. Measurements are updated every 2 seconds, though this interval is adjusted when combined with the pH sensor code to optimize resource usage. The app displays real-time water levels, and the system responds by alerting the user if the water level drops too low, prompting them to add more water.

I did not encounter any issues integrating this sensor into the system. However, one potential issue that could arise is interference caused by water turbulence, which may affect the sensor's accuracy.



See Appendix pg 115.

Here is the water level sensor connected to the Arduino. Information is sent through the digital converter, so the data wire can be connected to the digital port.



70

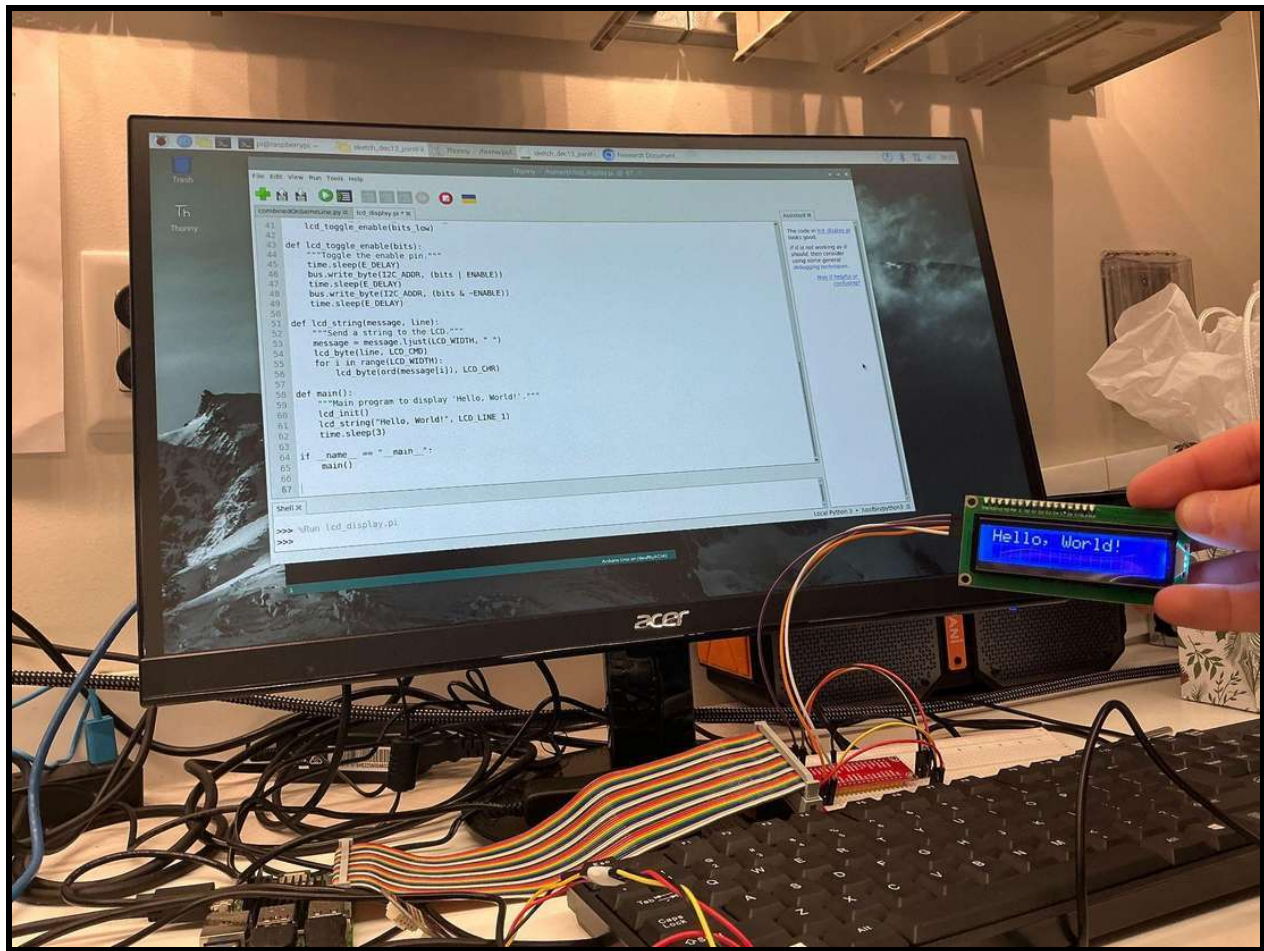
LCD Display

The project uses a 16x2 LCD with a resolution of 16 characters by 2 lines, powered by 5V from the Raspberry Pi. It utilizes an I2C module for simplified wiring and communication, reducing the number of required pins and making integration with the Raspberry Pi more efficient. The I2C module is wired to the 5V pin for power and to the SDA and SCL pins for data transmission, ensuring smooth communication between the Raspberry Pi and the display.

The programming language used for the LCD control is Python, with the LCD_I2C library handling the communication between the Raspberry Pi and the display. The program is written to display real-time sensor data, such as water temperature (in both Celsius and Fahrenheit) on the top line, and sensor data from the Arduino (pH, voltage, and water level) on the bottom line. Initially, readings were updated every 10 seconds for testing purposes, but now they are updated every minute to optimize performance and prevent overloading the system.

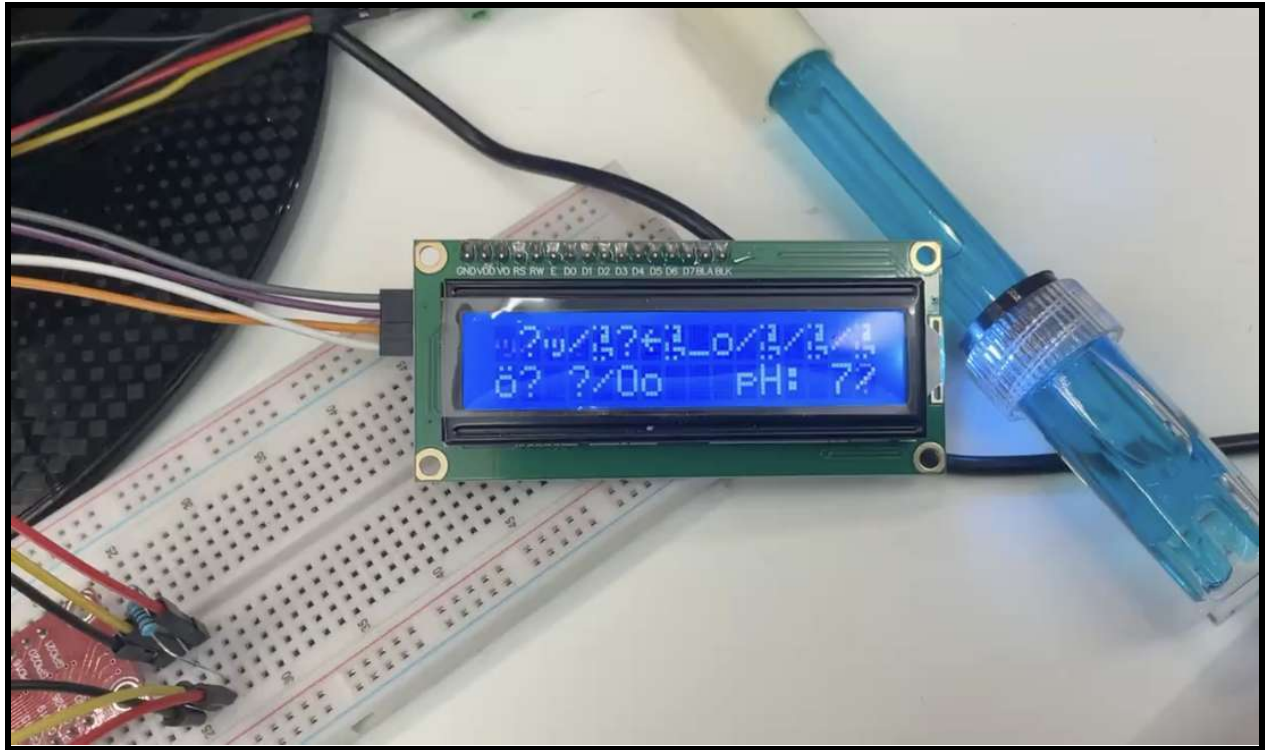
The data is formatted for easy readability, with the water temperature displayed as fixed values on the top row and the other sensor readings scrolling on the bottom row. The scrolling feature ensures that multiple pieces of data are displayed sequentially without overwhelming the user interface. The LCD also includes a backlight, which can be controlled to ensure clear visibility in different lighting conditions.

Initially, the LCD was connected to the 3.3V pin on the Raspberry Pi, which resulted in inadequate power for the display. This caused communication issues and incorrect character display. The problem was resolved by switching the power source to the 5V pin, which provided the necessary power for proper operation. Another issue arose when attempting to scroll the data on the bottom row of the display. Random symbols appeared, and the display seemed to jump around unpredictably. This was addressed by adjusting the timing between data updates and ensuring synchronization between the display and the data readings. After these adjustments, the scrolling feature worked as expected, and the display now presents the data clearly and accurately.



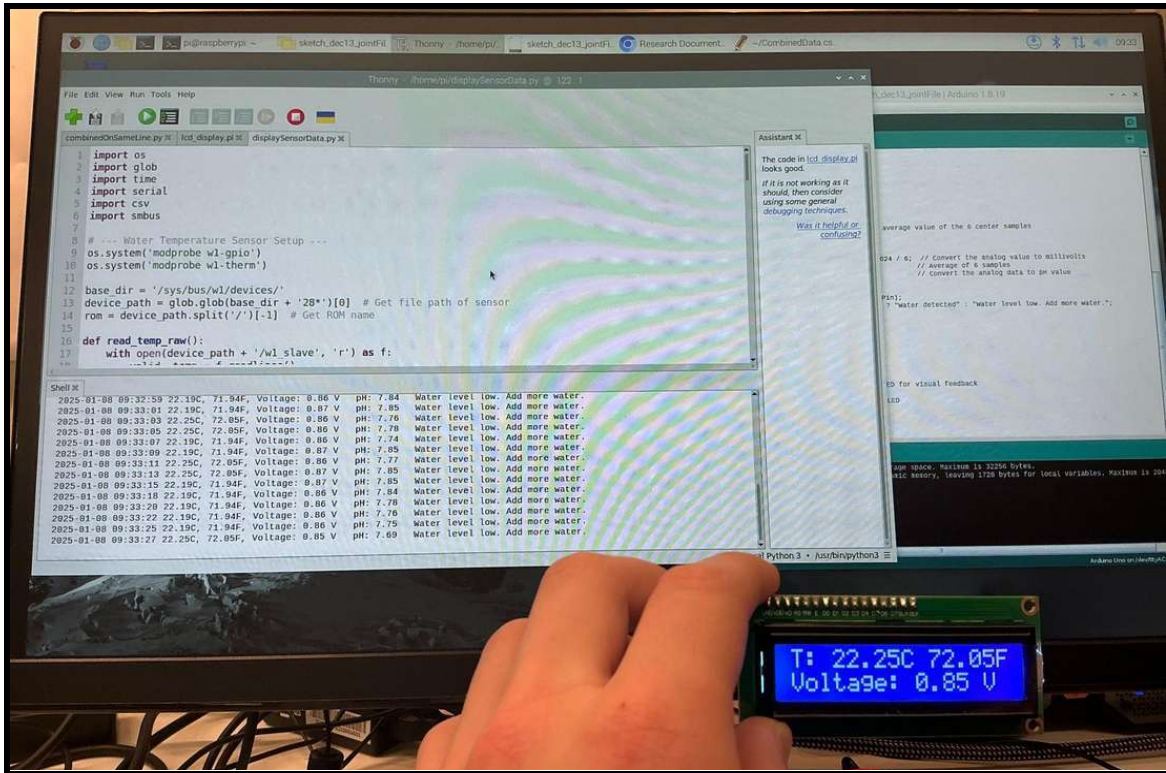
See Appendix pg 116.

This image shows the connection between the LCD and the Raspberry Pi via the GPIO extension board, in addition to the first success getting text to print on the LCD screen. For this, I used a separate Thonny file to test that the display works, and later, I made a new file that would display all the data.



See Appendix pg 116.

Here, when I tried to send the sensor data to the display, I received indecipherable symbols. The first attempt was a success where the first line included temperature readings and the second line included static arduino code. However, when I tried to get the arduino data to scroll so you can see the entire string, these symbols flooded the screen. I first checked the physical connection which was fine. The issue was syncing the data readings and sending that data to the LCD. The display was forced to process too much at once because the data was not synced, so once I added pauses in my code, the symbols did not appear, and the LCD worked as intended.



See Appendix pg 117.

This is an image of the LCD working as intended, and the data on the display matched the data in the output (only displaying the most recent set of data).

```
# Display temperature on Line 1
lcd_string(f"T: {sensor_data['temperature_C']}C
{sensor_data['temperature_F']}F", LCD_LINE_1)

# Scroll Arduino data on Line 2
arduino_data_string = f"V: {sensor_data['voltage']} pH: {sensor_data['pH']}
WL: {sensor_data['water_level']}"
start_time = time.time()
while (time.time() - start_time) < 30:
    lcd_scroll_string(arduino_data_string, LCD_LINE_2)

time.sleep(30) # Adjust this to allow sync
```

See Appendix pg 154.

This section of code, taken from my final Python program, sends the sorted sensor data to the LCD display. The first line calls the `lcd_string` function to print the temperature data, while the second part uses `lcd_scroll_string` to display and scroll the Arduino data on the second line for 30 seconds.

Raspberry Pi

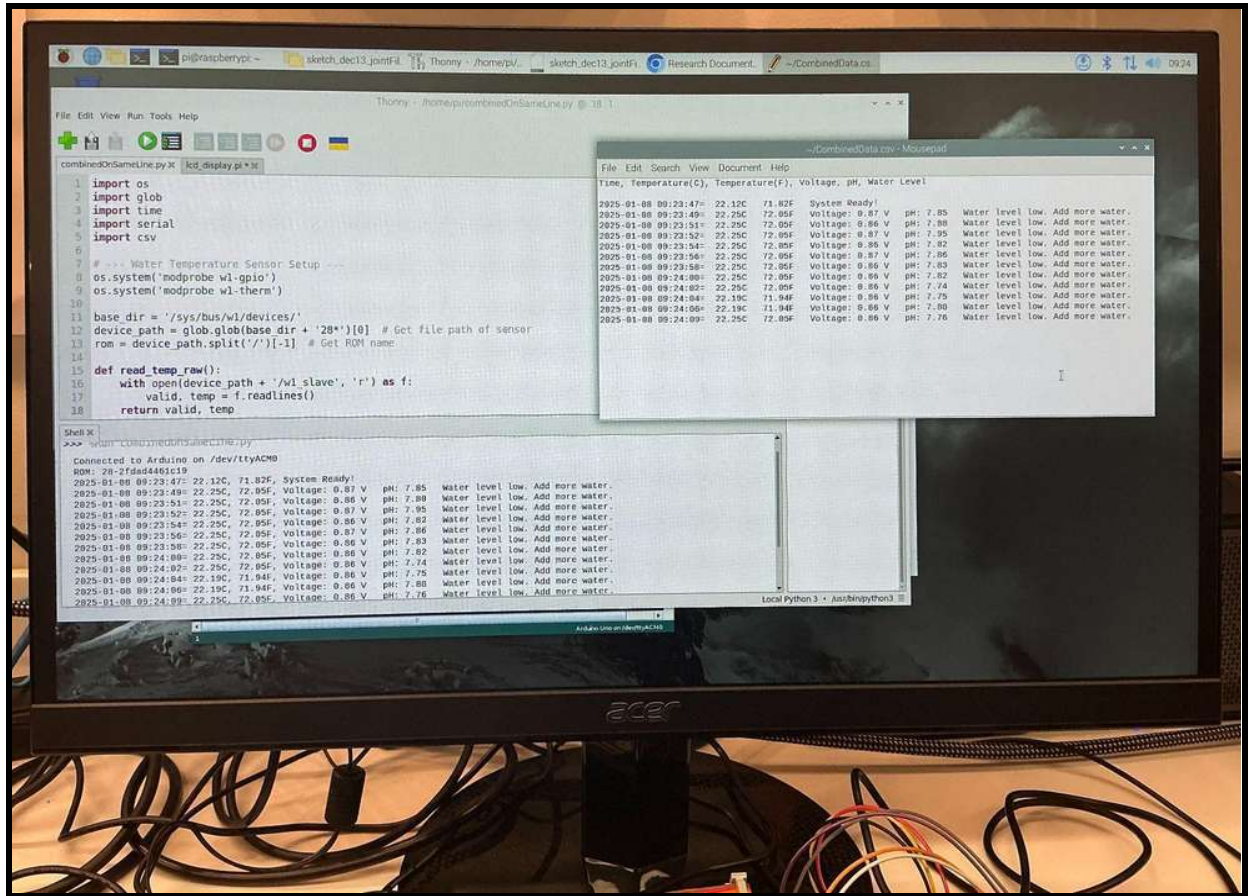
The Raspberry Pi serves as the central brain of my hydroponic project, managing sensor data processing, automation, and communication with the mobile app. It connects to key sensors—water temperature, water level, and pH—through GPIO pins, I2C communication, and an ADC module for analog inputs. It also reads data from the Arduino, which transmits sensor values over serial communication. A Python script continuously processes this data, logging it for analysis and writing it to a CSV file for historical tracking and external processing.

On the software side, the Raspberry Pi runs Raspberry Pi OS, with Python handling data collection, automation, and cloud integration. Libraries such as `smbus` for I2C communication and other sensor-specific modules facilitate accurate readings. Sensor readings are uploaded to Firebase for cloud integration, enabling real-time monitoring on the mobile app. The Raspberry Pi reads data from the Arduino using serial communication, processes sensor values and writes them to a log file for record-keeping and later analysis.

Using headless technology to access the Raspberry Pi, interactions can occur remotely via SSH. This setup allows for efficient system maintenance and updates without needing to hook up to the desktop. To ensure that the data collection and processing scripts run consistently, I use `crontab` to schedule them. The program is set to run from 6 AM to 9 PM daily, aligning with the system's operational hours to conserve power and resources when monitoring is unnecessary.

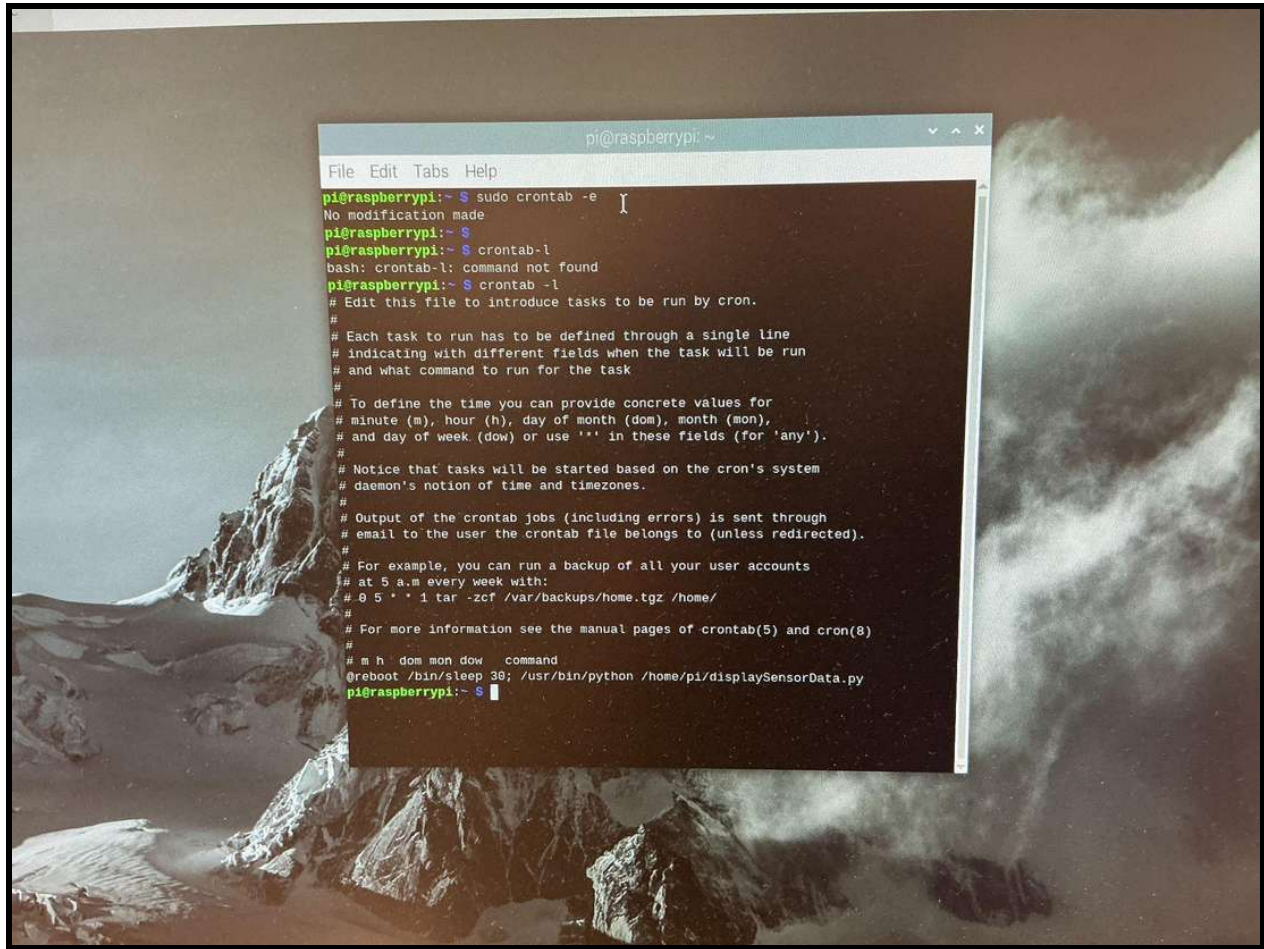
For real-time data visualization and mobile integration, the Raspberry Pi pushes sensor readings to Firebase, a cloud database that syncs with the mobile app. This setup allows users to access live sensor data from anywhere, ensuring they can monitor and adjust conditions as needed. Using Firebase also simplifies data management, as it eliminates the need for complex backend infrastructure while providing reliable and scalable cloud storage.

Throughout development, I encountered challenges such as GPIO conflicts, sensor calibration issues, and occasional connectivity problems. To ensure reliable system performance, I addressed these through debugging, careful wiring, and optimizing network stability. Another challenge was handling multiple sensor readings efficiently for display on the LCD. To improve readability, I concatenated the pH level, voltage, and water level data into a single line, allowing for smoother scrolling and preventing display glitches.



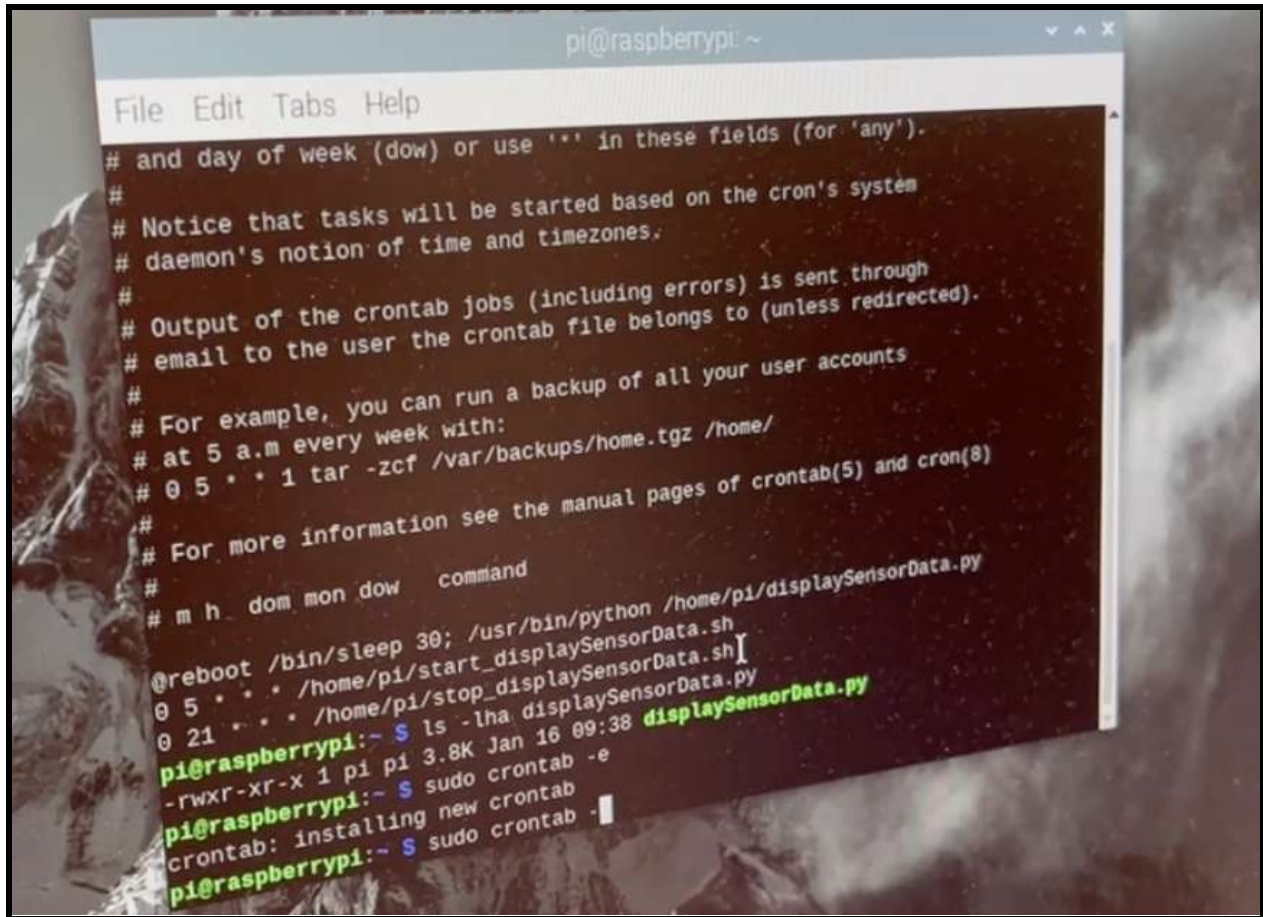
See Appendix pg 117.

This image shows the successful implementation of the .CSV file. Here, the temperature data and the Arduino data (voltage, pH, and water level) are all written to the same line, with the same timestamp.



See Appendix pg 118.

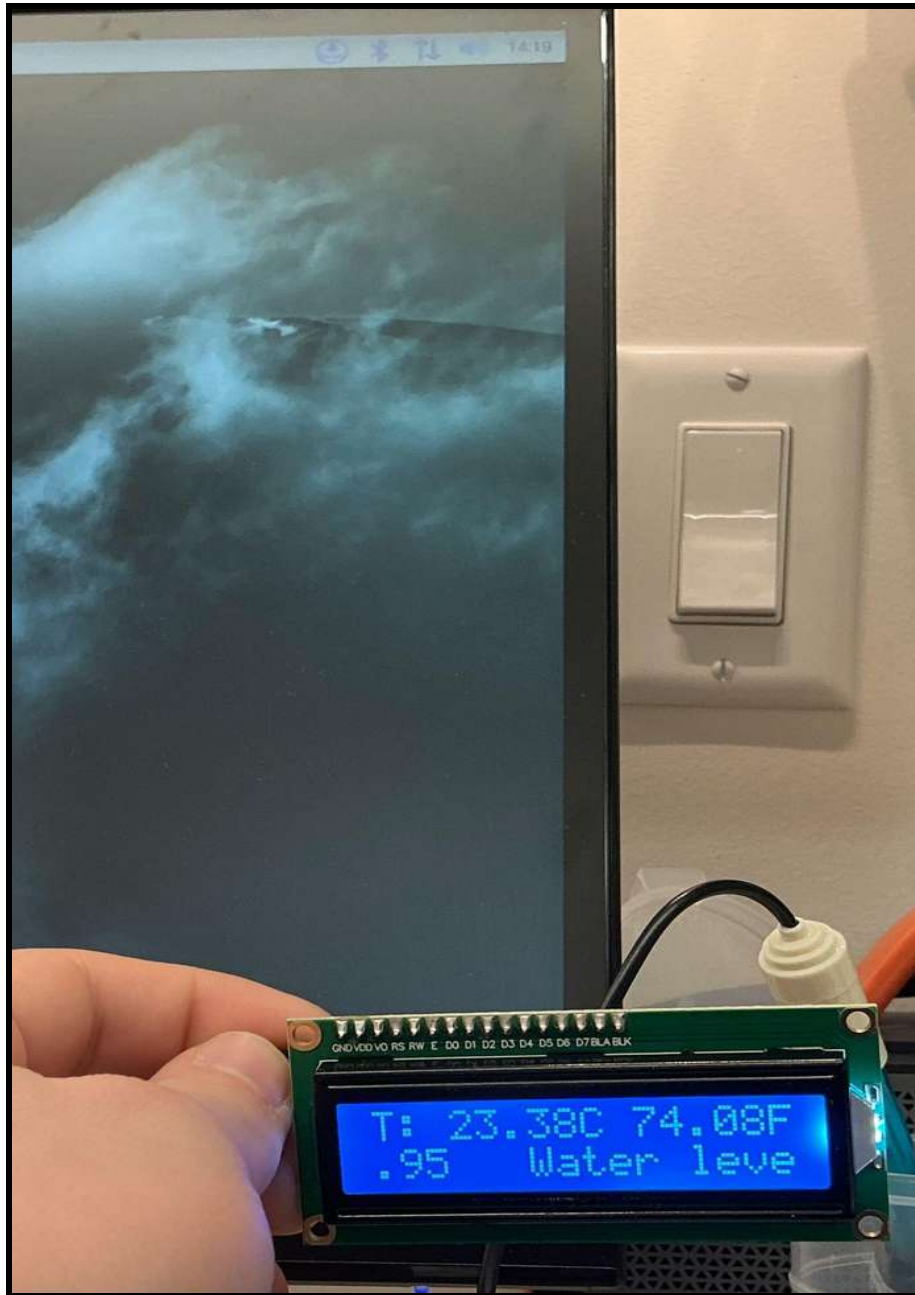
This crontab entry ensures that my `displaySensorData.py` script runs automatically when the system starts up again after a 30-second delay. `sudo crontab -e` opens the root user's crontab for editing, allowing me to schedule tasks that run with administrative privileges. This is where I coded my Raspberry Pi to run without connecton to the monitor and a “run” button to press. Once I saved and exited crontab, I can then typed `sudo crontab -l` which lists the crontab entries. The `@reboot` directive schedules the task to execute once after booting, while `/bin/sleep 30;` pauses execution to allow the system to stabilize before running the script using Python. This setup is useful for automating sensor data display without manual control.



```
pi@raspberrypi: ~  
File Edit Tabs Help  
# and day of week (dow) or use '*' in these fields (for 'any').  
#  
# Notice that tasks will be started based on the cron's system  
# daemon's notion of time and timezones.  
#  
# Output of the crontab jobs (including errors) is sent through  
# email to the user the crontab file belongs to (unless redirected).  
#  
# For example, you can run a backup of all your user accounts  
# at 5 a.m every week with:  
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/  
#  
# For more information see the manual pages of crontab(5) and cron(8)  
#  
# m h dom mon dow   command  
@reboot /bin/sleep 30; /usr/bin/python /home/pi/displaySensorData.py  
0 5 * * * /home/pi/start_displaySensorData.sh  
0 21 * * * /home/pi/stop_displaySensorData.sh  
pi@raspberrypi:~$ ls -lha displaySensorData.py  
-rwxr-xr-x 1 pi pi 3.8K Jan 16 09:38 displaySensorData.py  
pi@raspberrypi:~$ sudo crontab -e  
crontab: installing new crontab  
pi@raspberrypi:~$ sudo crontab -l
```

See Appendix pg 118.

In this crontab session, I coded my file `displaySensorData.py` with file path `/home/pi/displaySensorData.py` to run on a schedule. The program starts at 6 AM every day and ends at 9 PM every night. This cycle then repeats every day. The code for this are the two lines: `0 5 * * * /home/pi/start_displaySensorData.py` and `0 21 * * * /home/pi/stop_displaySensorData.py` (in programming you start counting at 0).



See Appendix pg 119.

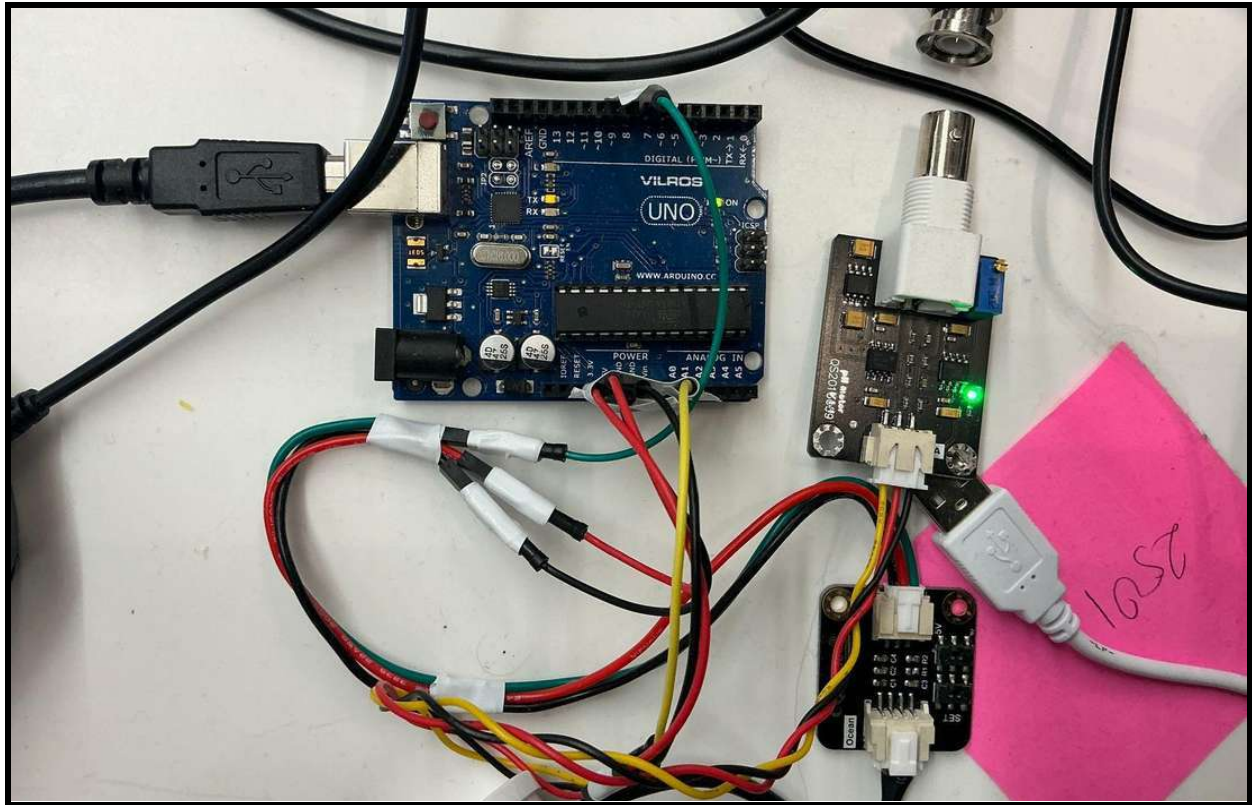
Here, you can see that this schedule works: to test this, I changed the end time to 14:19 as I was in class. The LCD instantly stopped scrolling when it was 14:19.

Arduino

The Arduino acts as a microcontroller for sensor data collection, specifically handling the water level and pH sensors due to its analog input capabilities. While the Raspberry Pi manages higher-level processing and cloud integration, the Arduino is responsible for monitoring the water level and pH sensors and sending the data to the Raspberry Pi.

For hardware setup, the Arduino is connected to the water level and pH sensors via its analog and digital input pins, allowing it to capture continuous sensor data. It communicates with the Raspberry Pi via serial communication (UART) to transmit sensor readings for further processing and cloud storage. The Arduino is programmed using the Arduino IDE in C++, utilizing core functions for accurate readings and Serial communication libraries for seamless data transfer. The Arduino IDE's built-in Serial Monitor displays real-time sensor data without requiring an external CSV file for storage. This feature simplifies debugging and allows immediate observation of water level, pH, and voltage readings, ensuring data accuracy before integration into the larger system.

One of the main challenges was efficiently handling and displaying pH level, voltage, and water level data on the LCD. Since the LCD has limited space, displaying all three values simultaneously led to formatting issues. To address this, I concatenated all three sensor readings into a single line of text, which scrolls smoothly on the LCD's bottom row. This approach ensures that all key data points are displayed without cluttering the interface while maintaining readability.



See Appendix pg 119.

The Arduino Uno microcontroller in this image is connected to both the water level sensor, which uses a digital pin, and the pH sensor, which uses an analog pin. A USB cable connects the Arduino to the Raspberry Pi, allowing communication between the two processors.

Developing The App for User Interaction and Data Monitoring

This section will describe the development of the mobile application that allows users to interact with the hydroponic system. The app will display real-time sensor data, including water temperature, water level, and pH level, and provide the user with the ability to control the system remotely, such as adjusting water flow or triggering alerts. The app is a crucial interface for the user to monitor and control the system's environment at any time.

Introduction to the App Development

The app serves as the bridge between the user and the hydroponic system, enabling them to monitor the system's status, receive alerts, and make adjustments based on real-time sensor data. Through this section, I will explore how the app is designed to be user-friendly, ensuring that users can easily interpret sensor data and take appropriate actions, and the code that makes the app run properly.

Essential Materials

- Raspberry Pi
- Thunkable App Development Platform
- Firebase

Stages

- App Design: Design wireframes and user interface for the app.
- Backend Setup: Set up Firebase or cloud service to handle sensor data storage and real-time updates.
- App Development: Develop the app to display real-time data, control settings, and alert users.
- Integration: Connect the app with the sensor data and automation system.

Frontend (User interface)

The front end of the hydroponic system is the mobile application that enables users to interact with and monitor their system. Its primary purpose is to display real-time sensor data, provide control over automation features, and offer alerts and notifications related to system conditions. The app is built using Thunkable, allowing users to easily navigate the system and manage various aspects of the hydroponic setup.

The app consists of five screens, starting with the home screen that serves as the navigation hub to access other screens in addition to important alerts like the sensors recording data outside of the desired range, or the water is too low. The second screen is the water management screen, which displays important data like water temperature, pH level, voltage, and water level. It also allows the user to adjust the set water temperature, ensuring optimal growing conditions. The third screen is the lighting control screen, where users can see whether the lights are on or off, modify the light cycle, and set a new schedule for the lights. The fourth screen is a notes app, allowing users to log observations, and track the system's progress. The final screen is the resources page, which provides helpful links for growing plants and troubleshooting. Users can also add or delete links, ensuring the app remains up-to-date with the latest information.

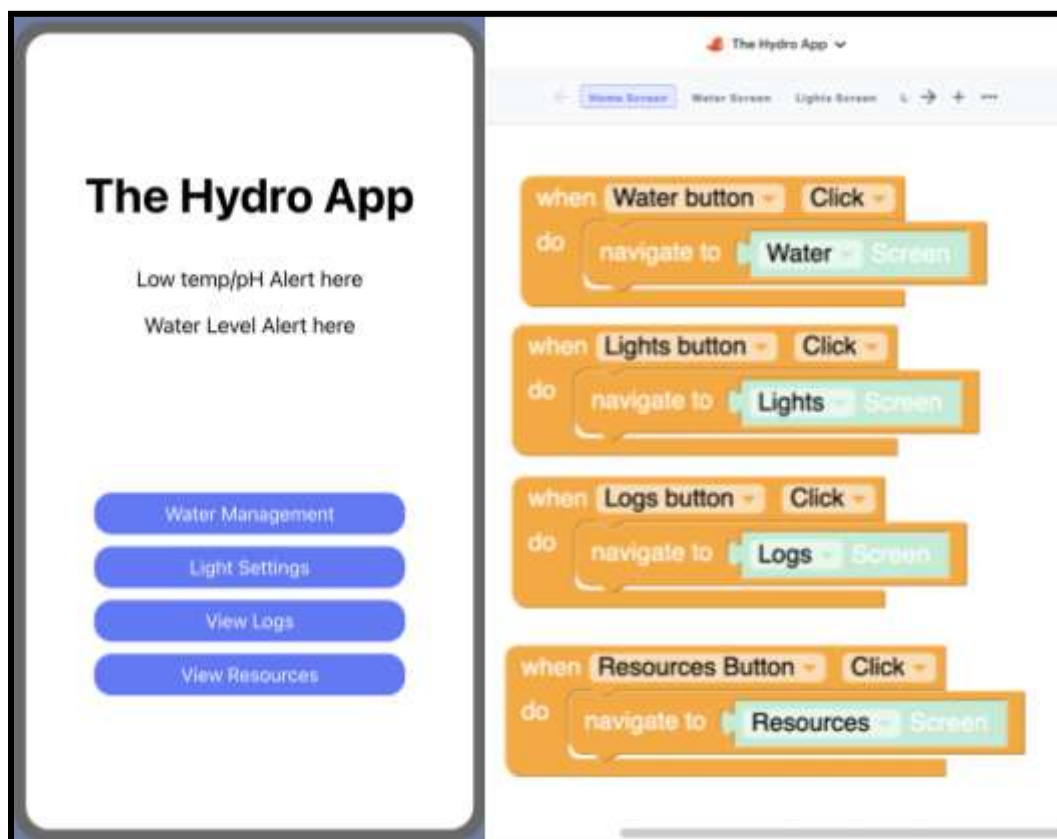
The user interface is designed with simplicity in mind to ensure ease of navigation and accessibility. The app is compatible with different devices and screen sizes, one reason I chose to use Thunkable. Visual design elements like color coding are used to indicate sensor statuses, where green signals normal conditions and red alerts the user to any issues that require attention. This ensures that users can quickly and easily assess the state of their hydroponic system and make adjustments as needed.

Thunkable uses a visual programming interface that allows developers to create functionality through "blocks" of code instead of writing traditional lines of code. This approach makes it easy to build the app by dragging and connecting different blocks that represent actions, events, or data handling. For example, you can drag a block to retrieve data from Firebase, another to update the UI, and another to handle user input. The visual nature of blocks allows users to quickly find and modify components, speeding up the development process without requiring deep programming knowledge. This block-based system simplifies frontend development by enabling quick iteration and user interface design, making it particularly useful for app development without heavy coding. Although it involves components that

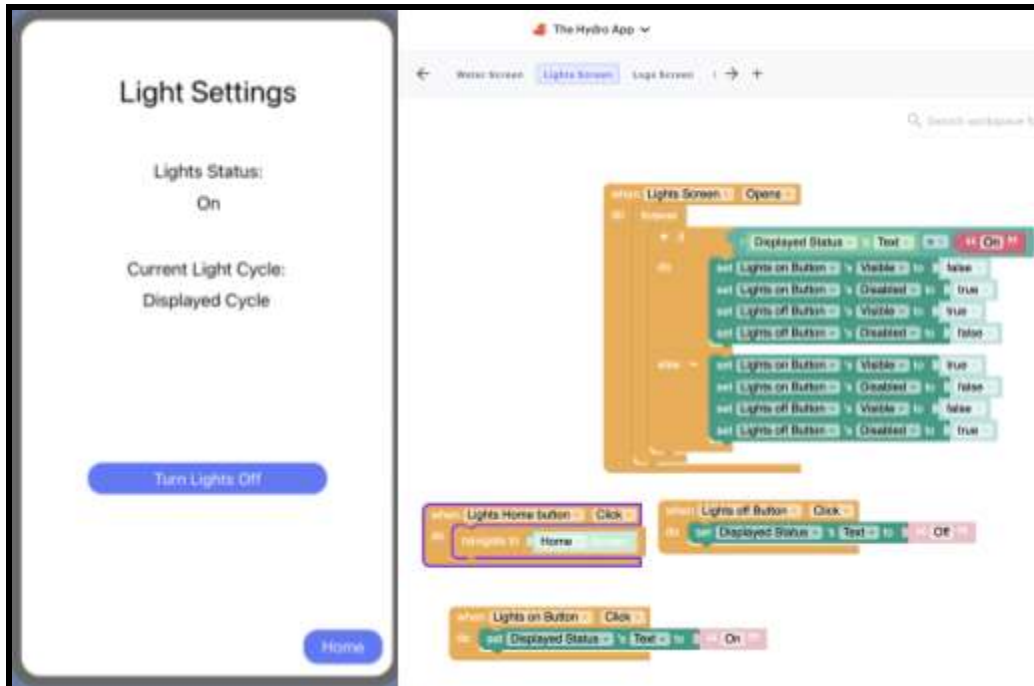
interact with the backend, such as Firebase, it is primarily concerned with the app's interface and logic.

Using blocks of code allows me to define specific actions when the user interacts with the app. For example, when the user presses a button, I can set up blocks to trigger certain behaviors, such as navigating to another screen or sending data to the backend. I also use lists to enable the user to create, view, and delete notes and resources, making it easy for them to manage their content within the app. Additionally, blocks are used to display real-time sensor data on the app, updating the screen with the latest information from the Raspberry Pi, and ensuring the user is always viewing the most current readings.

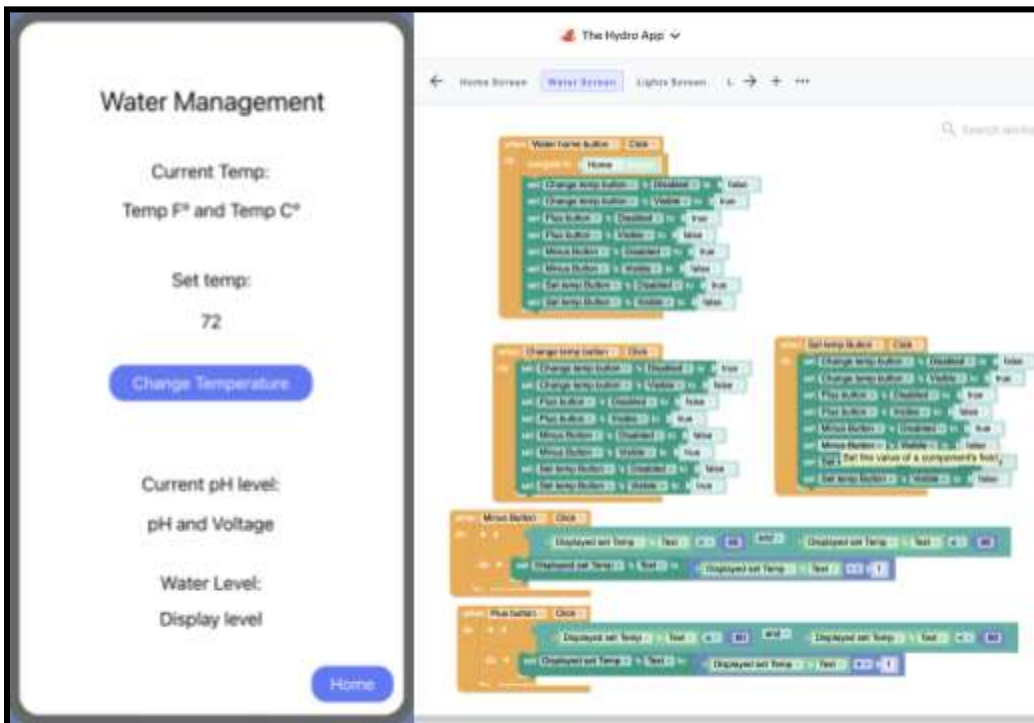
The following images include each screen in the app in addition to the code blocks that control the functions on that screen to allow for a simple user interface.



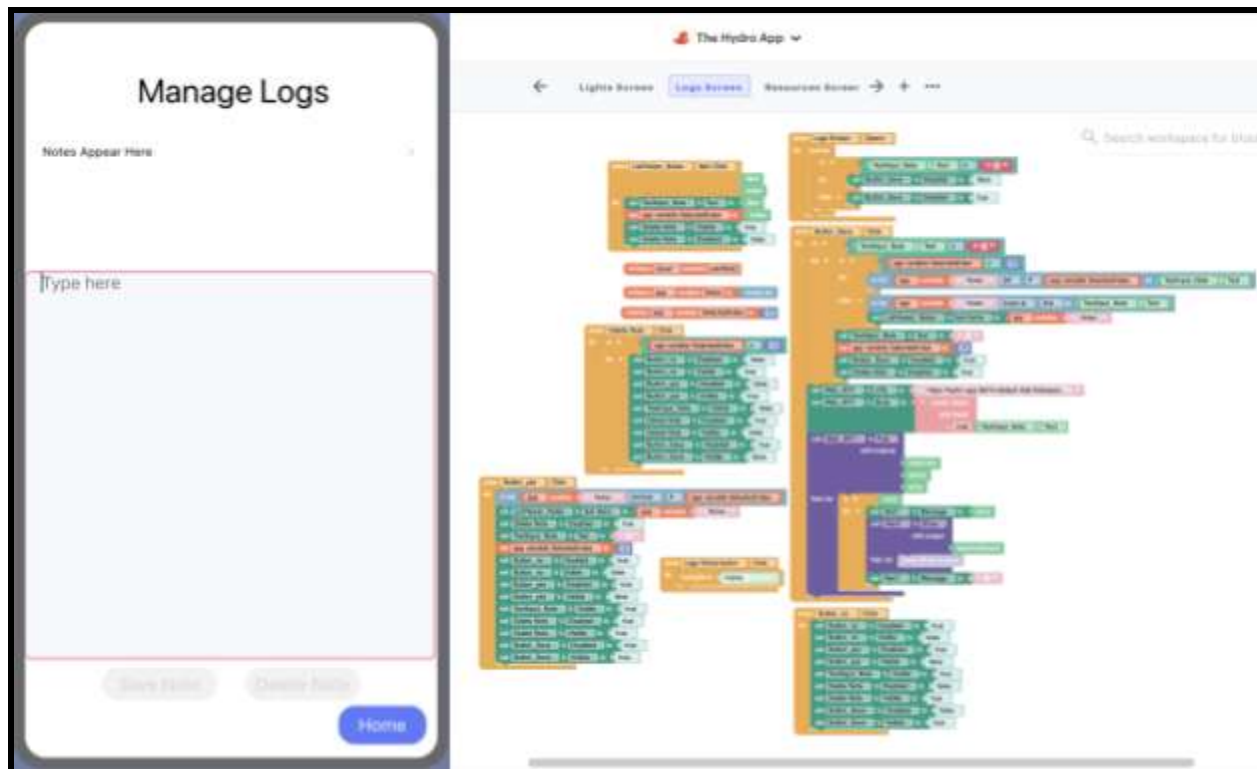
See Appendix pg 120.



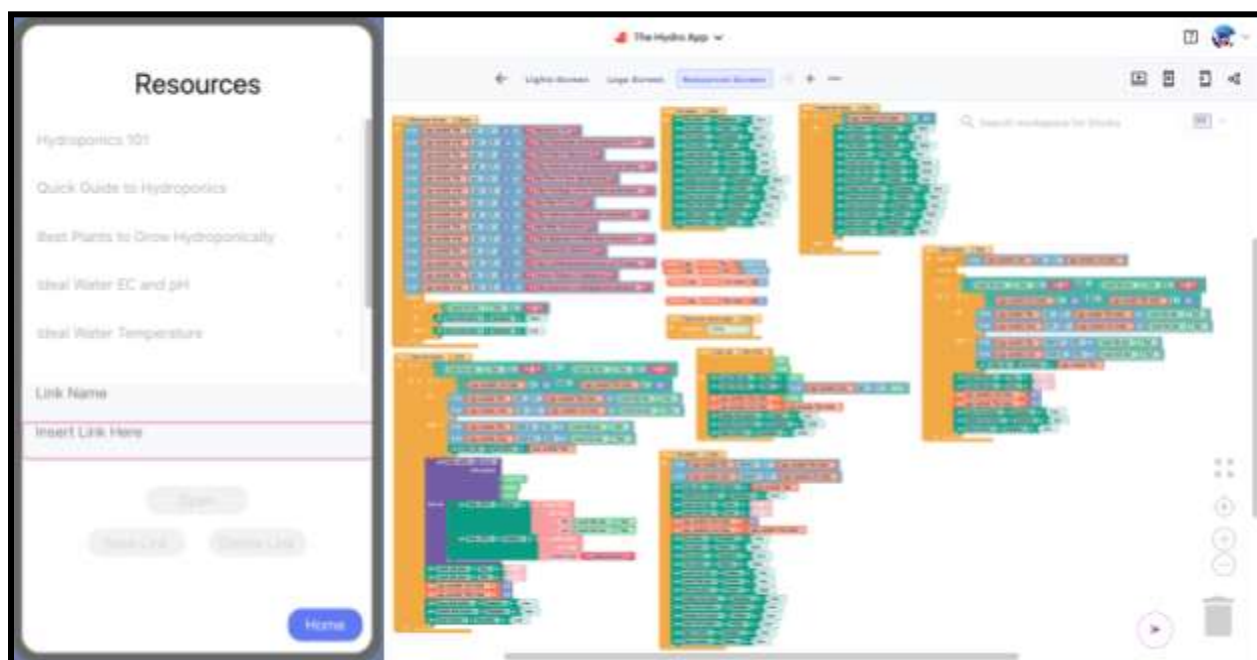
See Appendix pg 121.



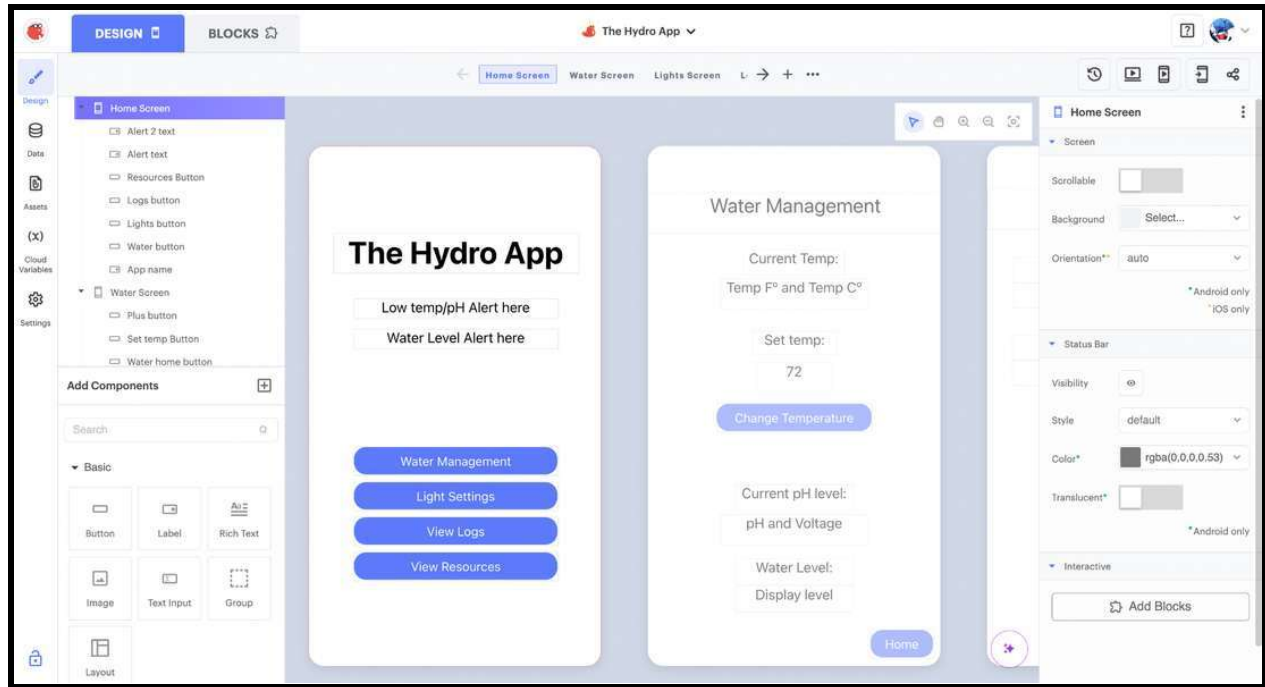
See Appendix pg 121.



See Appendix pg 122.



See Appendix pg 120.



See Appendix pg 122.

This is the design section of the Thunkable app, where I can arrange and format the elements of each screen before programming them in the blocks section.

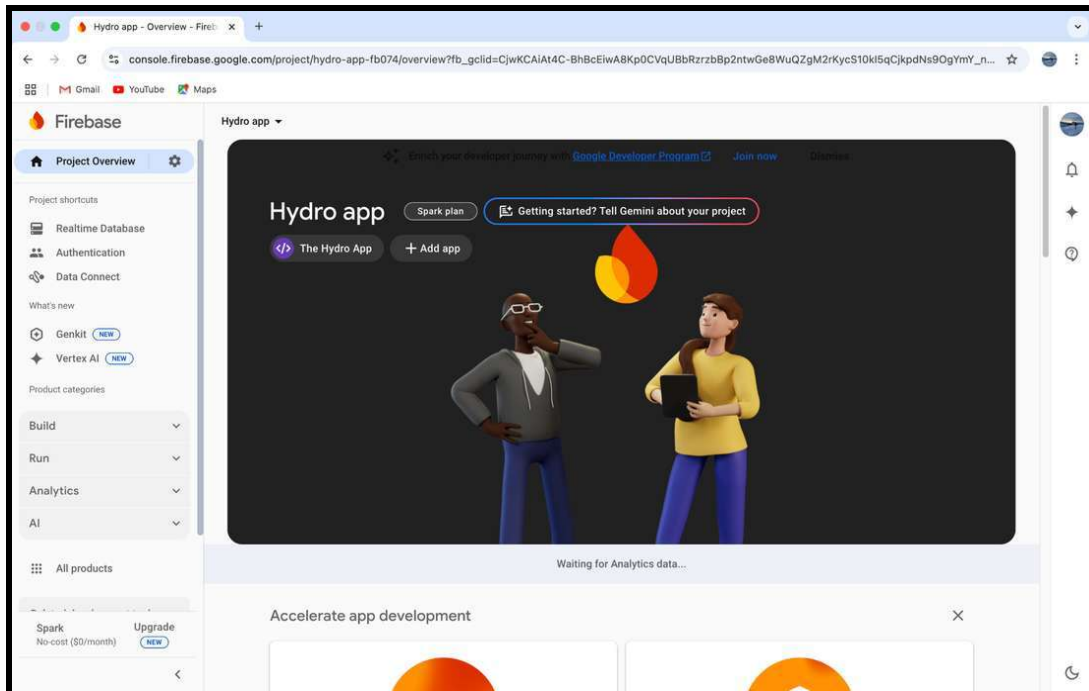
Backend (Data Processing and Automation):

The backend of the hydroponic system is responsible for processing sensor data, managing automation, and enabling communication between the Raspberry Pi, sensors, and the mobile app. It collects and analyzes sensor data, and then stores it in the cloud for easy access and further processing. This setup ensures efficient system operation and reliable data storage.

The Raspberry Pi serves as the microcontroller, reading data from various sensors such as water temperature, pH level, and water level. The backend processes this data, making decisions based on sensor readings. For example, if the water temperature rises above or falls below a set threshold, an alert is triggered to notify the user via the mobile app. Similarly, when the pH or water level sensors detect suboptimal conditions, the system sends a warning to the user's phone. At this stage, alerts ensure that the user can take appropriate actions to maintain the optimal environment for plant growth.

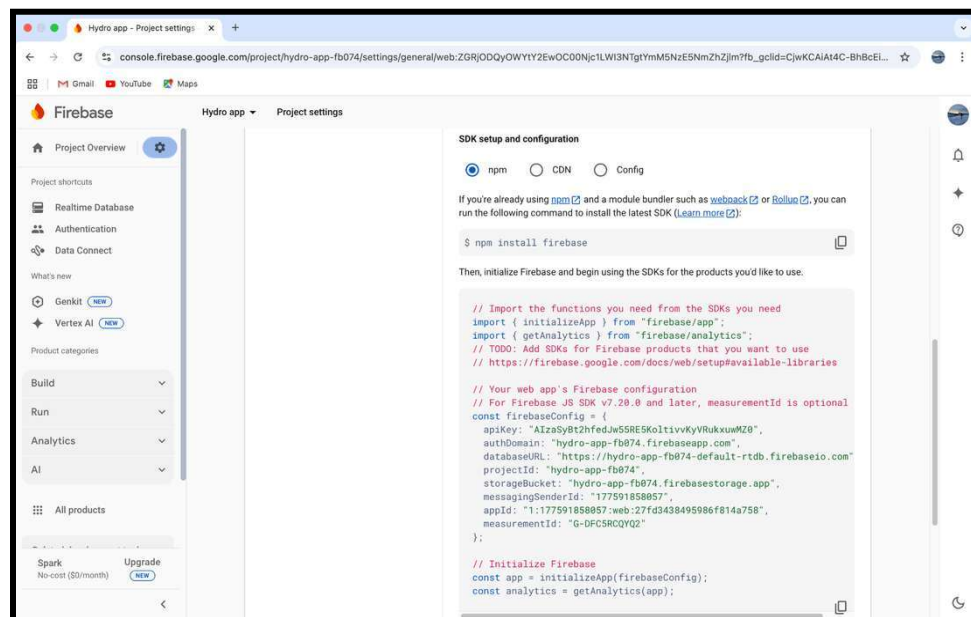
Data from the Raspberry Pi is transmitted to Firebase, which stores it in the cloud for secure, centralized access. Firebase uses HTTP for data transmission and WebSockets for real-time updates. This allows for seamless communication between the backend and front end, ensuring that the mobile app always displays the most up-to-date sensor data. In addition to sensor readings (such as water temperature, pH, and water level), Firebase also stores user-generated content, such as notes and resources added through the app. This means that whenever a user logs new notes or adds/deletes links in the resources section, those changes are saved in Firebase and are accessible in future app sessions. By storing both sensor data and user data in the cloud, Firebase ensures the app can retrieve and display the necessary information each time the user opens the app.

The backend also employs automation logic to manage system functions based on sensor readings. Although pumps and actuators are not yet integrated, the system can send real-time alerts when sensor readings fall outside of predefined thresholds. For example, when the pH or water level is abnormal, the system alerts the user, ensuring they can take corrective action quickly. To protect data integrity, encryption is used for secure data transmission, and fail-safes are implemented to handle issues like sensor malfunctions or network connectivity problems.



See Appendix pg 123.

This is the Firebase home page, the cloud-based storage platform I used in my project. From here, I can access my project's settings and configure how the storage is managed through code.



See Appendix pg 123.

This is the settings page where I can find the project ID, URL, and API key—unique identifiers that allow me to connect my app and Raspberry Pi to the Firebase cloud storage.

Physical Structure of the System and 3D-Printed Sensor Housing

In this section, I will discuss the development of the physical infrastructure of the hydroponic system, including the Nutrient Film Technique(NFT) system, and the materials used. Additionally, I will detail the design and 3D printing of the sensor housings, which are critical for ensuring that sensors are securely within the system

Introduction to Physical Structure

The physical setup of the hydroponic system provides the necessary environment for plant growth, while the 3D-printed sensor housing ensures that the electronics are protected and safely placed within the system. This section will describe the assembly and design considerations for the structure, including the integration of the sensors and other components.

Essential Materials:

- Computer-Aided Design(CAD) Platform
- 3D Printer
- PVC Pipe Structure

Stages:

- Design the Housing: Design 3D housing for sensors that protect them from water damage.
- Hydroponic Setup: Build the hydroponic system (PVC pipes, pumps, lights).
- Printing & Assembly: 3D print the housing for the sensors and assemble the system.
- Testing: Ensure all components fit well together and work as intended.
- Planting: Incubate plant seeds in a germination medium and transplant seedlings to a hydroponic system once sprouted
- Monitor: Ensure plants are growing and the system works as intended

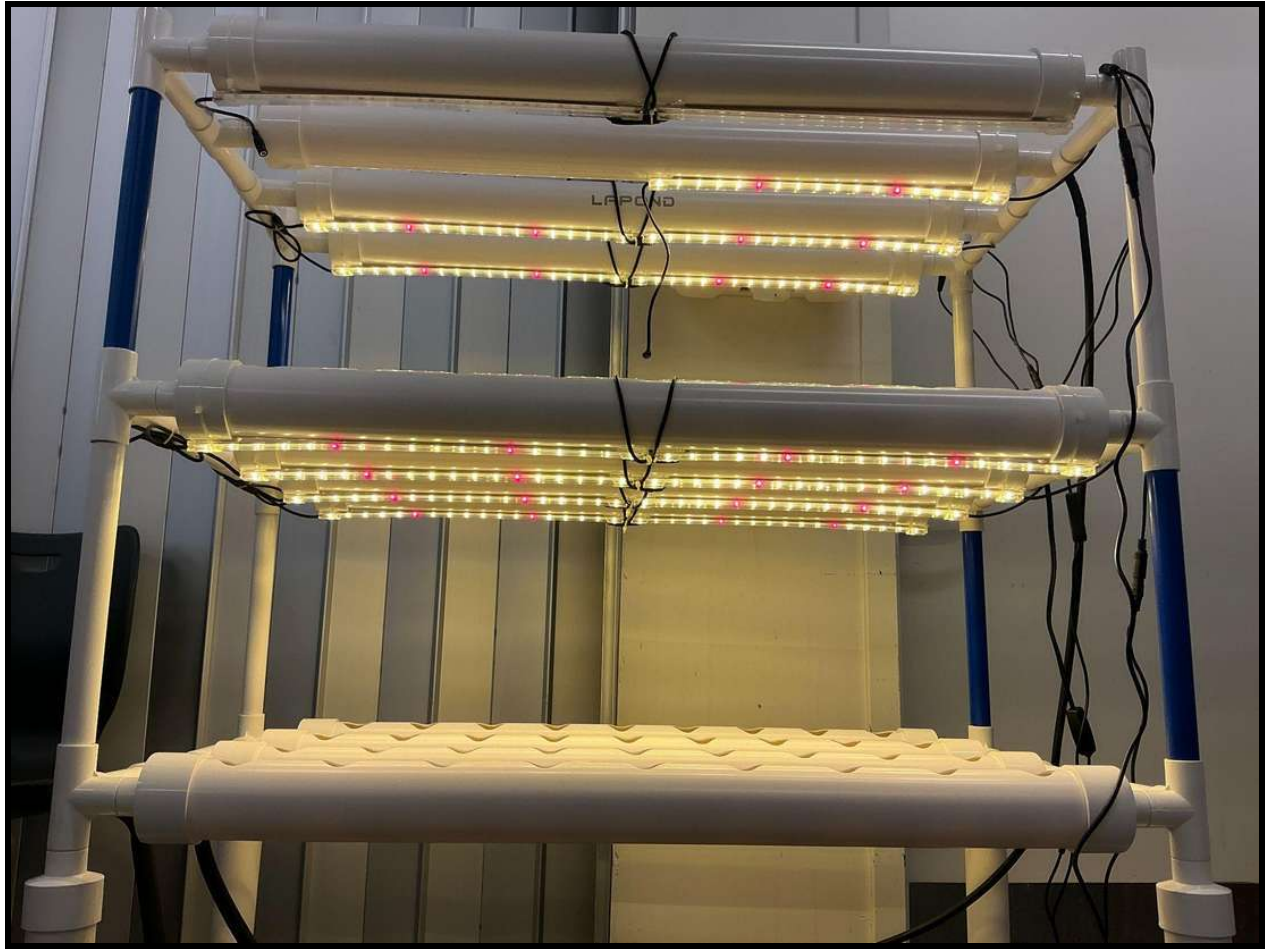
Hydroponic Structure:

The hydroponic system is a Vertical Nutrient Film Technique (NFT) setup, consisting of three layers of four PVC pipes for vertical plant growth. This system supports plant growth by providing a continuous, shallow flow of nutrient-rich water that nourishes plant roots. The integration of sensors and automation ensures that the environment is optimized for plant health.

Materials used include PVC pipes for the plant channels, a water tank for nutrient storage, and a pump for circulating the water through the system. Grow lights are also placed above each layer to provide uniform light distribution. Water moves through the system in a pump-driven circulation mechanism, allowing gravity to carry the water back down, ensuring even nutrient delivery to the plants. The water tank facilitates easy adjustments to the water composition, allowing for efficient management of nutrient levels and pH. This is essential for maintaining optimal conditions for plant growth, as it provides a centralized location for monitoring and modifying the water quality.

The 3D-printed sensor housing is securely mounted within the system to protect the electronics from water exposure. The overall structure is designed for stability, with reinforcement to prevent tipping. The system is also designed for future scalability, allowing for expansion with additional plant sites or sensors as needed.

No issues with leaks, water flow, or material durability were encountered, but design changes explained under “Future Development” could further improve efficiency, ease of maintenance, and automation for optimized plant growth.



See Appendix pg 124.

This is the completed structure with water in the tank and the pump circulating the water through all layers of the NFT system, and grow lights on the top two layers shining on the grow sites below. When assembling the lights, the wire to one of the strips fell off, cutting off the flow of electricity to 3 of the lights. Using a solder, I soldered the wires back to the light and finished attaching them to the channels.



See Appendix pg 124.

Here, the wires have been soldered back to the light and the strip was placed back into the NFT system. When the lights were plugged back into the outlet, everything worked as intended.

3D Printed Sensor Housing:

The 3D printed box serves as the protective enclosure for key components of the hydroponic system, including the sensors (water temperature, pH, water level), LCD, and electronic components such as the Raspberry Pi and wiring. Its primary role is to ensure that the sensors are securely housed, providing water resistance, durability, and proper positioning within the system. The housing ensures that all components are protected from environmental factors.

The housing was designed using Onshape, a 3D modeling software, to create a functional and durable enclosure. The structure is designed with holes, fits perfectly for the water level sensor and LCD. The design also includes ventilation features to prevent the Raspberry Pi and Arduino from overheating. The 3D model was exported as a .stl file and processed through Bambu Labs slicing software to prepare the print file. The printing itself was done on the Bambu Labs X1 Carbon with a 0.4mm nozzle and a textured PEI plate.

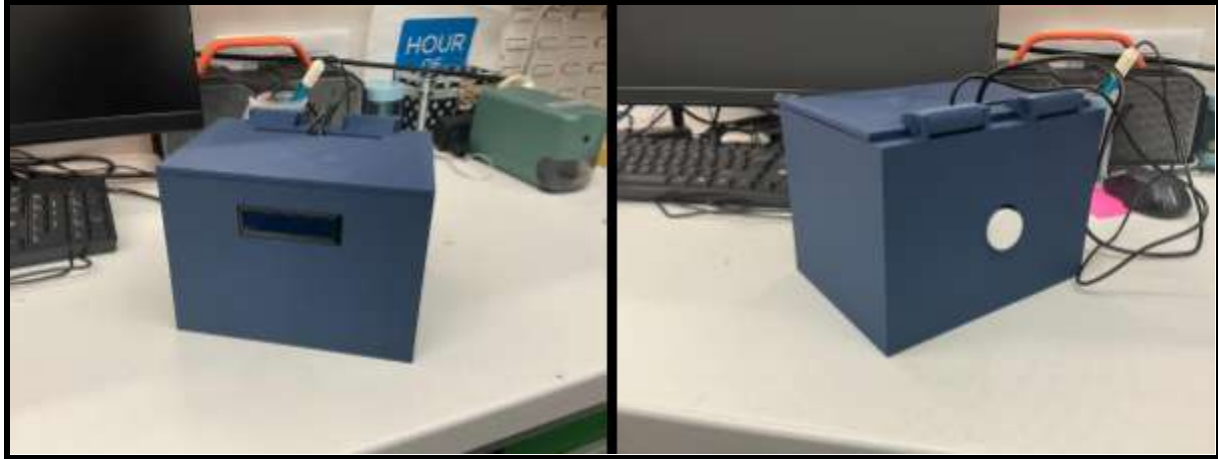
The filament used for printing the housing is Bambu PLA Basic. For the print settings, I used a layer height of 0.2mm, an infill percentage of 15%, and a moderate print speed to balance quality and time efficiency.

The initial print included shelves for the Raspberry Pi and Arduino, but the printer had difficulty printing supports for these shelves, so the design was revised. In the second print, I encountered an issue with the dimensions being off, specifically with the hinge connecting the top and bottom parts of the box. The hinge wouldn't rotate due to improper rounding of the design. Additionally, during the printing process, despite the box fitting within the plate dimensions, the print was failing. I had to scale the print by 90% in the slicer software to resolve this. However, this scaling caused the sensor cutouts to shrink, leading to the sensors not fitting properly. After some adjustments, the dimensions were corrected to ensure the sensors would fit when the print was scaled.



See Appendix pg 125.

This is the old print that was too small for the water level sensor and the instruments inside. Additionally, it did not include a hole for the LCD display or the pH sensor.



See Appendix pg 125.

This is the front and back of the 3D printed box. In the front, a rectangular hole allows the LCD to sit and face the user, displaying live data. In the back the water level sensor protrudes from the box enough to get an accurate reading of the water level in the tank the the box is attached to. The top of the box provides ventilation and space for the sensors to access the water.

Testing

Overview

This section focuses on evaluating the performance and reliability of my hydroponic system to ensure it meets the project's objectives. Testing includes assessing sensor accuracy, display functionality, and system durability under real conditions. Each component is tested individually and as part of the integrated system to identify any issues. Testing requires a structured approach, comparing benchmarks, analyzing errors, and checking performance against my success criteria. By testing and refining the system step by step, I can ensure it runs efficiently and holds up over time.

Testing Table

Test No.	Test Description	Test Data (input)	Expected Outcome	Outcome
1	Water temp sensor	70.8°F	Temperature in Celsius and Fahrenheit as a float	Success
2	pH Sensor	pH: 7	pH and voltage values as a float	Fail (4.19)
3	pH Sensor	pH: 7	pH and voltage values as a float	Fail (244.31)
4	pH Sensor	pH: 7	pH and voltage values as a float	Success
5	Water Level Sensor	"Water level low"	Boolean outcome displayed as a string	Success
6	Combining the Arduino file with the Raspberry Pi file	2025-01-08 09:23:49= 22.5C 72.85F Voltage: 0.86V pH: 7.85 Water level low. Add more water.	Prints the time, then temperature data, then Arduino data in the output	Fail (Arduino data printed out of order)
7	Combining the Arduino file with the Raspberry Pi file	2025-01-08 09:23:49= 22.5C 72.85F Voltage: 0.86V pH: 7.85 Water level low. Add more water.	Prints the time, then temperature data, then Arduino data in the output	Success

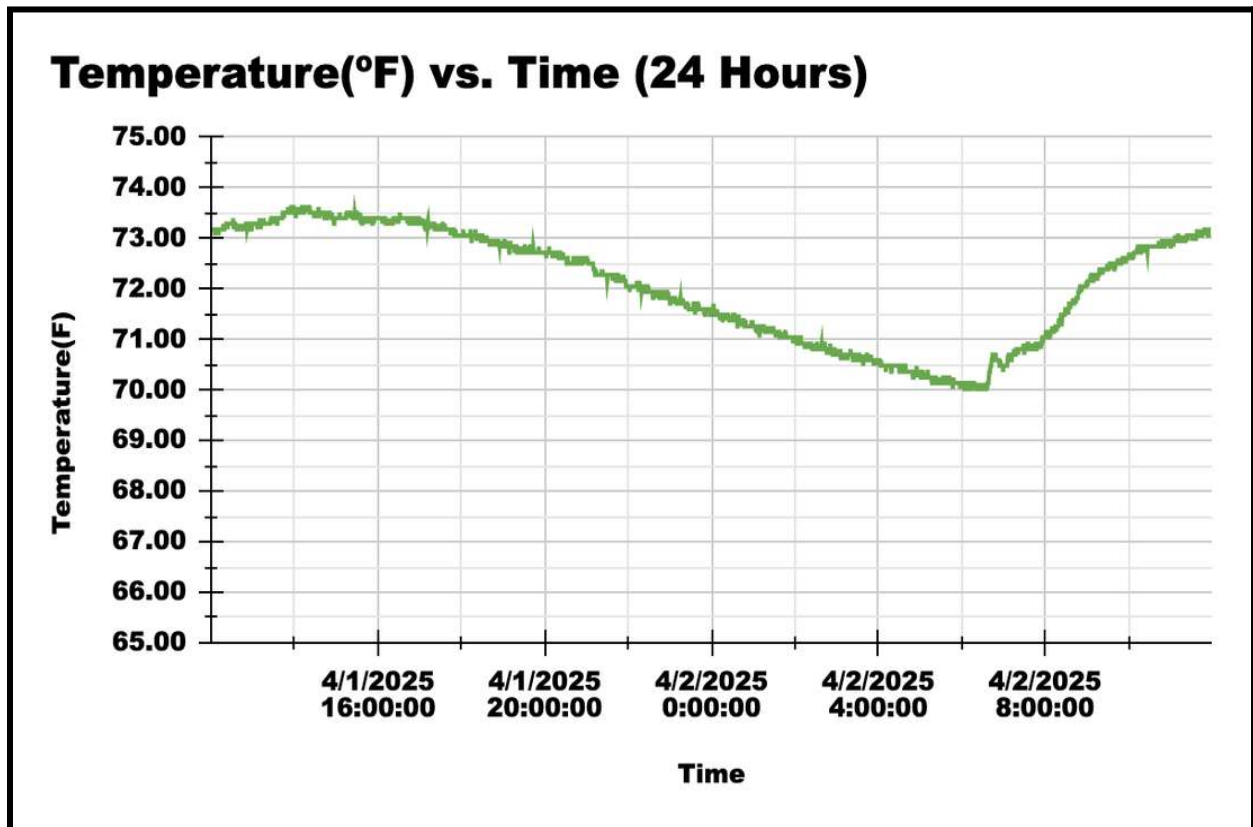
8	Write sensor data to a .csv file	2025-01-08 09:23:49= 22.5C 72.85F Voltage: 0.86V pH: 7.85 Water level low. Add more water.	Prints the time, then temperature data, then Arduino data in a new .csv file	Success
9	Display sensor data on the LCD	22.5C 72.85F Voltage: 0.86V pH: 7.85 Water level low. Add more water.	Prints the temp data on line 1, and scrolls Arduino data on line 2	Fail (the whole screen glitched and showed symbols and both lines scrolled)
10	Display sensor data on the LCD	22.5C 72.85F Voltage: 0.86V pH: 7.85 Water level low. Add more water.	Prints the temp data on line 1, and scrolls Arduino data on line 2	Success
11	Allows the raspberry to run on a schedule using crontab	0 6 * * * /home/pi/start_displaySensorData.sh 0 21 * * * /home/pi/stop_displaySensorData.sh	Raspberry Pi runs from 6AM to 9PM every day	Success
12	Headless technology	@reboot /bin/sleep 30; /usr/bin/python /home/pi/displaySensorData.py	Makes the Raspberry Pi run without a wired connection to the monitor	Success
13	User interface for the app	Clicking buttons	The user can toggle screens and use the control buttons	Success
14	Local storage for the notes app	Testing 123	App temporarily stores user-inputted notes	Success
15	Local storage for resources	Testing title Testing link	App temporarily stores user-inputted links and titles	Success
16	Cloud storage for the app	Enters sample notes, closes app, and comes back	The app permanently stores data and user input until it is deleted	Fail
17	Cloud storage for the app	Enters sample notes, closes app, and comes back	The app permanently stores data and user input until it is deleted	Success

18	3D Structure print	(30x20x20) cm	A 3D box has enough room to house the sensors and the lid fits the body tightly	Fail (Print failed, and dimensions were wrong)
19	3D Structure Print	(30x20x20) cm	A 3D box has enough room to house the sensors, and the lid fits the body tightly	Success
20	Constructing the frame	Water pumped through	The frame pieces fit securely and no water leaks	Success
21	Installing the lights	Plugged into an outlet	Lights turn on when plugged in	One strip light broke (had to solder wires back)
22	Installing the lights	Plugged into an outlet	Lights turn on when plugged in	Success
23	Water pump	Pump turned on for 5 minutes, rest for 30 minutes	When activated, water is pumped out at a constant rate with no leaking	Success
24	Grow lights	Lights turned on for 3 hours	When connected, the lights shine uniformly on the plants below for 3 hours	Success

Process

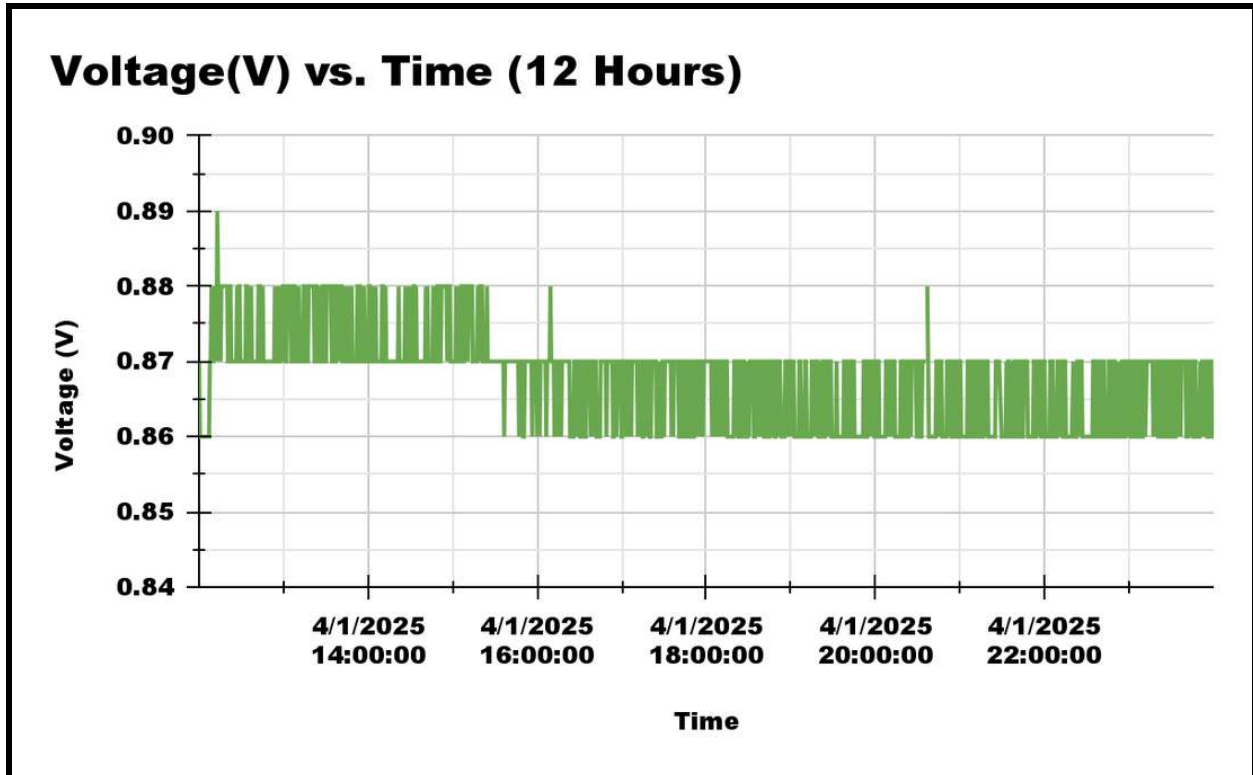
The testing process involved leaving the sensors submerged in water for a duration of 70 hours. Since the plants had not yet fully germinated, the water tank contained filtered, nutrient-free water. In the next testing phase, nutrient-enriched water will be used, and the plants will be placed into the system. Data from the sensors was automatically recorded in a CSV file, which I imported into Google Sheets for analysis. There, I filtered the dataset to extract only the numerical values from over 7,000 rows, removing any extraneous text. After organizing the data, I created three graphs: temperature (in °F) versus time, voltage versus time, and pH value versus time. These graphs are included below. In future tests, these graphs will reflect the dynamics of a fully functioning hydroponic system—for example, temperature spikes in the temperature vs. time graph will indicate heater activation, while shifts in the pH

vs. time graph will correspond to the user adding pH-balancing solutions. Gradual pH changes may also indicate the rate at which plants absorb nutrients and thereby affect the water's pH. One issue encountered during testing was a coding error: instead of setting the CSV file write mode to "a" (append), it was mistakenly set to "w" (overwrite), which resulted in the loss of previously collected sensor data from the nutrient-enriched water phase.



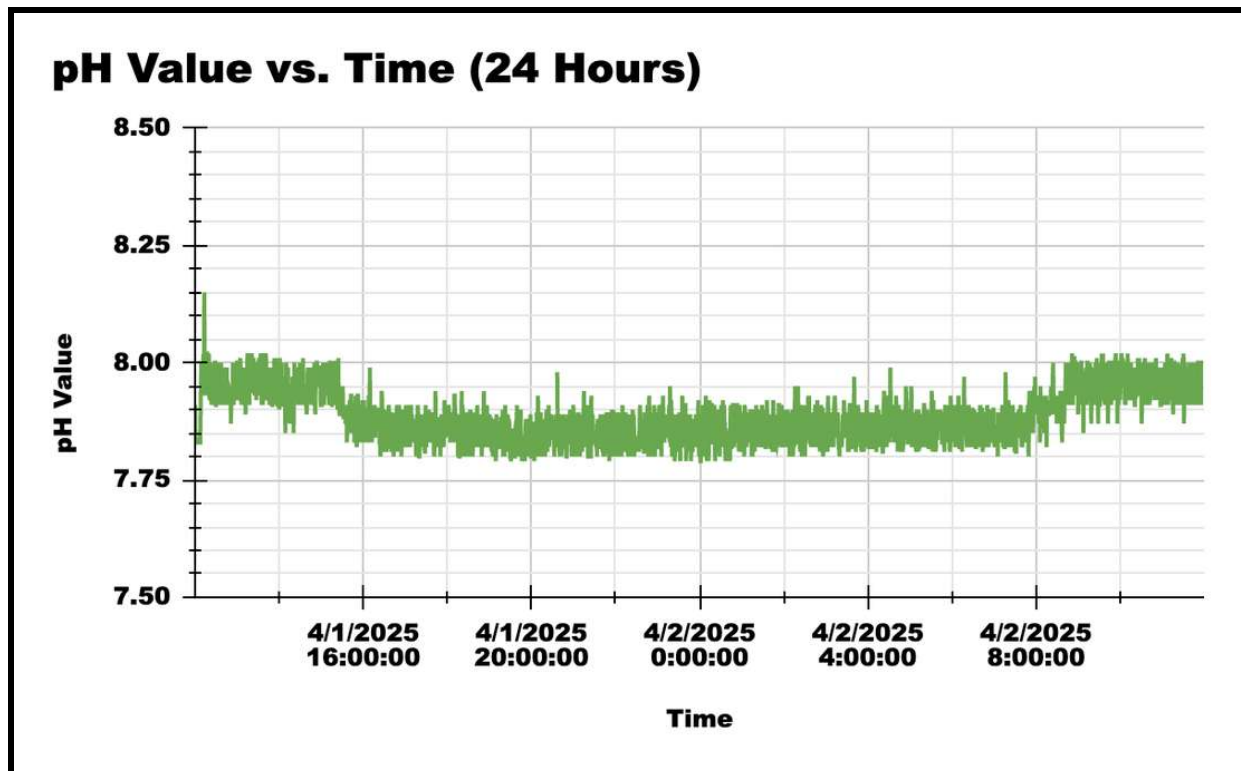
See Appendix pg 126.

This graph displays the temperature readings in fahrenheit over the course of 24 hours, starting at noon on April 1st, and ending at noon on April 2nd.



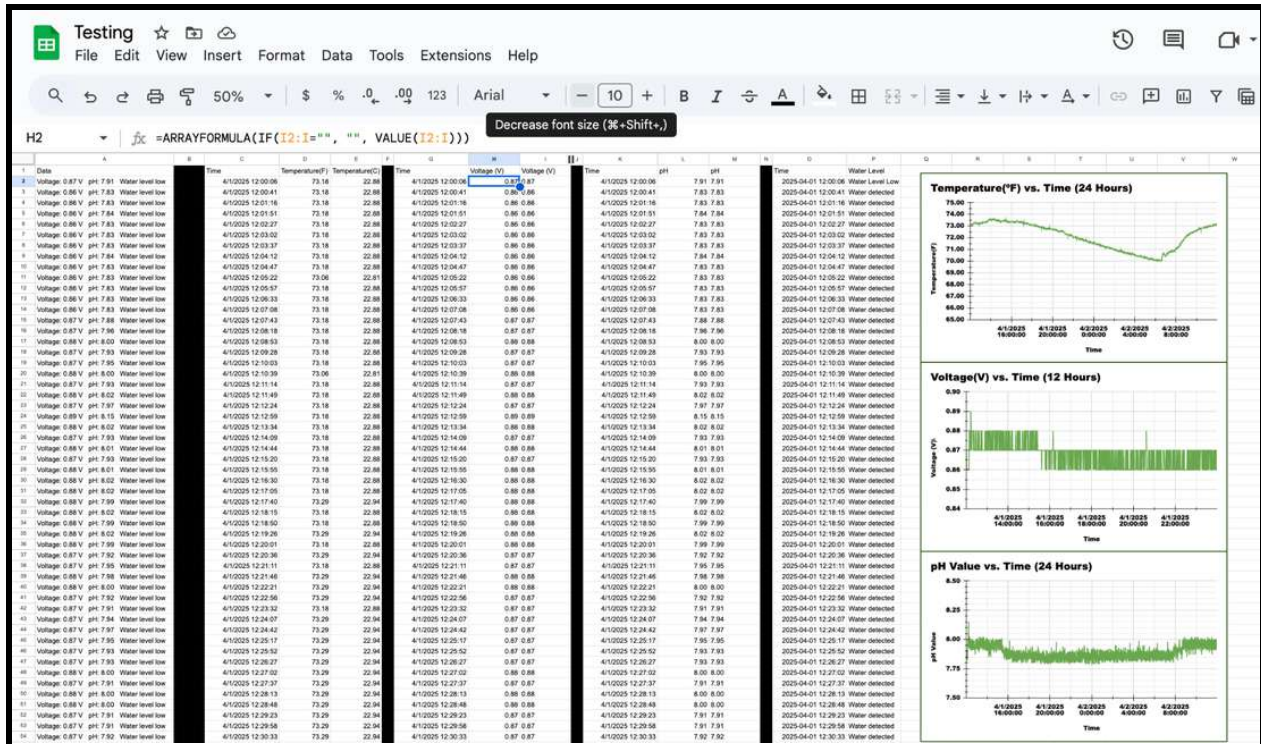
See Appendix pg 126.

This graph displays the voltage readings in over the course of 12 hours, starting at noon on April 1st, and ending at midnight. For the most part, the readings fluctuate between 0.86 and 0.88, which is accurate for this sensor.



See Appendix pg 127.

This graph displays the pH readings in over the course of 24 hours, starting at noon on April 1st, and ending at noon on April 2nd. The average of these readings is around 7.9, which is helpful information because if there were plants in the system at this time, I would need to lower the pH or else the plants would not be growing in their optimal conditions.



See Appendix pg 127.

This image shows the spreadsheet of data obtained from the sensors during testing. Once I imported the .CSV file, I organized the data so I could create graphs to visualize the data. The leftmost column and the first block of columns, including the time and temperature in °C and °F, is the data imported from the .CSV file. The time and temperature were imported into separate columns which made it easier to create graphs from. However, the voltage, pH, and water level readings were grouped into one cell, so I had to index each sensor reading to a respective column, and apply that formula to the whole column. I then had to create a column next to the voltage and pH columns that formatted the data as numbers, instead of a visual representation of the indexing formula. From there, I copied the time so I could select two adjacent columns to produce a graph. As the water level data is formatted as a string, it cannot be turned into a line graph.

Evaluation

To evaluate the project, we must first go back and look at the success criteria. The success criteria for this project falls into four categories: Project Focus, Technical Components, Technological Integration, and User Interaction and Data Accessibility. The hydroponic system successfully meets its goals by integrating technology with sustainable agriculture techniques. The project effectively reduces water usage compared to traditional soil-based farming by delivering nutrients directly to plant roots. This efficiency supports stable agriculture despite global warming, ensuring consistent crop growth in controlled environments.

A key success of the system is its sensor integration and accuracy. The DS18B20 temperature sensor, pH sensor, and water level sensor are properly installed and provide reliable real-time data. These sensors allow for precise monitoring of water conditions, ensuring optimal plant health. Additionally, the mobile application is fully functional, allowing users to track sensor data seamlessly. The app's real-time feedback and user-friendly interface make it accessible for both home growers and larger-scale operations.

In terms of scalability, the system shows promise for expansion and automation. While it currently functions as a monitoring tool, future iterations could integrate automated pumps and actuators to regulate nutrient delivery dynamically. This would further enhance its efficiency and reduce the need for manual intervention.

Overall, the hydroponic system achieves its objectives by providing a sustainable, technology-driven approach to farming. Its success demonstrates the potential of hydroponics and IoT-based smart farming in addressing food security and agricultural challenges posed by climate change. Future developments could refine automation capabilities, expand its applications for commercial and home use, and be used in experimentation for discovering optimal conditions for a variety of plants.

Conclusion

This project demonstrates how integrating sensors and automation with a hydroponic system enhances efficiency by allowing the user to closely monitor the environment and make adjustments, boosting the plants' yield. By using sensors to track water temperature, pH levels, and water levels, users can maintain optimal conditions with greater precision and less manual effort. The usage of hydroponics as a way to control environmental factors such as nutrients, water, and light distribution is already a sustainable practice. Now, integrate the system with technology, and you have a sustainable growing method with minimal effort. The applications of hydroponics are diverse, and the benefits are profound.

One limitation of this setup is the reliance on manual intervention for changes in water chemistry and temperature. Future improvements, also stated in the next section, could involve automated responses to suboptimal environmental factors and AI-driven analysis to optimize plant growth. Additionally, expanding the mobile application to include data logging, like how much each plant weighs or how much water they use versus receive, and trend analysis would provide growers with deeper insights into their system's performance.

Overall, this project highlights the potential of hydroponics as a sustainable solution for food production in a changing climate. With further development, these systems could contribute to increased efficiency and enhanced food security, whether in urban and resource-limited environments or rural areas impacted by climate change.

Future Development

This project has significant potential for further development, whether through experiments on optimal growth conditions and plant growth patterns or by scaling hydroponics for sustainable large-scale farming. Given more time and funding, I would enhance the project in three key areas: sensors and hardware, the app, and the physical structure.

First, I would conduct controlled experiments on optimal growth conditions by adjusting one variable at a time. To achieve this, I would integrate cameras with photo and live-streaming capabilities from multiple angles, a water heater, a water chiller, and a light sensor.

Additionally, I would develop programmable lights that communicate with the app to adjust intensity and duration automatically. Environmental monitoring would be expanded with air quality, temperature, and humidity sensors, allowing for real-time adjustments. An EC sensor would help track nutrient concentration in the water, ensuring plants receive the ideal balance. These enhancements would allow for precise control and monitoring of all contributing factors to plant growth while keeping them constant during variable testing.

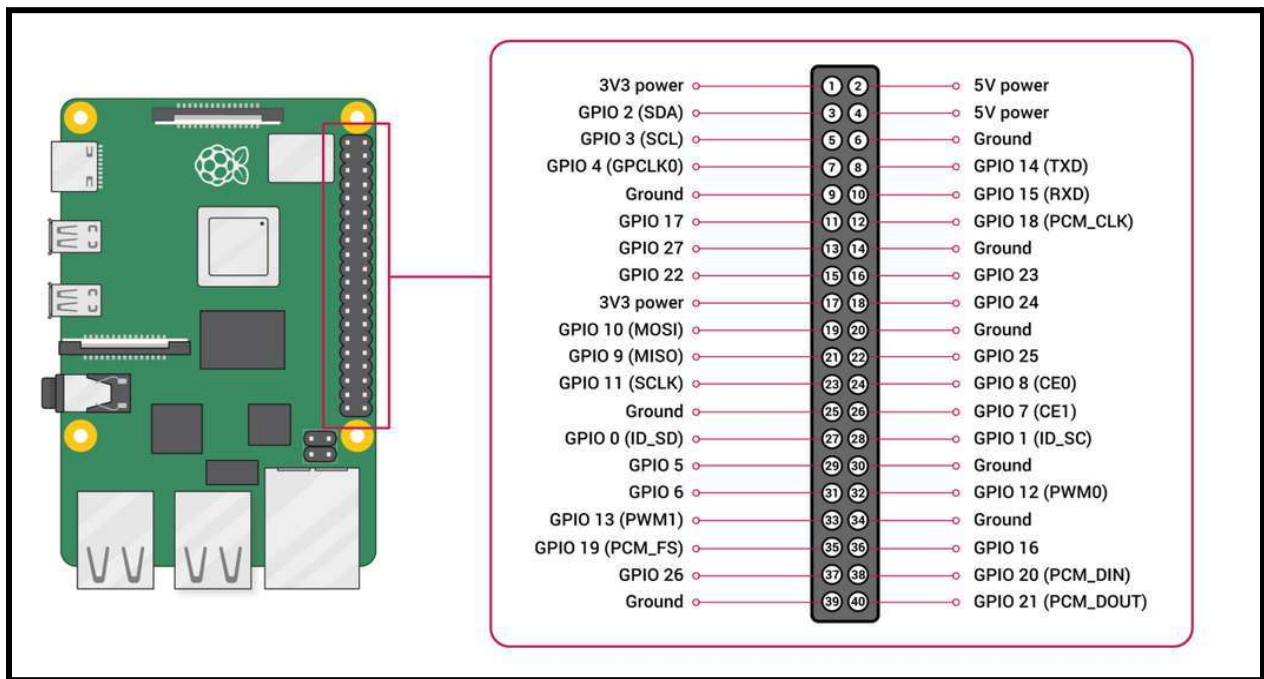
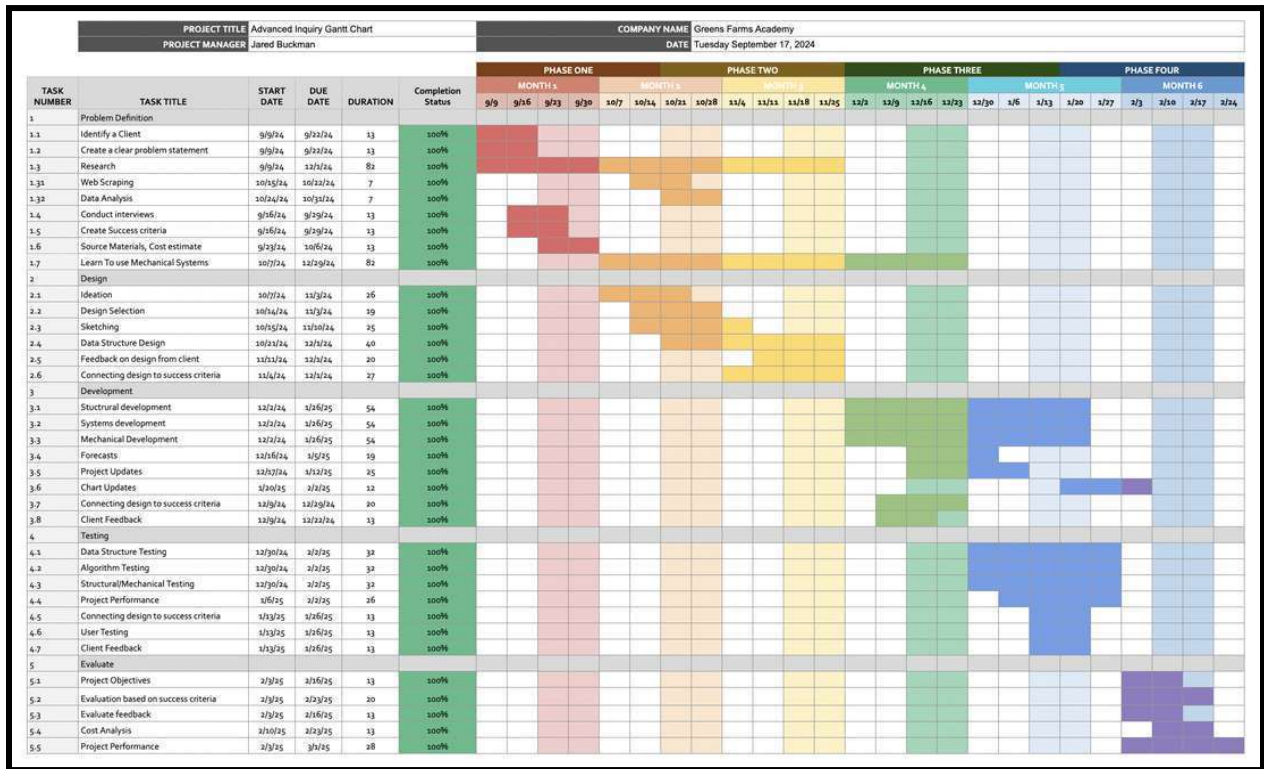
Next, I would incorporate AI-powered automation to elevate the system beyond traditional hydroponic setups. The AI would analyze sensor data and camera feeds to make real-time adjustments, such as activating the heater or chiller and automatically dosing nutrients and pH regulators based on user-defined conditions. Additionally, the system would adjust plant levels to prevent them from growing too close to the lights, which can cause damage. AI would also assist users by identifying plant issues like discoloration or burns and providing tailored recommendations using sensor data. Furthermore, it would suggest optimal plant pairings based on environmental compatibility and notify users when pruning is necessary or when immediate action is required.

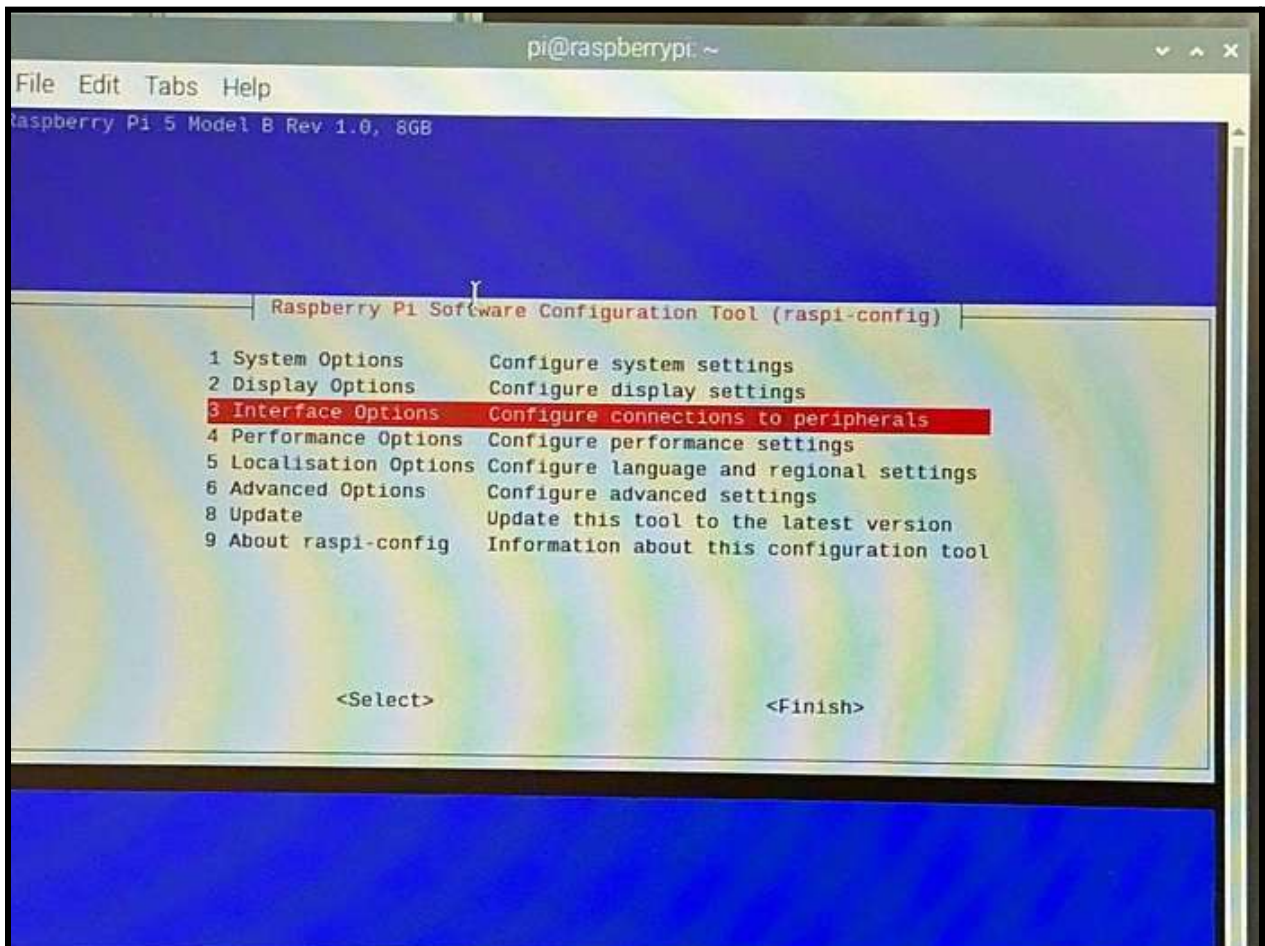
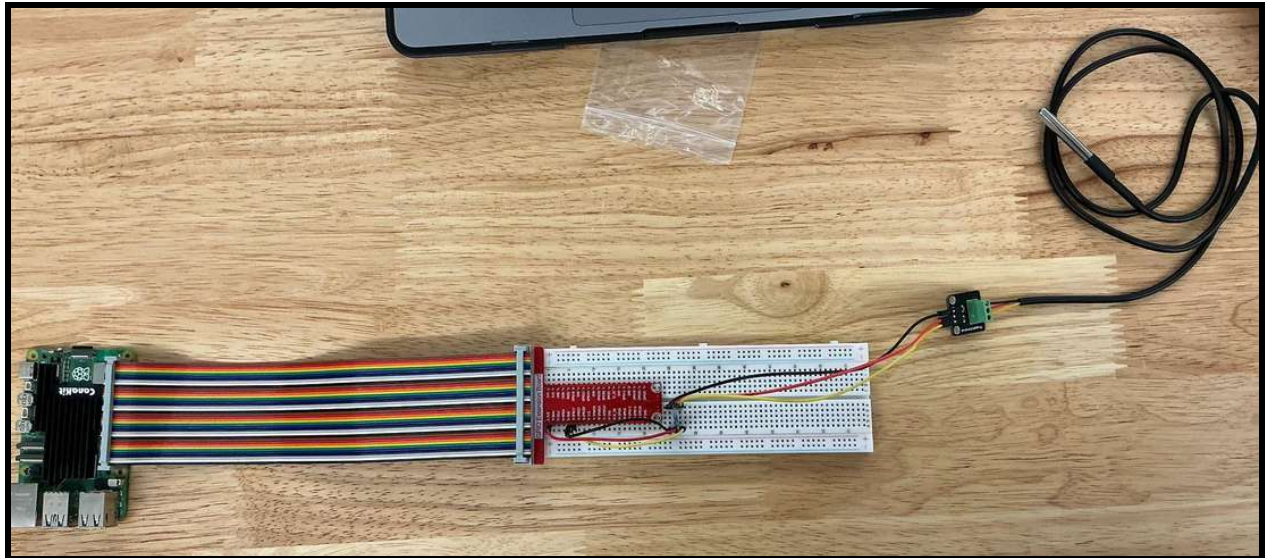
For the physical structure, I would use more durable materials to support taller plants and implement adjustable shelving to accommodate varying plant heights. In research settings, the spacing between plant levels would remain constant, only changing once I analyze the effects of light distance and intensity on plant growth. Some layers would also include detachable grow cages for plants requiring additional support, such as tomatoes.

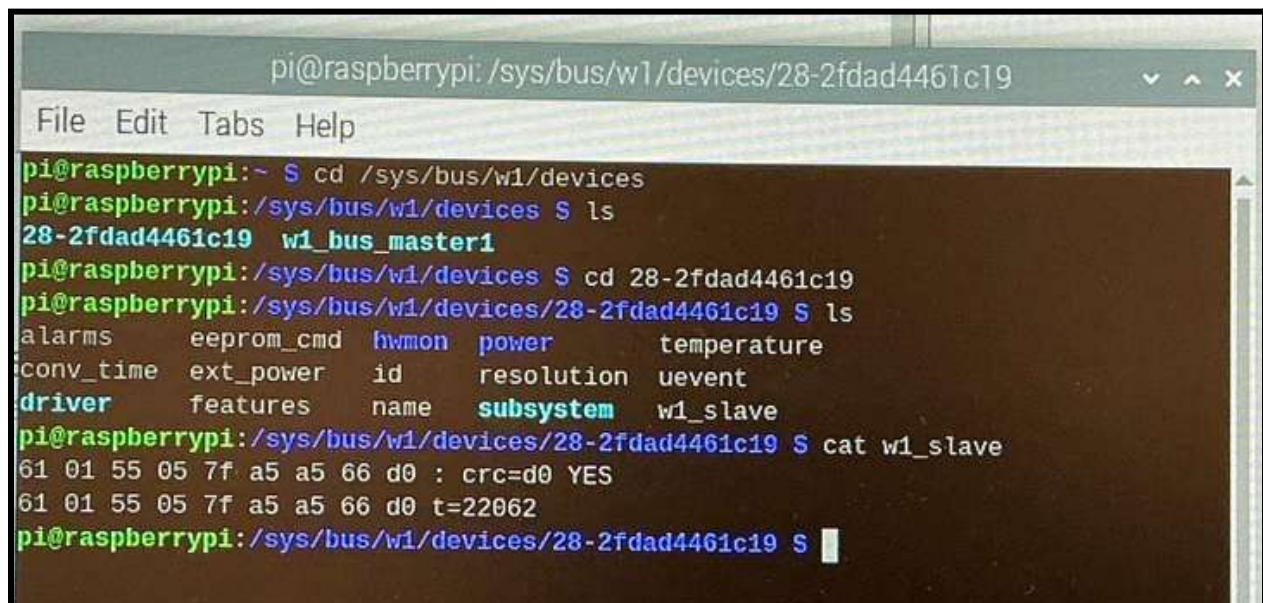
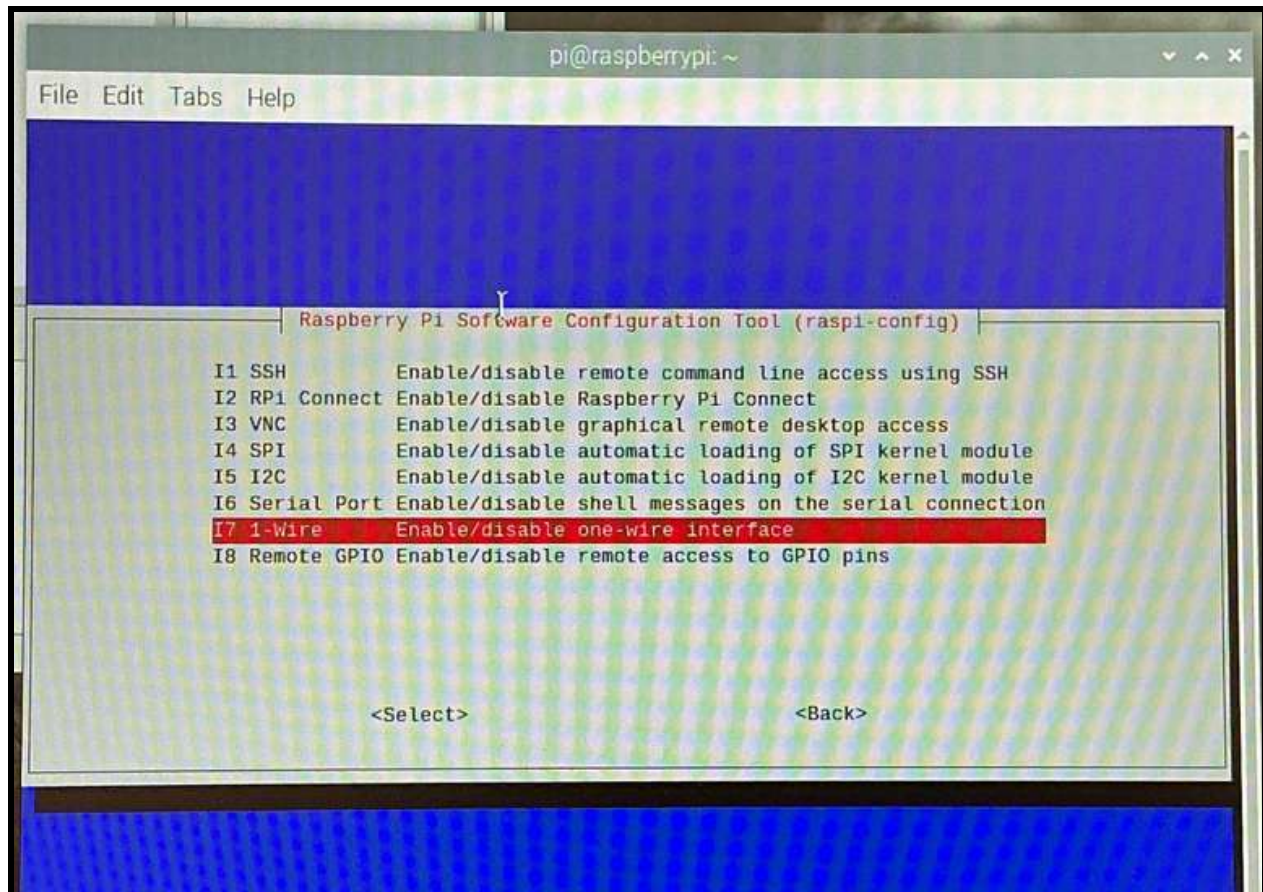
Finally, I would explore cost-effective ways to integrate these enhancements without compromising system quality. From a business perspective, reducing costs would increase profit margins, while for research purposes, sourcing affordable materials would allow for continued experimentation within budget constraints.

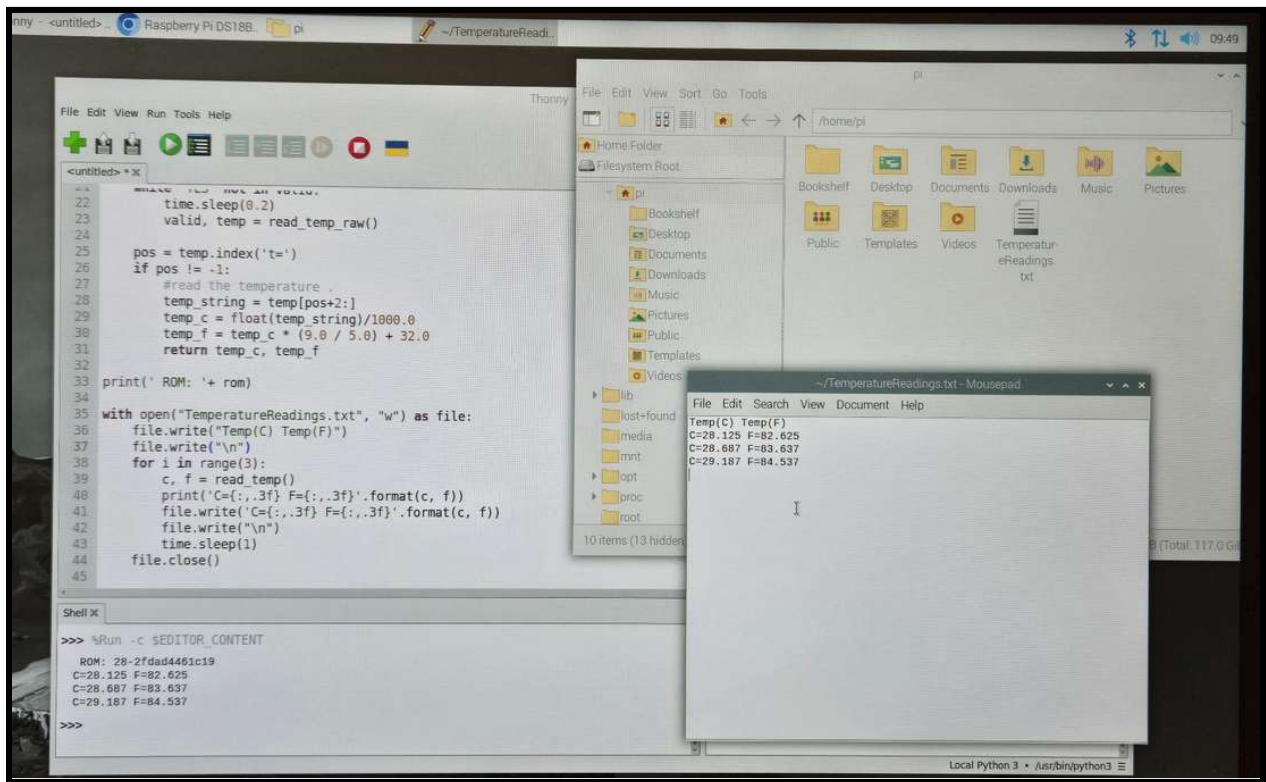
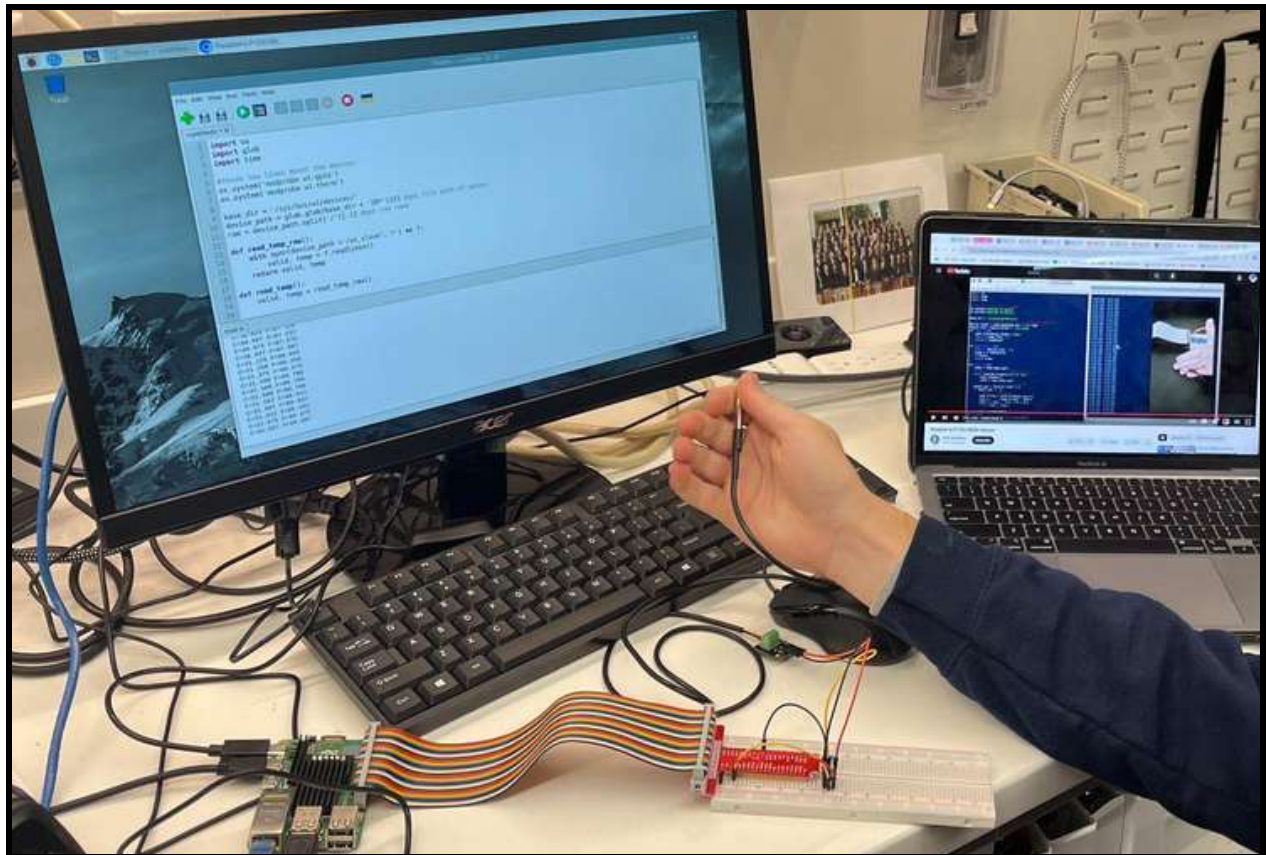
Appendix

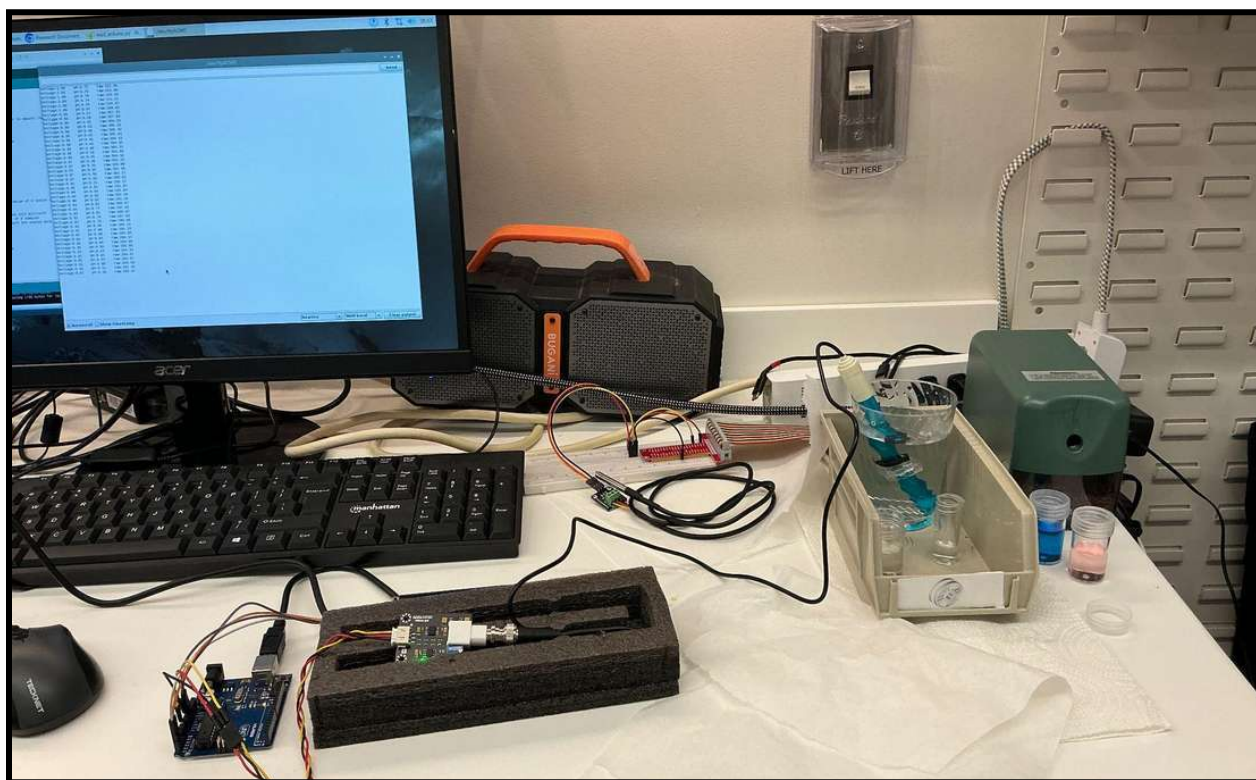
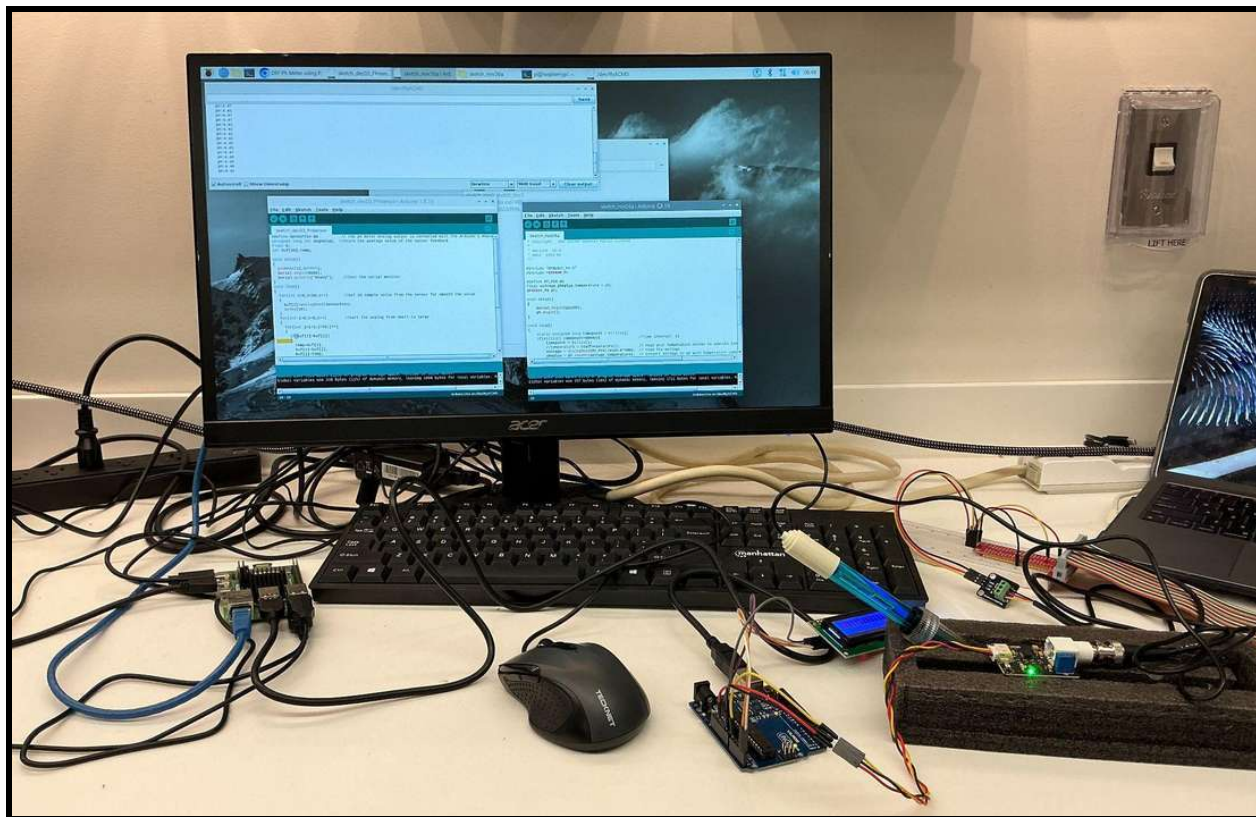
Images and Graphs











pi@raspberrypi: ~
sketch_dec03_PHsen. Research Document... temp_readings_1115. /dev/ttyACM0

sketch_dec03_PHsensor | Arduino 1.8.19

```
File Edit Sketch Tools Help
sketch_dec03_PHsensor
void setup()
{
  pinMode(13, OUTPUT);
  Serial.begin(9600);
  Serial.println("Ready"); //Test the serial monitor
}

void loop()
{
  for(int i=0; i<10; i++) //Get 10 sample values from the sensor
  {
    buf[i]=analogRead(SensorPin);
    delay(20);
  }
  for(int i=0; i<9; i++) //sort the analog from small to large
  {
    for(int j=i+1; j<10; j++)
    {
      if(buf[i]>buf[j])
      {
        temp=buf[i];
        buf[i]=buf[j];
        buf[j]=temp;
      }
    }
  }

  avgValue=0;
  for(int i=2; i<8; i++) //take the average value
  {
    avgValue+=buf[i];
  }

  float voltage=(float)avgValue*5.0/1024/6; //convert the analog in
  float pHValue=(float)voltage/6; //take average of 6
  float calibratedpH = m*pHValue + b; //convert t

  Serial.print("voltage:"); //print voltage heading
  Serial.print(voltage, 2); //print voltage data
  Serial.print(" pH:"); //print pH heading same line
  Serial.print(calibratedpH, 2); //print pH data
  Serial.print(" pH:"); //print pH heading same line
  Serial.print(pHValue, 2); //print pH data
  Serial.println(" ");
  digitalWrite(13, HIGH);
  delay(800);
}
```

sketch_dec03_PHsensor | Arduino 1.8.19

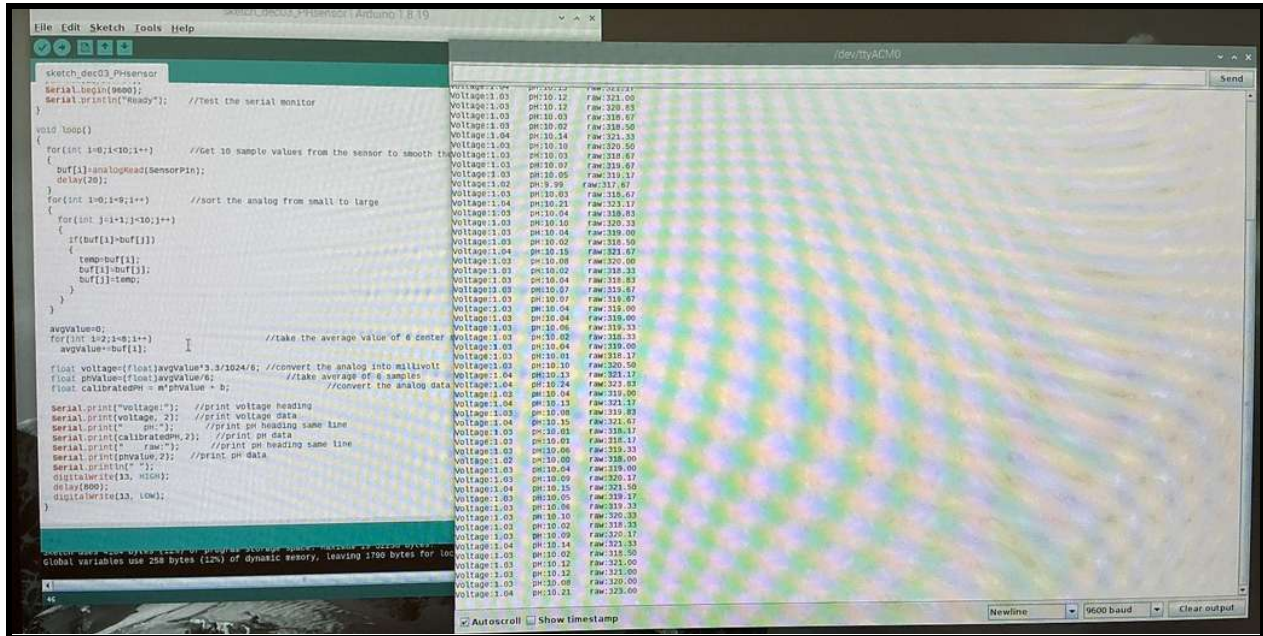
Voltage	pH	pH
0.83	3.99	170.17
0.83	3.99	170.33
0.83	3.99	170.17
0.83	3.99	170.33
0.83	3.95	169.17
0.83	3.94	169.00
0.83	3.98	170.00
0.83	3.95	169.33
0.83	3.99	170.33
0.83	4.00	170.50
0.84	4.03	171.17
0.83	3.99	170.17
0.83	3.95	170.00
0.83	4.02	171.00
0.83	3.96	169.50
0.83	3.99	170.00
0.83	3.95	170.33
0.83	3.97	169.83
0.83	4.00	170.50
0.83	3.98	170.00
0.83	3.98	170.00
0.83	3.98	170.00
0.83	4.01	170.67
0.83	3.95	169.17
0.83	4.00	170.50
0.84	4.03	171.17
0.83	3.99	170.33
0.83	3.99	170.33
0.83	4.01	170.83
0.83	4.00	170.50
0.83	4.00	170.50
0.83	4.01	170.67
0.83	3.98	170.00
0.83	4.01	170.83
0.83	3.98	170.00
0.83	3.99	170.17
0.83	4.02	171.00
0.84	4.03	171.17
0.83	3.97	169.67
0.83	3.99	170.17
0.83	3.95	169.17
0.83	3.96	169.50
0.83	3.98	170.00
0.83	3.99	170.17
0.84	4.03	171.17
0.83	3.99	170.33
0.83	4.00	170.50
0.83	4.00	170.50

Autoscroll Show timestamp Newline 9600

Help /dev/ttyACM0

```
//Test the serial monitor
06:32:05.192 -> Voltage:1.20 pH:7.10 pH:249.17
06:32:05.178 -> Voltage:1.21 pH:7.17 pH:248.50
06:32:06.171 -> Voltage:1.22 pH:7.19 pH:248.83
06:32:07.163 -> Voltage:1.22 pH:7.19 pH:248.83
06:32:08.190 -> Voltage:1.20 pH:7.09 pH:246.33
06:32:09.183 -> Voltage:1.21 pH:7.15 pH:248.00
06:32:10.176 -> Voltage:1.20 pH:7.08 pH:246.17
06:32:11.202 -> Voltage:1.21 pH:7.12 pH:247.17
06:32:12.196 -> Voltage:1.20 pH:7.05 pH:245.33
06:32:13.189 -> Voltage:1.20 pH:7.09 pH:246.33
06:32:14.214 -> Voltage:1.20 pH:7.04 pH:245.17
06:32:15.206 -> Voltage:1.21 pH:7.15 pH:248.00
06:32:16.198 -> Voltage:1.20 pH:7.06 pH:245.67
06:32:17.224 -> Voltage:1.21 pH:7.11 pH:247.00
06:32:18.218 -> Voltage:1.21 pH:7.17 pH:248.33
06:32:19.210 -> Voltage:1.22 pH:7.20 pH:249.17
06:32:20.206 -> Voltage:1.21 pH:7.13 pH:247.50
06:32:21.232 -> Voltage:1.21 pH:7.11 pH:246.83
06:32:22.224 -> Voltage:1.20 pH:7.09 pH:246.50
06:32:23.217 -> Voltage:1.21 pH:7.13 pH:247.50
06:32:24.243 -> Voltage:1.20 pH:7.03 pH:245.00
06:32:25.235 -> Voltage:1.21 pH:7.15 pH:247.83
06:32:26.228 -> Voltage:1.21 pH:7.13 pH:247.50
06:32:27.254 -> Voltage:1.21 pH:7.11 pH:247.00
06:32:28.247 -> Voltage:1.21 pH:7.11 pH:247.00
06:32:29.273 -> Voltage:1.21 pH:7.11 pH:246.83
06:32:30.265 -> Voltage:1.20 pH:7.10 pH:246.67
06:32:31.258 -> Voltage:1.21 pH:7.12 pH:247.17
06:32:32.284 -> Voltage:1.20 pH:7.09 pH:246.33
06:32:33.277 -> Voltage:1.20 pH:7.07 pH:246.00
06:32:34.270 -> Voltage:1.20 pH:7.10 pH:246.67
06:32:35.263 -> Voltage:1.20 pH:7.05 pH:245.33
06:32:36.289 -> Voltage:1.21 pH:7.11 pH:247.00
06:32:37.281 -> Voltage:1.21 pH:7.15 pH:248.00
06:32:38.274 -> Voltage:1.21 pH:7.15 pH:247.83
06:32:39.300 -> Voltage:1.20 pH:7.07 pH:245.83
06:32:40.293 -> Voltage:1.21 pH:7.14 pH:247.67
06:32:41.286 -> Voltage:1.21 pH:7.13 pH:247.33
06:32:42.313 -> Voltage:1.21 pH:7.15 pH:248.00
06:32:43.305 -> Voltage:1.20 pH:7.10 pH:246.67
06:32:44.303 -> Voltage:1.20 pH:7.05 pH:245.50
06:32:45.328 -> Voltage:1.20 pH:7.05 pH:245.33
06:32:46.321 -> Voltage:1.20 pH:7.07 pH:245.83
06:32:47.313 -> Voltage:1.20 pH:7.05 pH:245.50
06:32:48.339 -> Voltage:1.19 pH:7.01 pH:244.50
06:32:49.332 -> Voltage:1.19 pH:7.00 pH:244.33
06:32:50.324 -> Voltage:1.20 pH:7.08 pH:246.17
06:32:51.350 -> Voltage:1.20 pH:7.03 pH:245.00
06:32:52.342 -> Voltage:1.20 pH:7.08 pH:246.17
06:32:53.335 -> Voltage:1.20 pH:7.06 pH:245.67
06:32:54.328 -> Voltage:1.21 pH:7.11 pH:247.00
06:32:55.354 -> Voltage:1.21 pH:7.18 pH:248.67
06:32:56.347 -> Voltage:1.21 pH:7.11 pH:246.83
06:32:57.339 -> Voltage:1.21 pH:7.12 pH:247.17
```

Autoscroll Show timestamp Newline



Calibrating the pH Sensor

12/3/24

PH Solution	PH Sensor Output 1 (pre-commenting out the existing conversion code)	PH Sensor Output 2	Sensor Calibrated (trial 2)
4.0	2.86	170.54	3.99
7.0	4.19	244.31	
10.0	7.17	323.58	10.229706

Trial 1:

$$y=mx+b, y=2.2556x - 6.1624$$

$$m: \Delta y/\Delta x = (7-4)/(4.19-2.86) = 3/1.33 = 2.2556$$

$$b: 2.86-2.2556(4) = -6.1624$$

Trial 2:

$$y=mx+b, y=2.2556x - 6.1624$$

$$m: \Delta y/\Delta x = (7-4)/(4.19-2.86)$$

$$b = y-mx: 2.86-2.2556(4) = -6.1624$$

(170.54,4) and (244.31,7)

$$y=0.0407x-2.94$$

(170.54,4) and (323.58,10)

12/10/24: with the conversion from 3.3V (voltage=avgValue*3.3/1024/6)

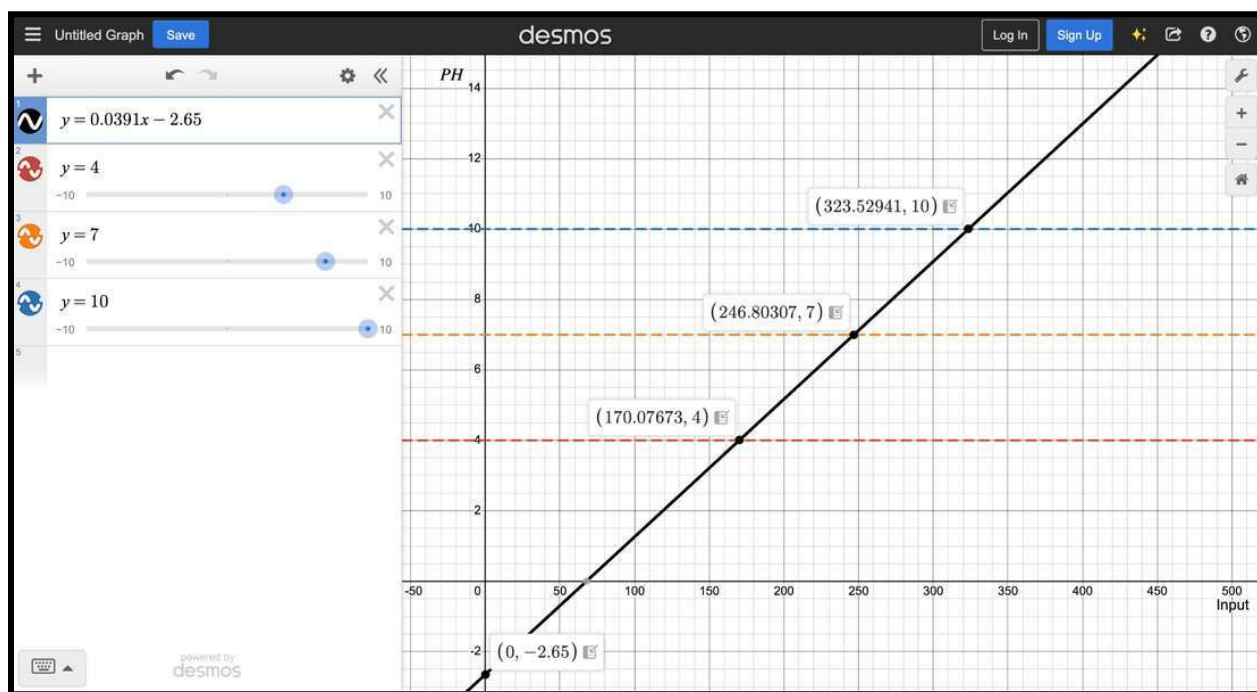
Solution	PH Tester stick	PH Sensor Output (average of 60 values)	Sensor Calibrated (average)
4.0	4.01	170.14	3.995
7.0	7.01	246.90	6.99
10.0	9.98	319.21	9.91

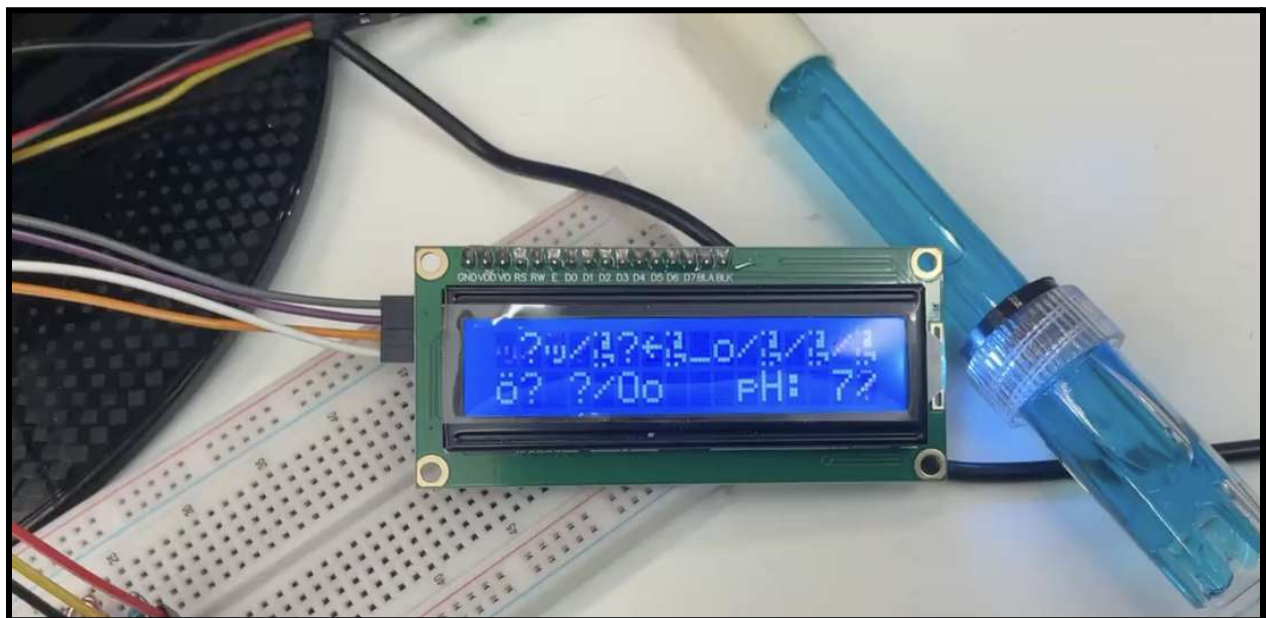
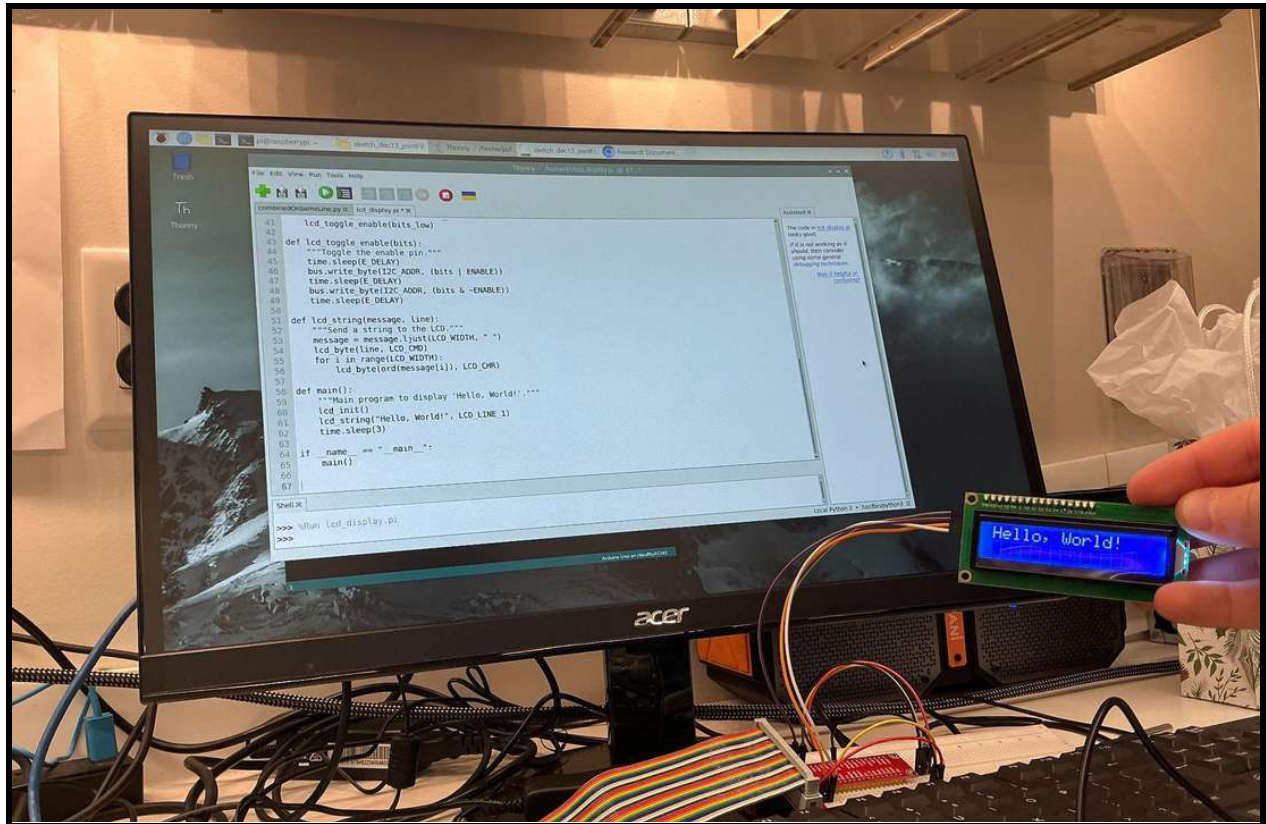
y = pH solution, x = sensor readings

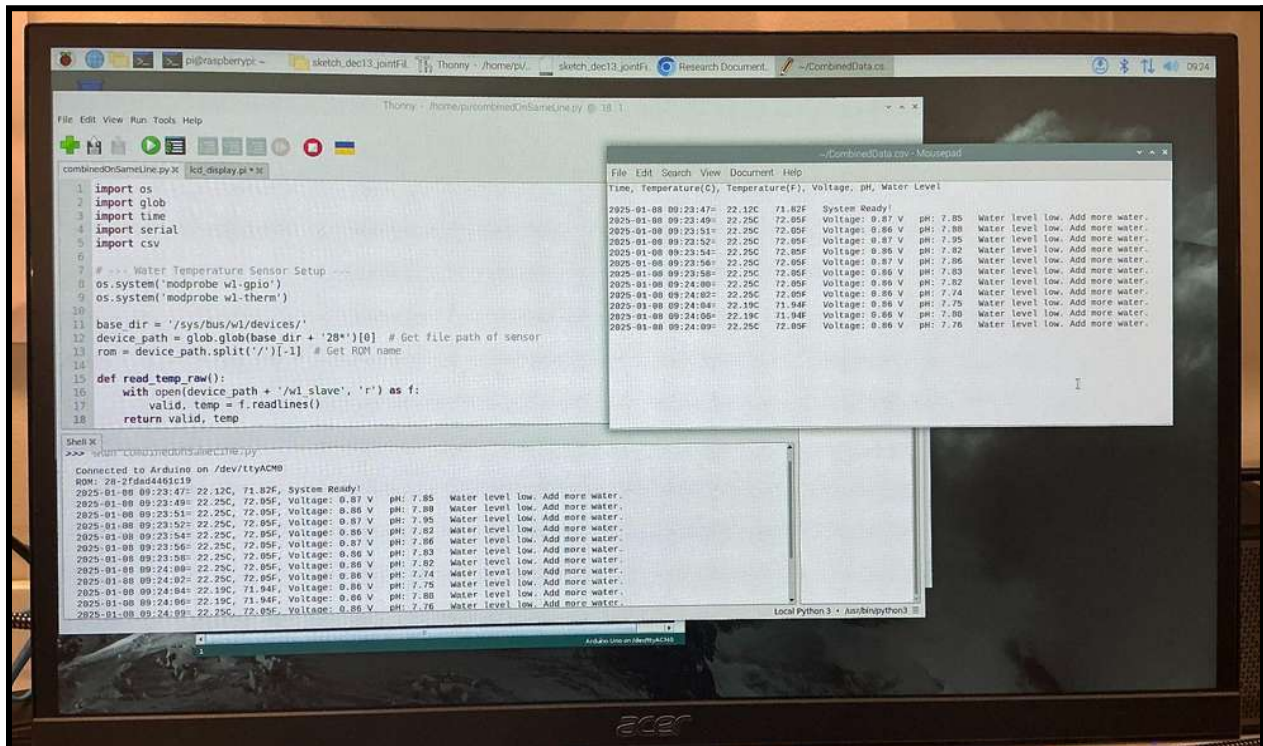
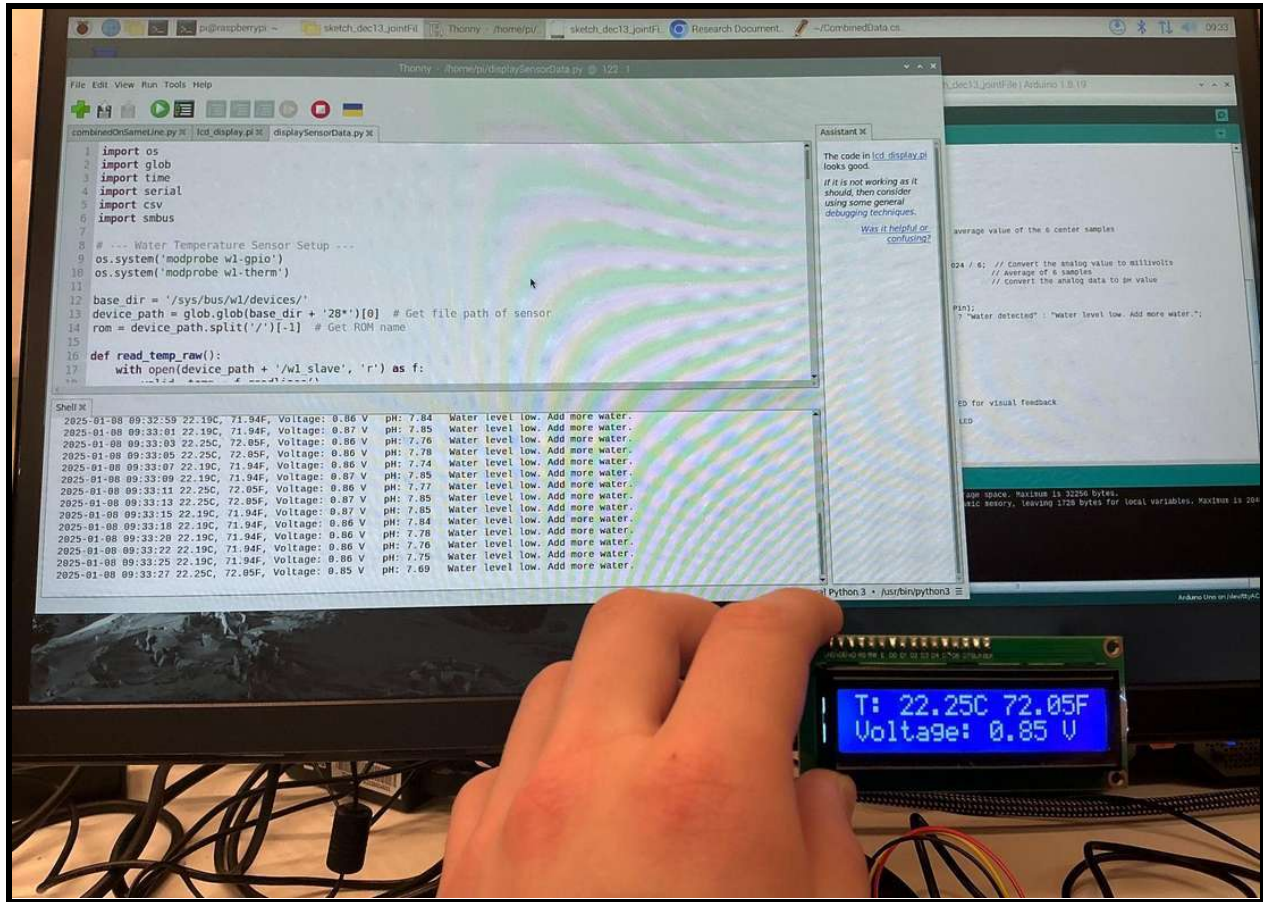
y = m*x + b: **y= 0.0391x - 2.65**

m: $\Delta y / \Delta x = (7-4) / (246.90-170.14) = 0.0391$

b = y-mx: $4-0.0391(170.14) = -2.65$

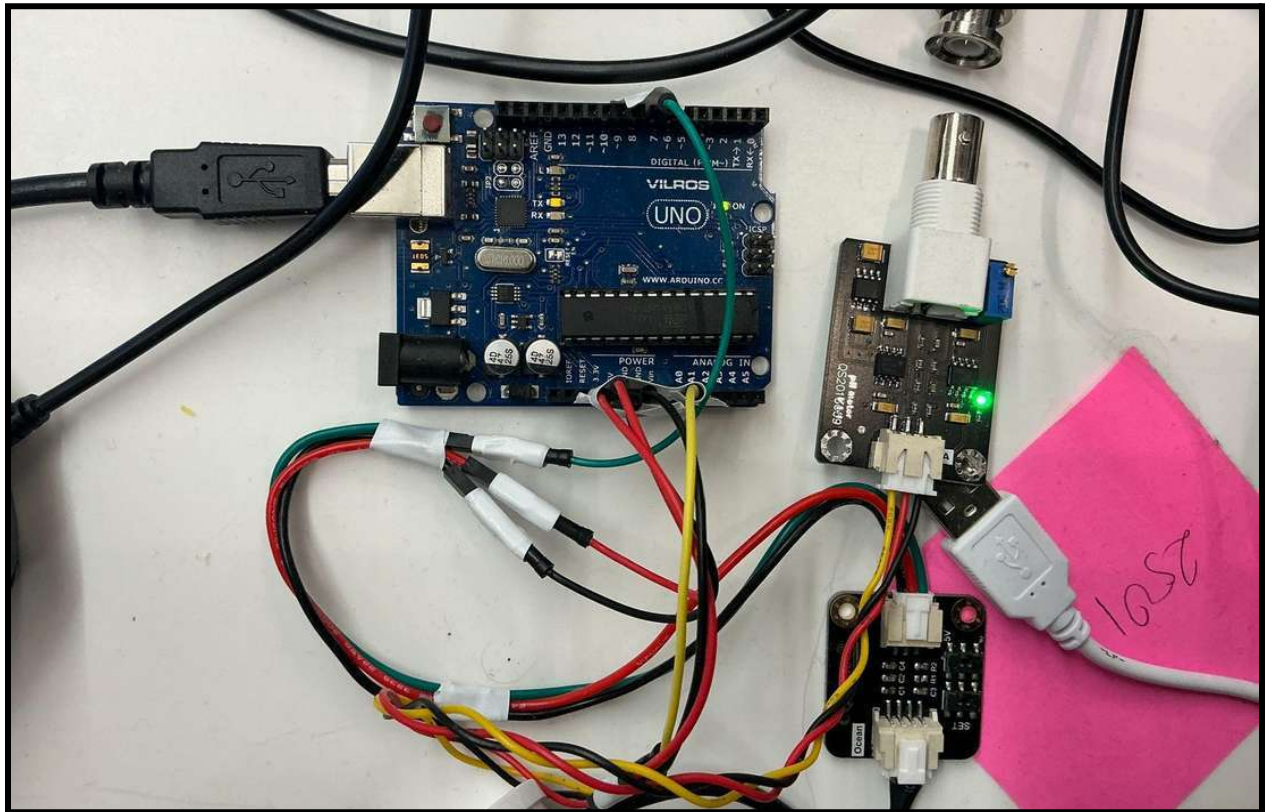
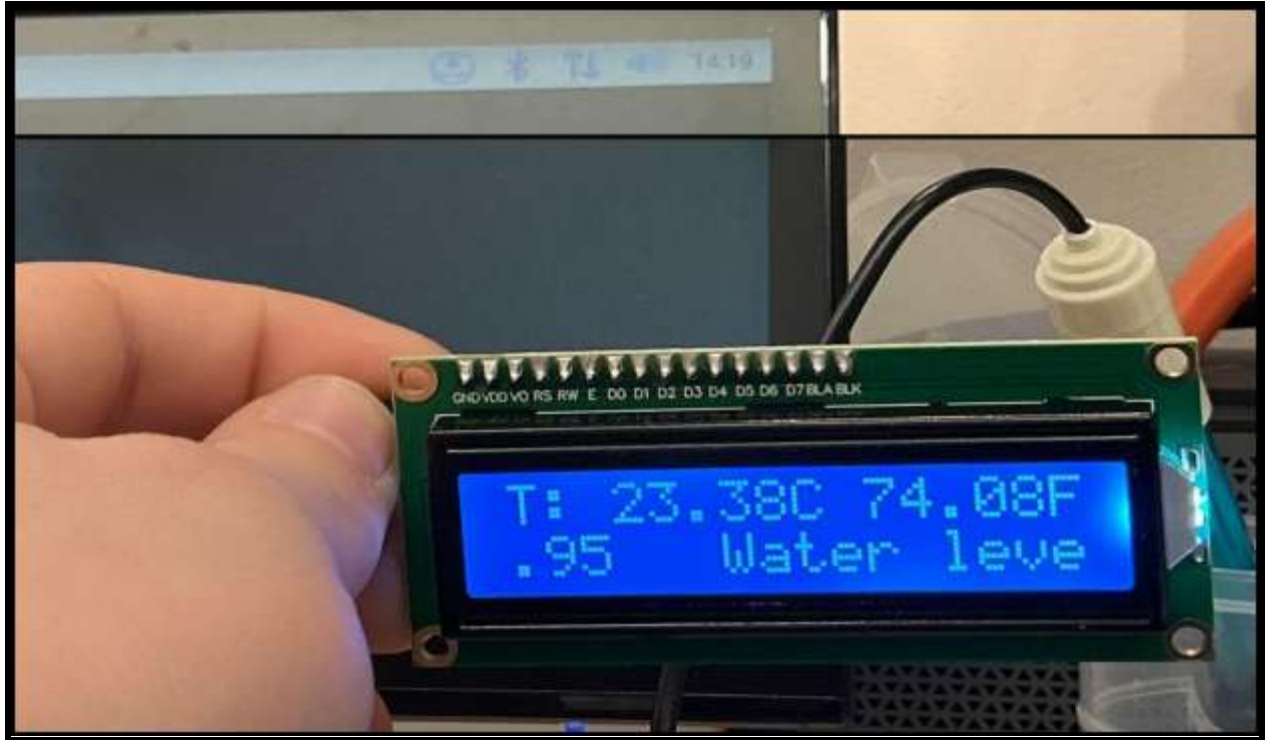


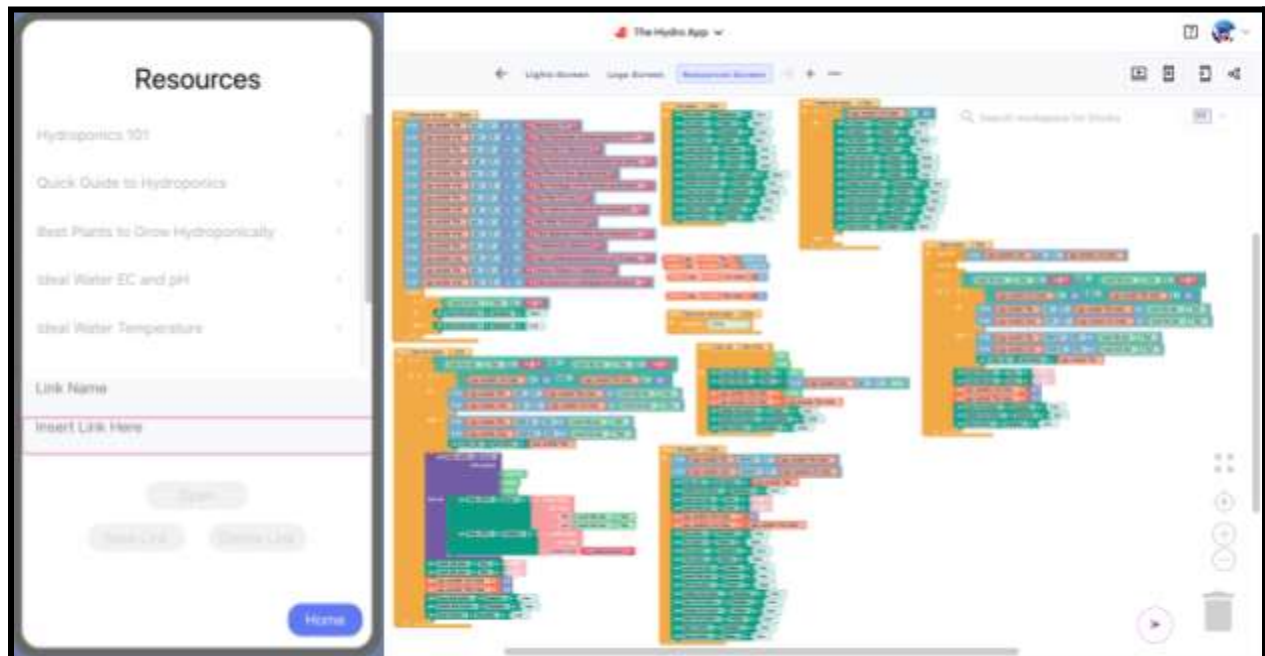
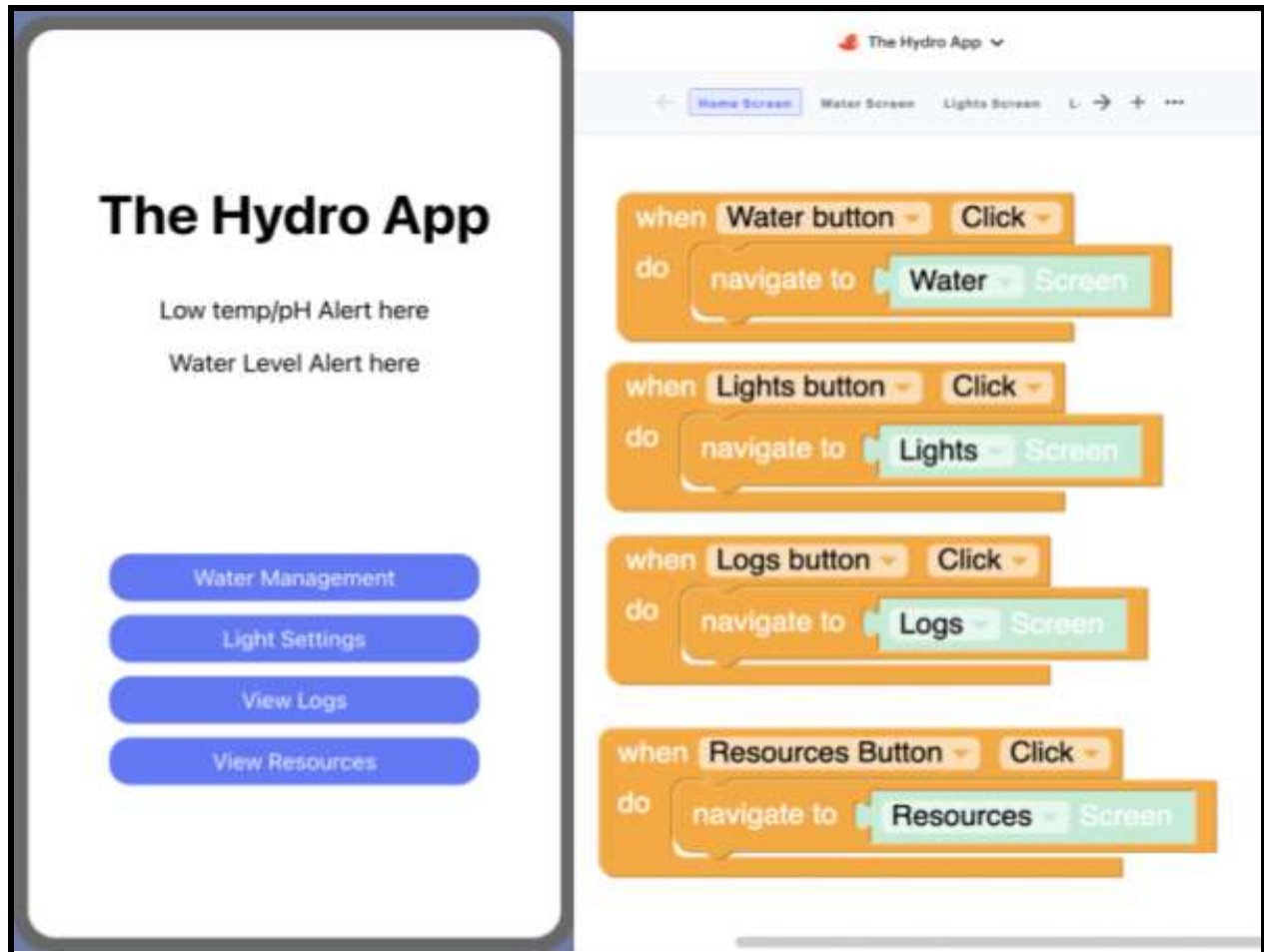


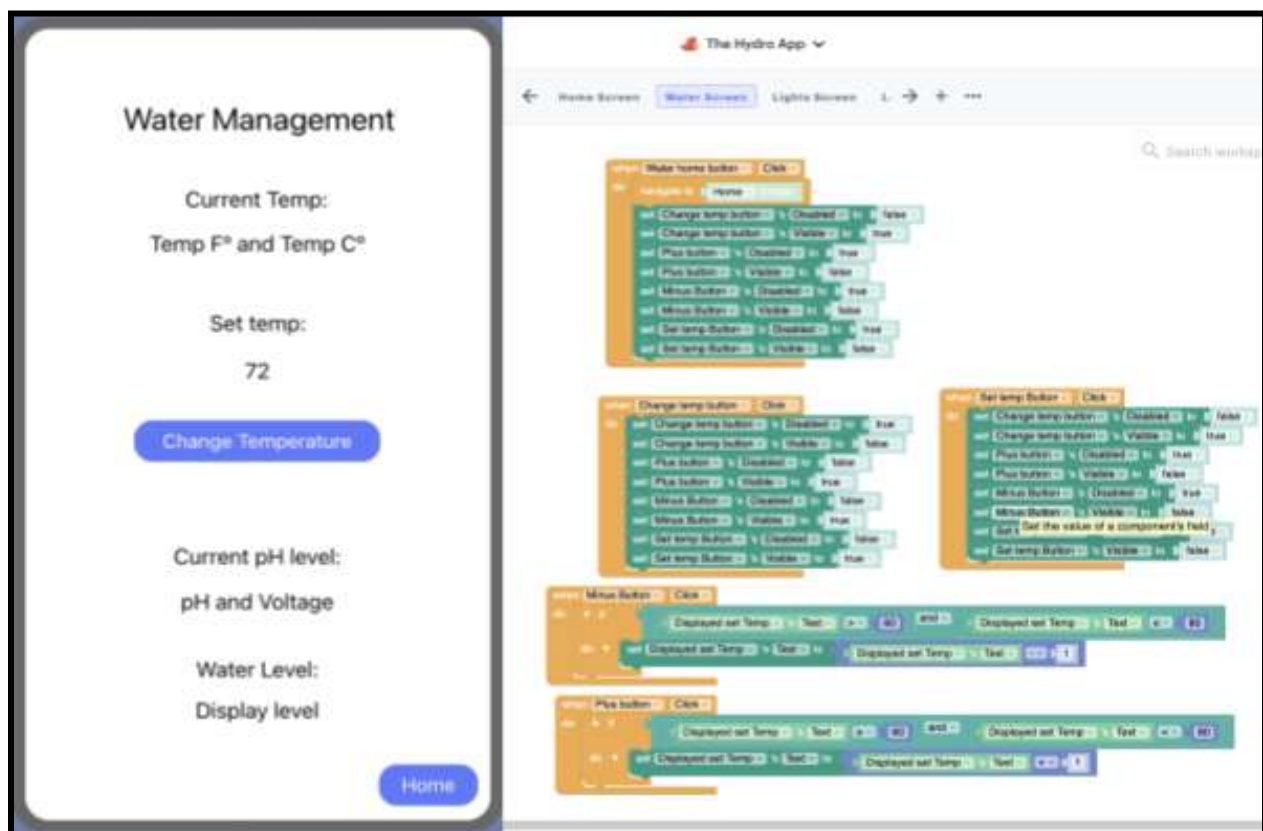
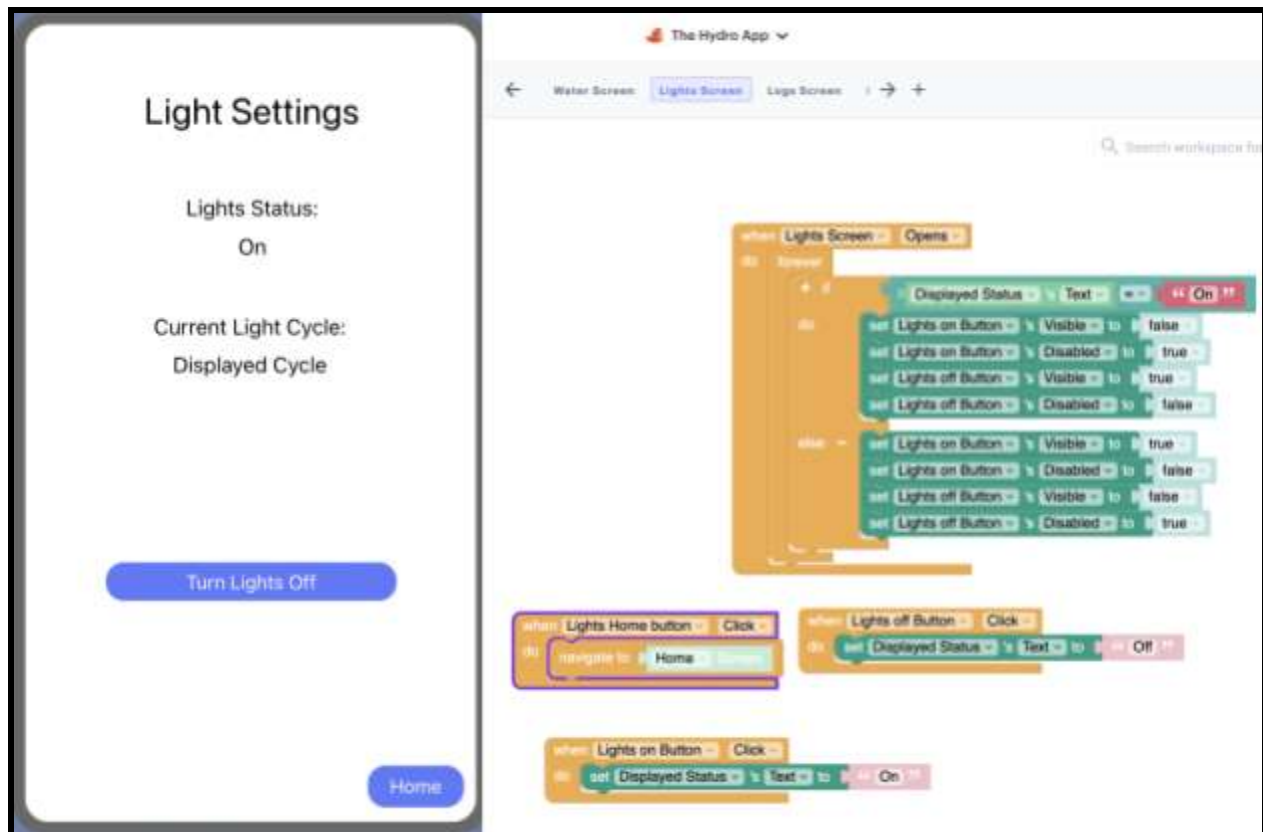


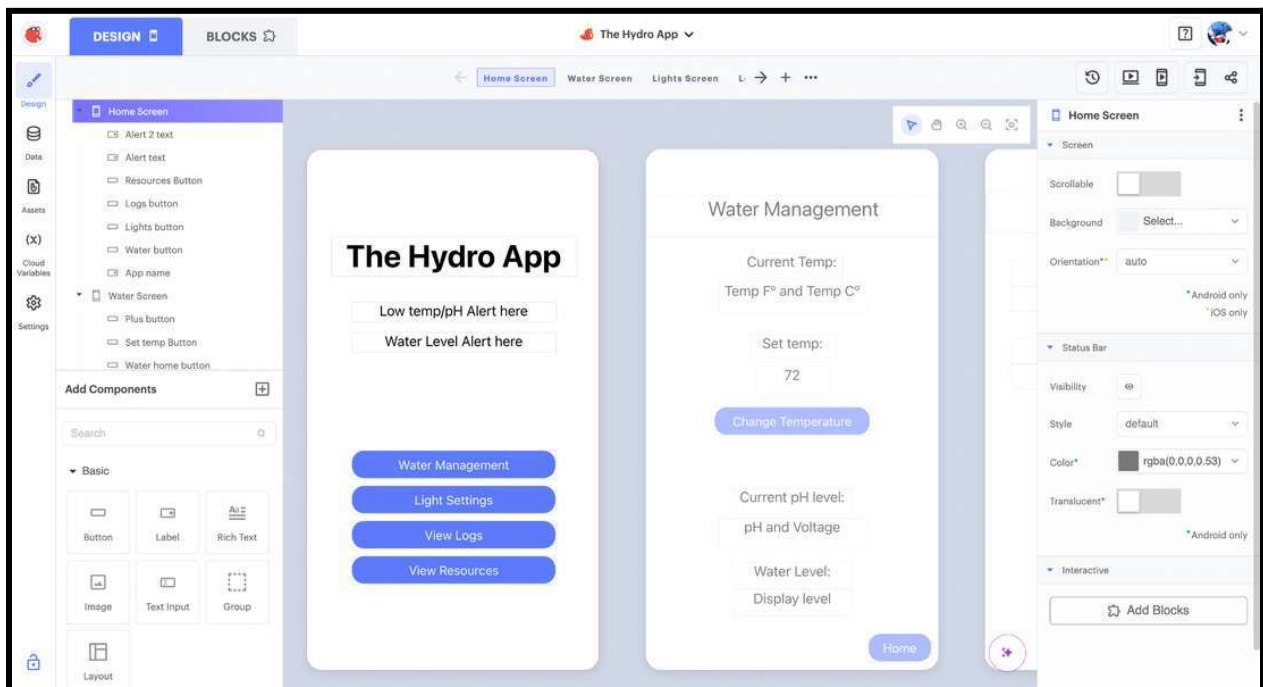
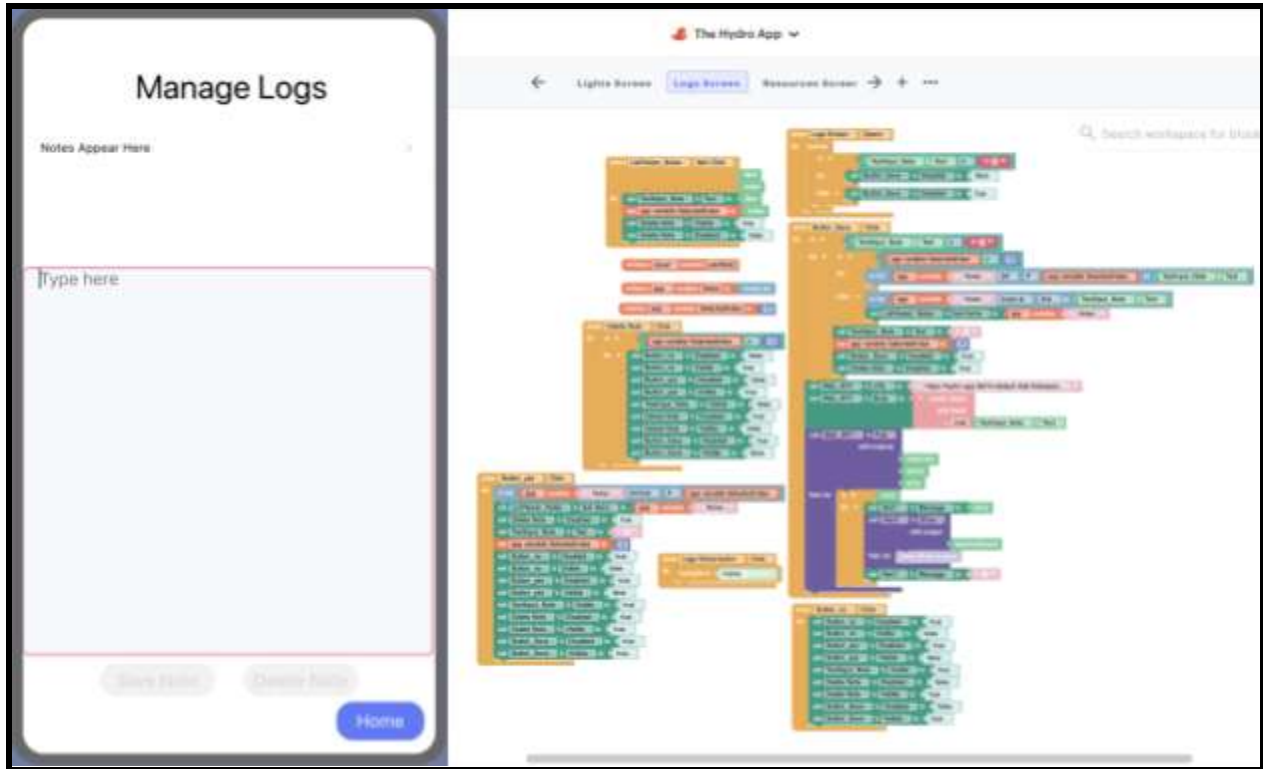
```
pi@raspberrypi: ~
File Edit Tabs Help
pi@raspberrypi:~$ sudo crontab -e
No modification made
pi@raspberrypi:~$ 
pi@raspberrypi:~$ crontab -l
bash: crontab -l: command not found
pi@raspberrypi:~$ crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
@reboot /bin/sleep 30; /usr/bin/python /home/pi/displaySensorData.py
pi@raspberrypi:~$
```

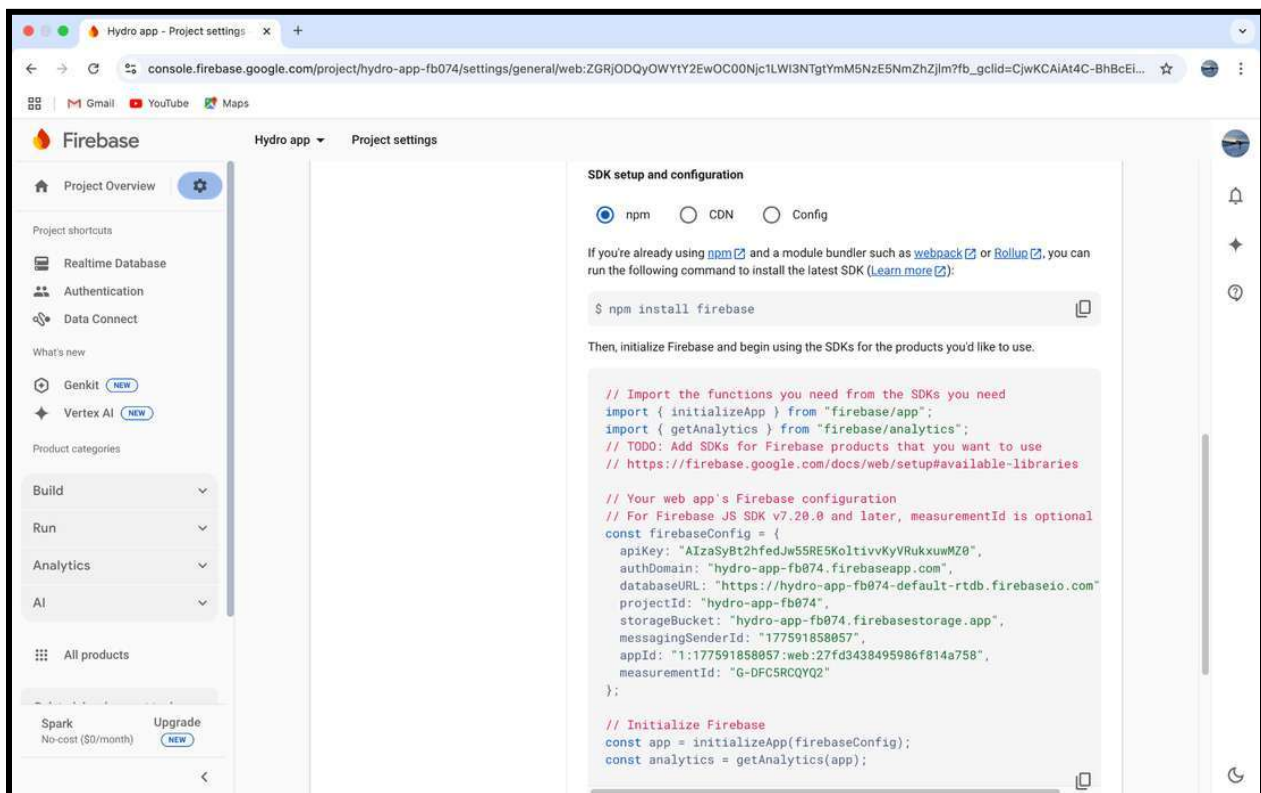
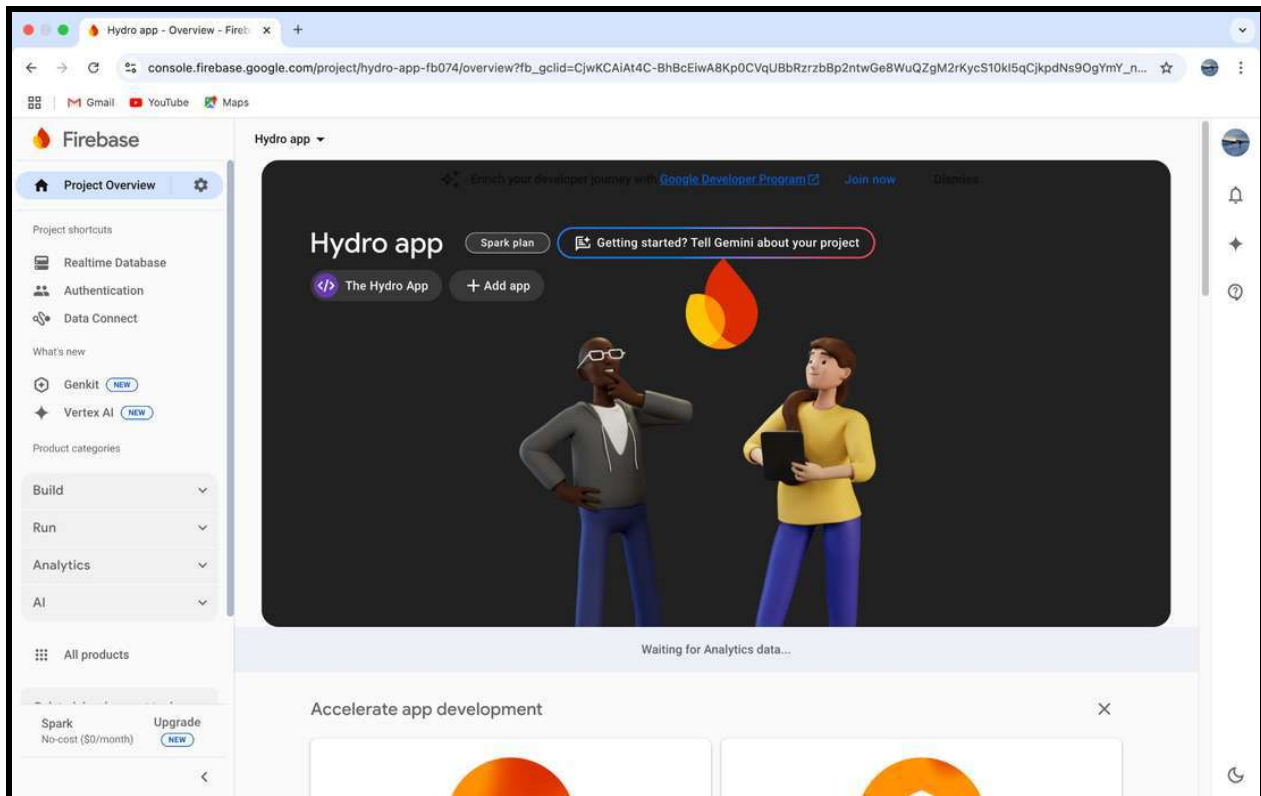
```
pi@raspberrypi: ~
File Edit Tabs Help
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
@reboot /bin/sleep 30; /usr/bin/python /home/pi/displaySensorData.py
0 5 * * * /home/pi/start_displaySensorData.sh
0 21 * * * /home/pi/stop_displaySensorData.sh
pi@raspberrypi:~$ ls -lha displaySensorData.py
-rwxr-xr-x 1 pi pi 3.8K Jan 16 09:38 displaySensorData.py
pi@raspberrypi:~$ sudo crontab -e
crontab: installing new crontab
pi@raspberrypi:~$ sudo crontab -l
```

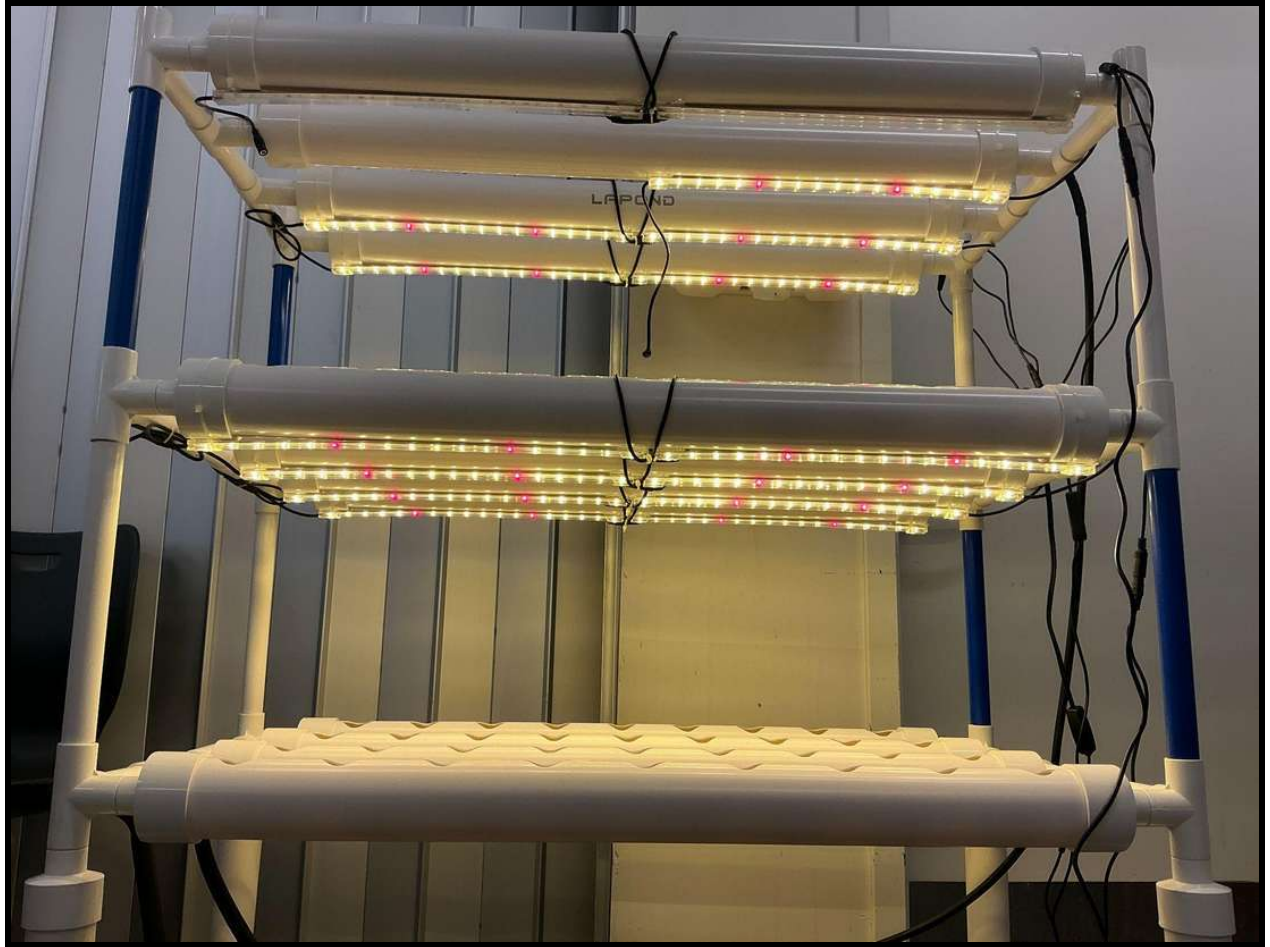


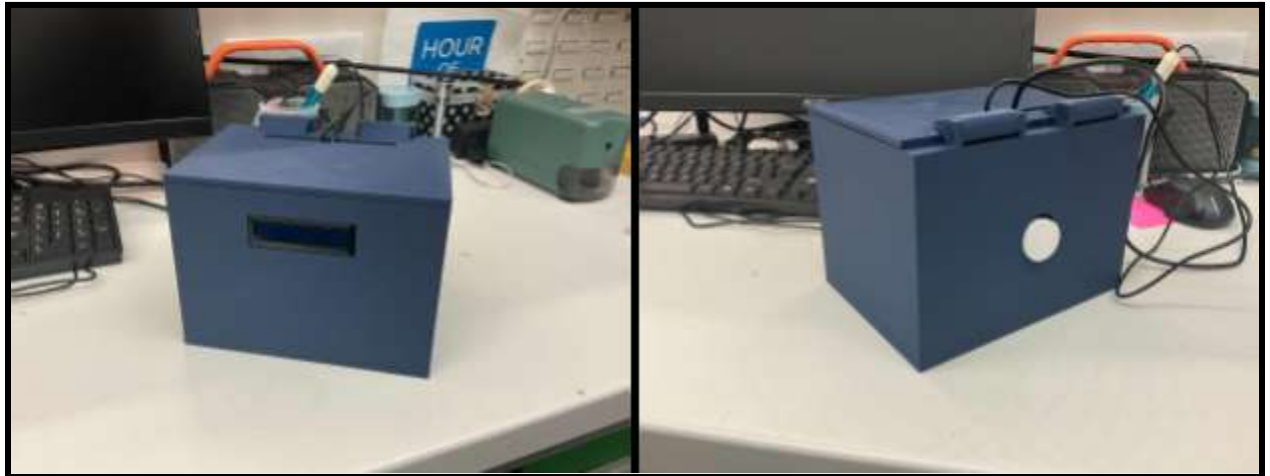




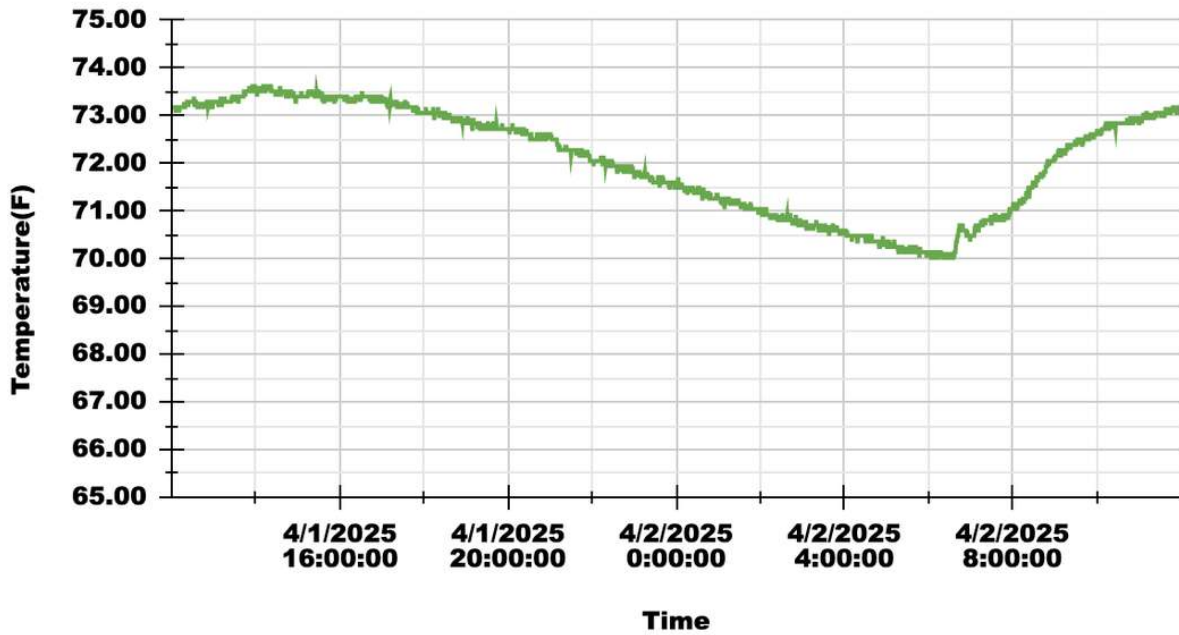




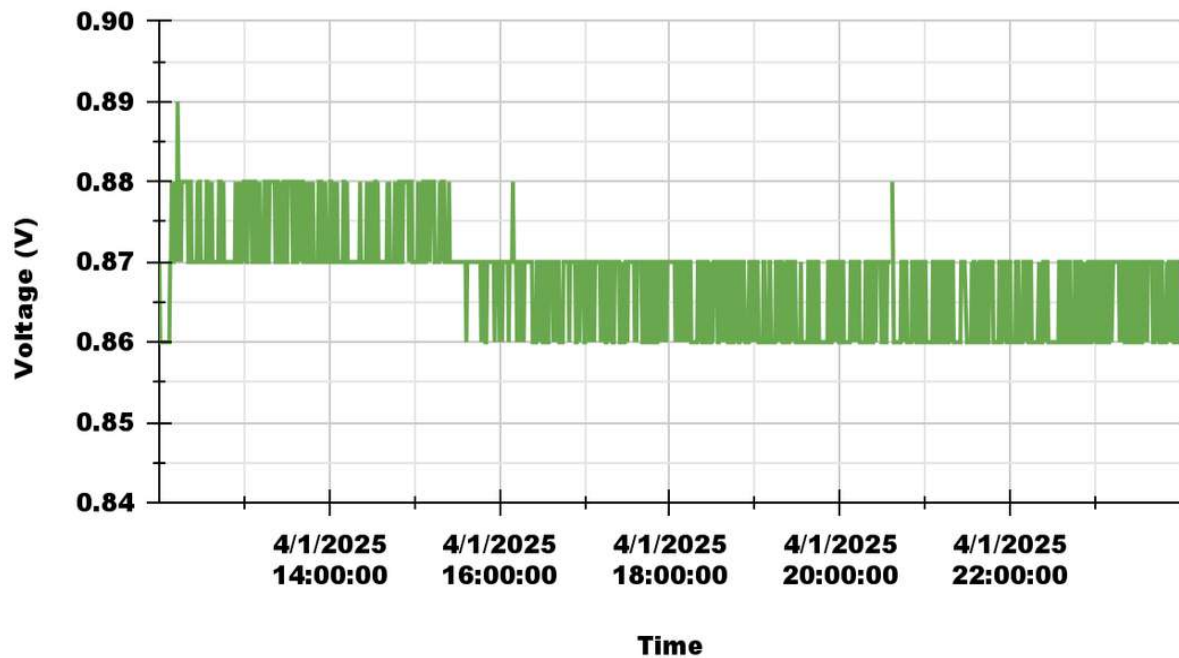




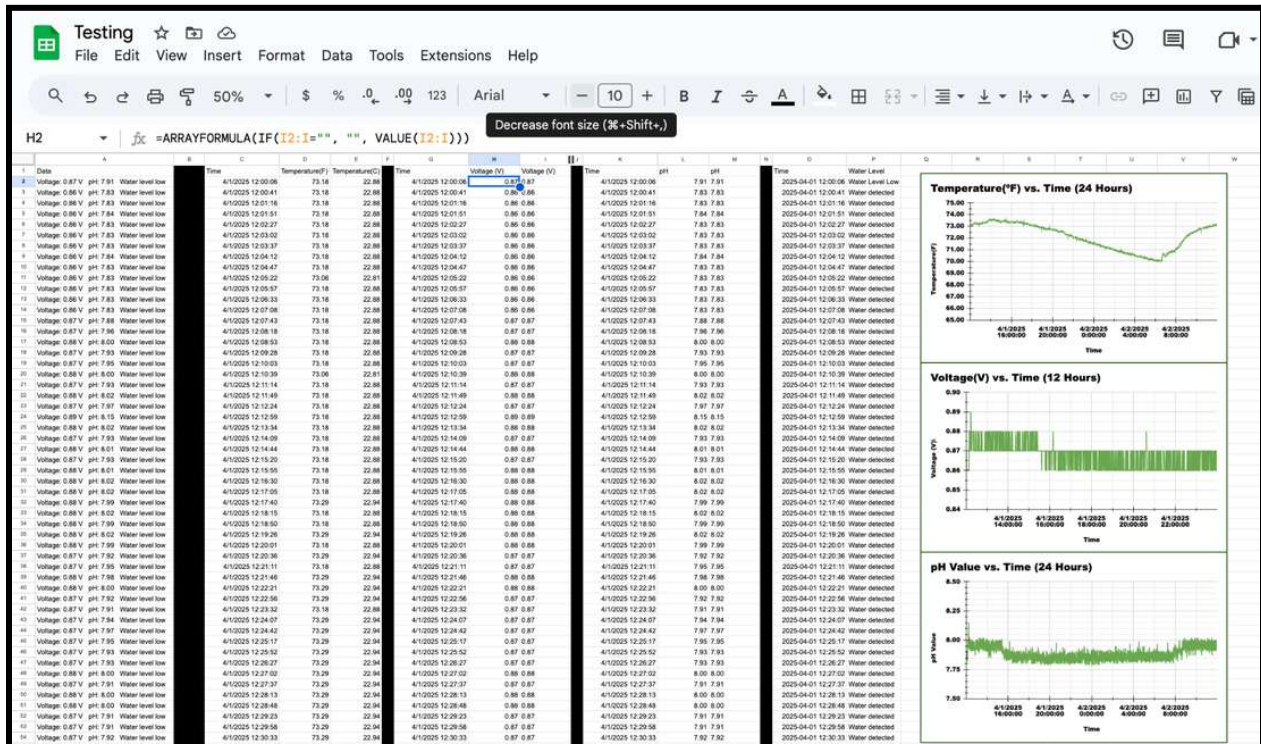
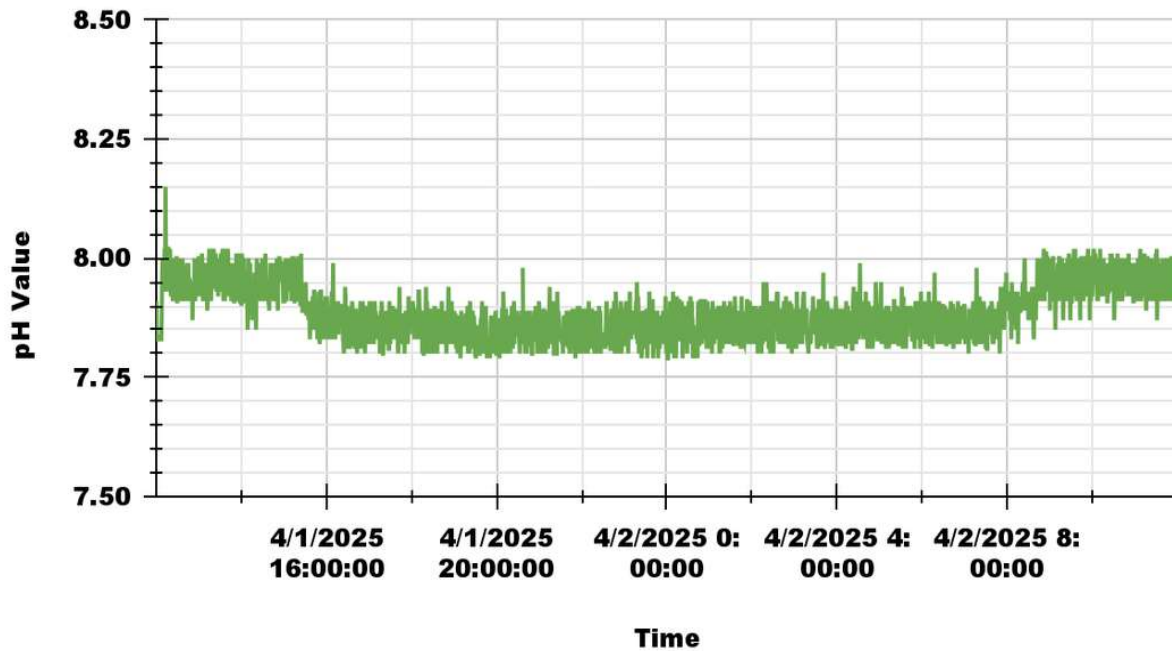
Temperature(°F) vs. Time (24 Hours)



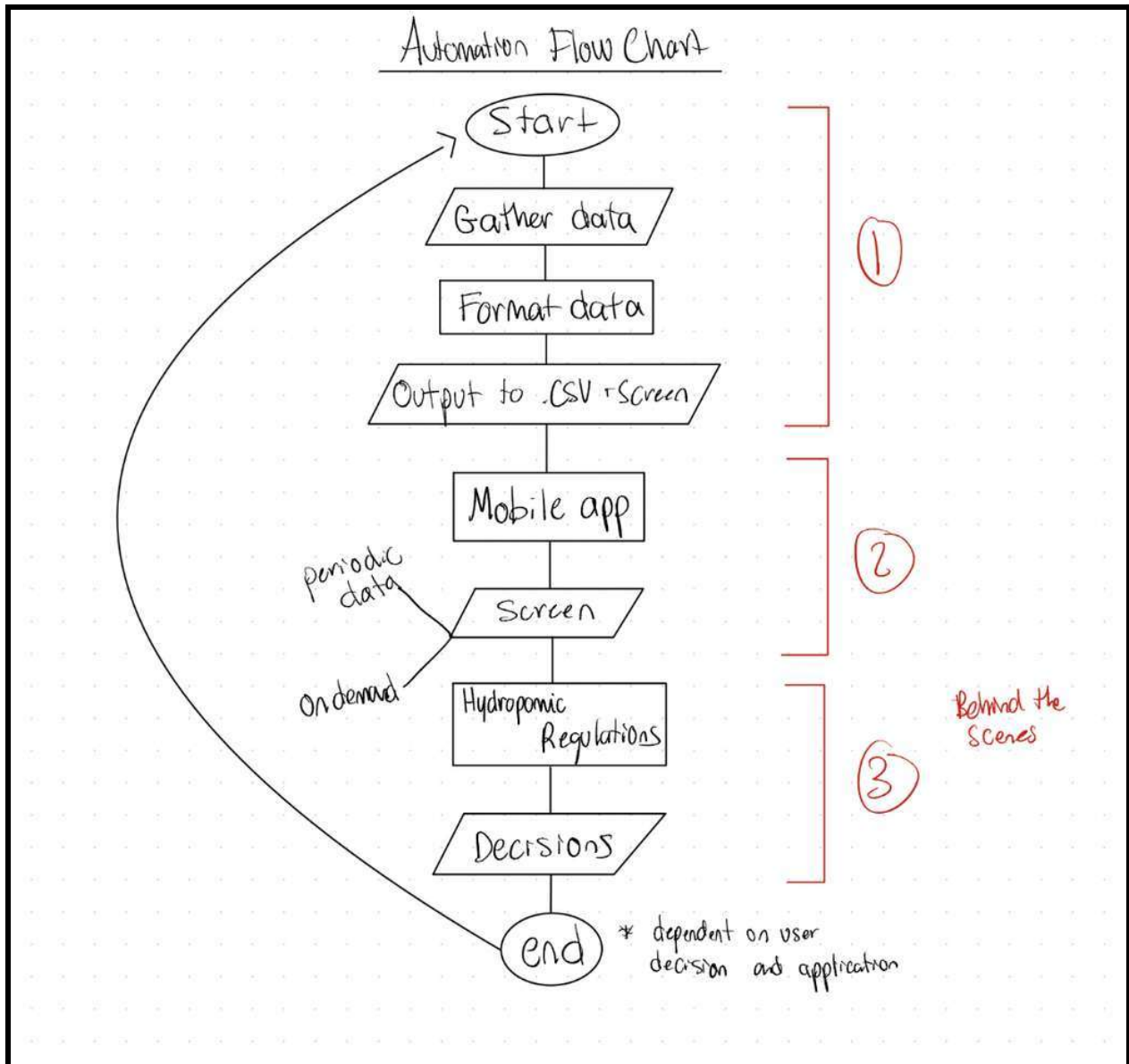
Voltage(V) vs. Time (12 Hours)



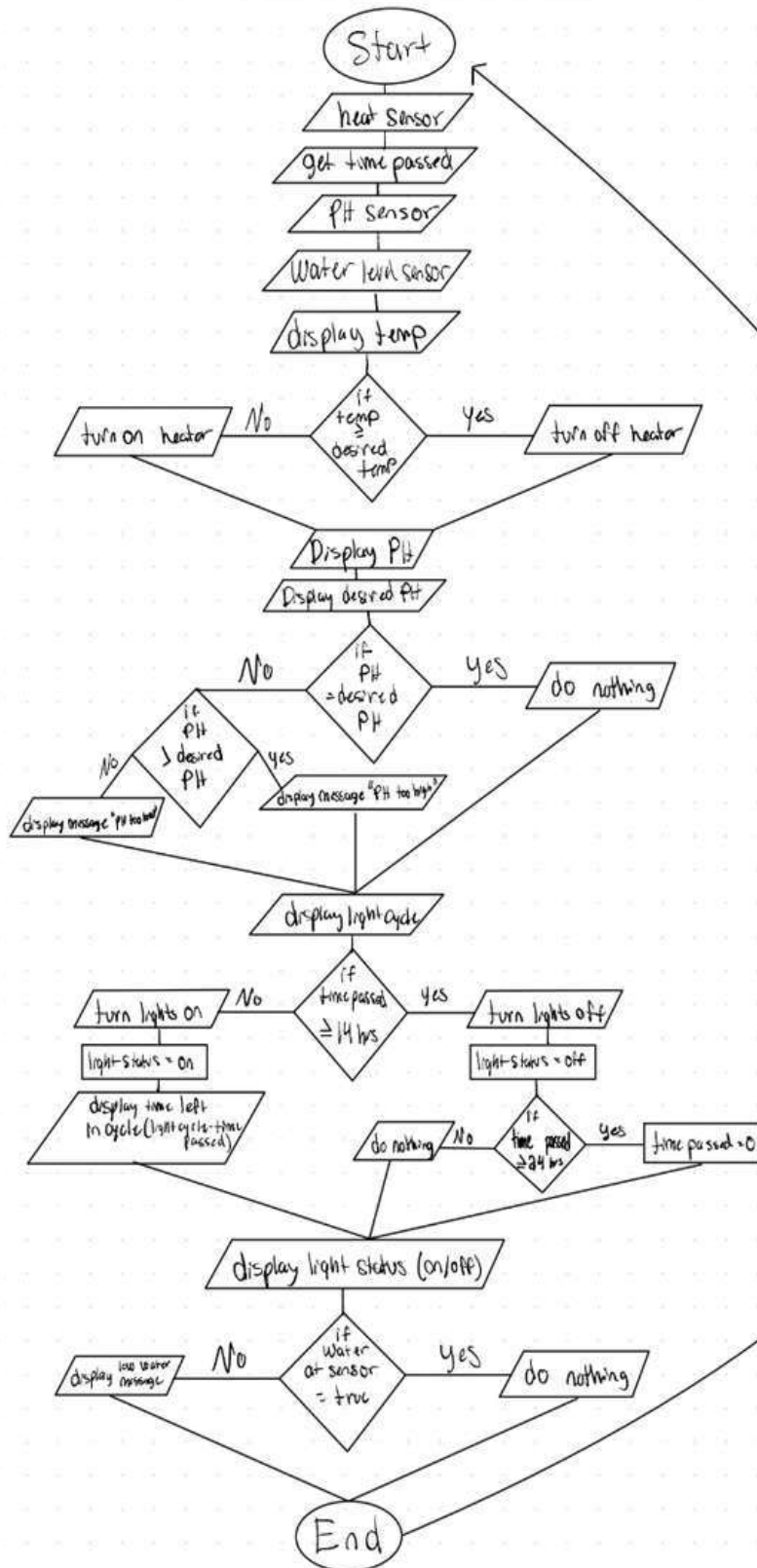
pH Value vs. Time (24 Hours)



Sketches



Automation Flow Chart



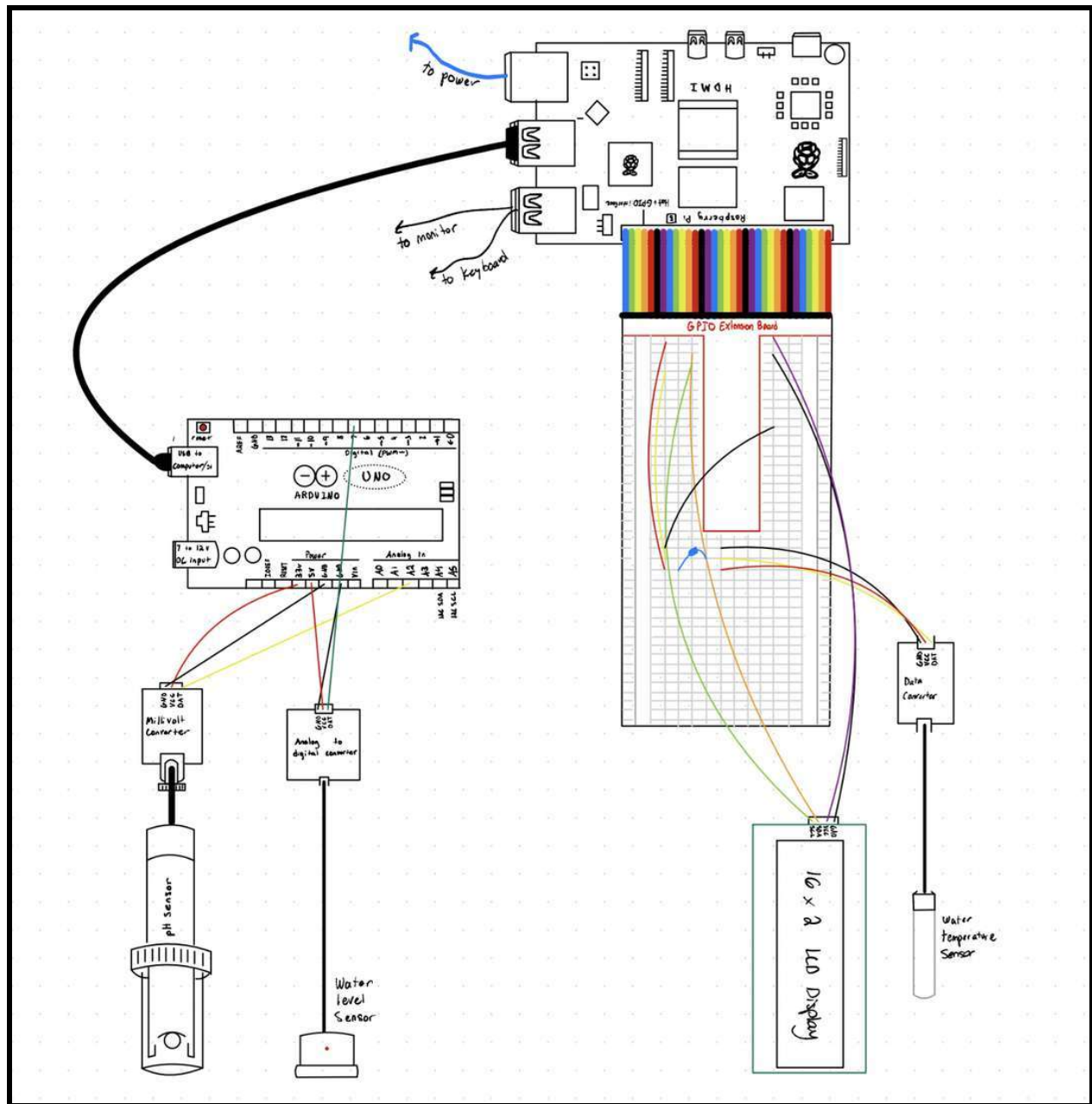
Variables (user input)

- desired temp = 72 (deg)
- desired pH = 5.5
- light cycle = 14 (hours)

Can be in minutes for program, then converted to hours for user

time passed initialized at 0
light status initialized at ON

* Controlling the pH automatically is too advanced and expensive, so this is only sensing it



Mobile App for Controlling and Monitoring 10/24/24

Home Screen

The Hydro App

** Note: displaying 72° is the temp in the hydro app*

Temperature Settings

Light Settings

View PH Level

View Logs

Resources

Temp Settings

72°

Set Temperature

Home

Set temp

72°

72°

- +

Set

Light Settings

Time left in __ hr cycle:

1 hour 56 minutes

Lights Status: On/ Off

On

Off ↕

Home

PH Level

Current PH Level:

5.5

Home

View logs

New note

like the Notes App

Font style bold italic photo library

Home

Resources

Resources

Pocket Garden

When to Plant

MQ Green Thumb

Plant Doctor

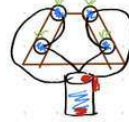
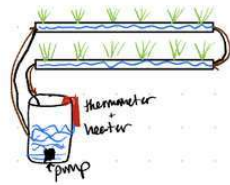
Garden Compass

Home

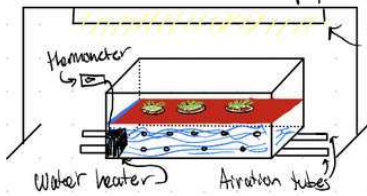
Hydroponic Idea sketches:

#1: PVC Pipes

Pro: has pump to circulate water and limit disease spread
Con: No aeration tubes



#2: Terrarium w pipes on bottom for circulation

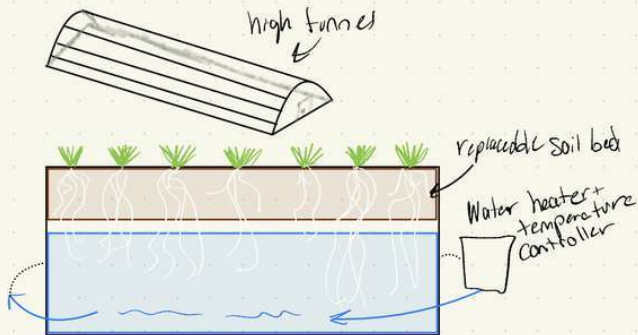


Pro: aeration tube for temp control + gives oxygen to plants

Con: No pump to circulate water

Terrarium sketch/idea #2: Hydroponics

9/20/24



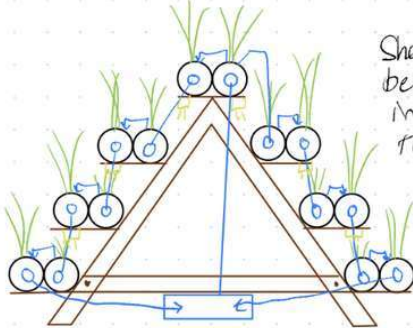
Features:

- High tunnels to protect plants against rising heats
- Ventilation
- Lined with hydroponics
 - ↳ Water can be heated/cooled
 - ↳ filtration pumps water + nutrients
- thermometer
- LED light (to mimic sun)
- Solar panels (power the mechanics)
- Soil bed can be replaced

Hydroponic structure sketch #3 (10/24/24)

I want to have 3 or 4 levels with 4 tubes of plants (2 on each side). Each tube will have 8-10 (probably 9) plant spots.

Side view:

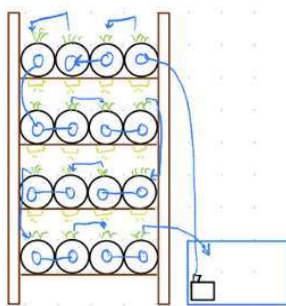


Shorter plants can be placed on the inside, larger on the outside.

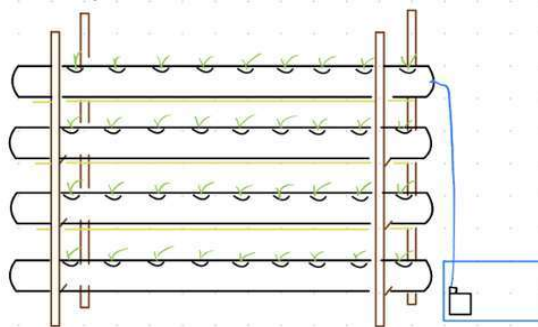
* I might not be able to incorporate lights into this design because plants are spread out. Unless I place lights only over smaller plants that are covered anyways.

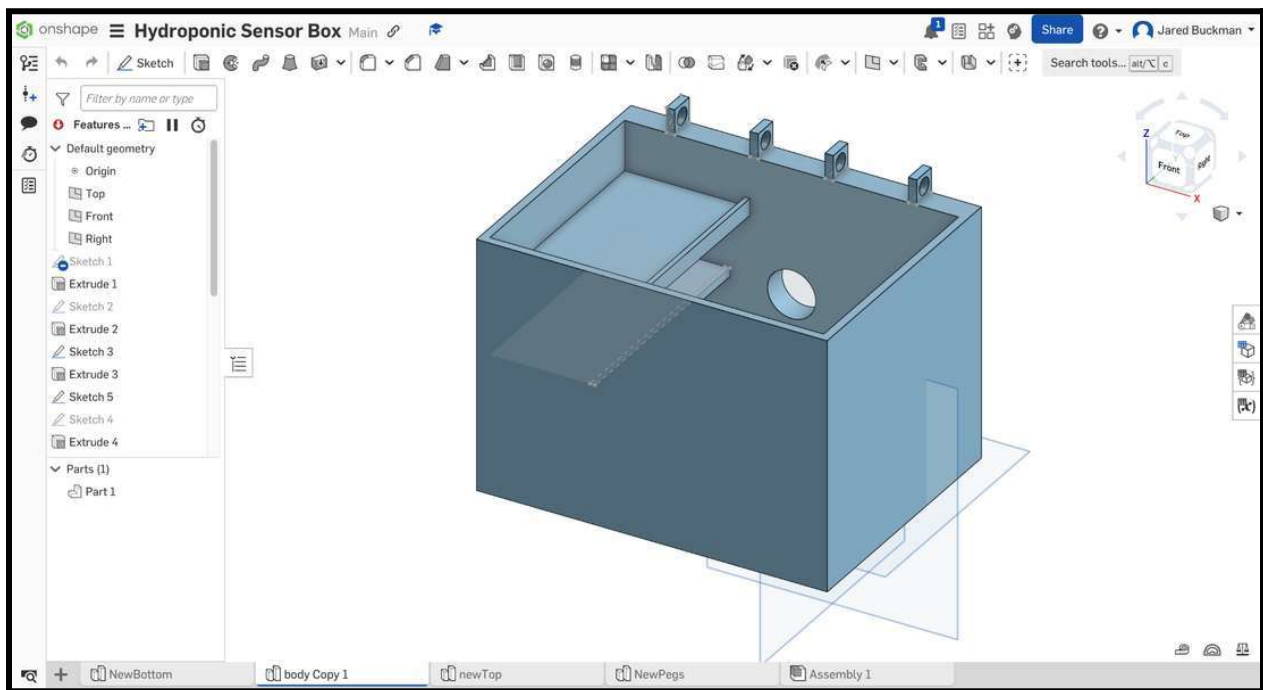
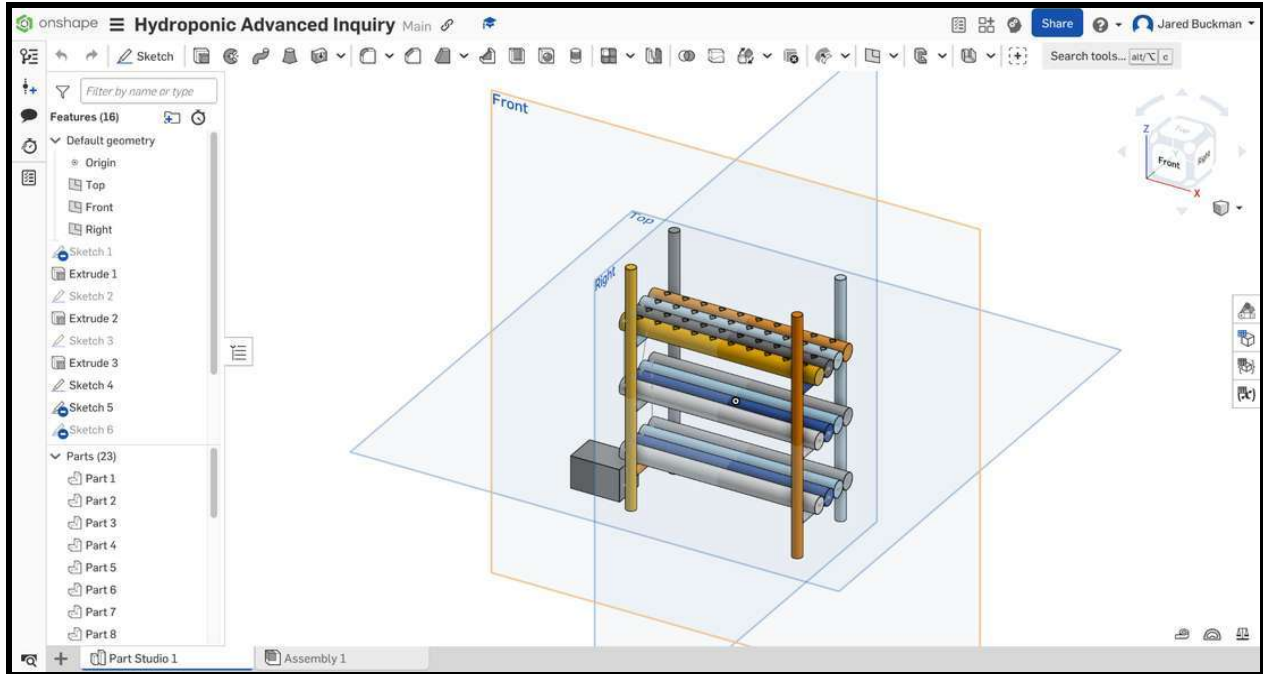
Structure Sketch #4 (10/24/24)

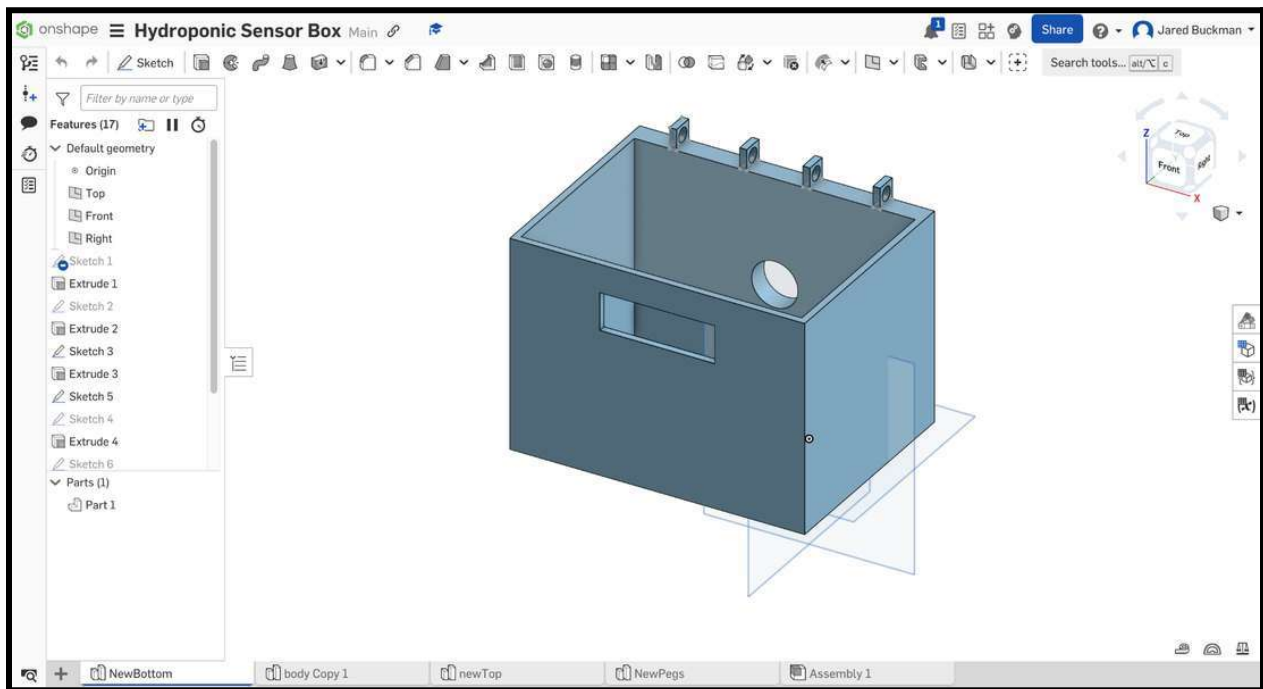
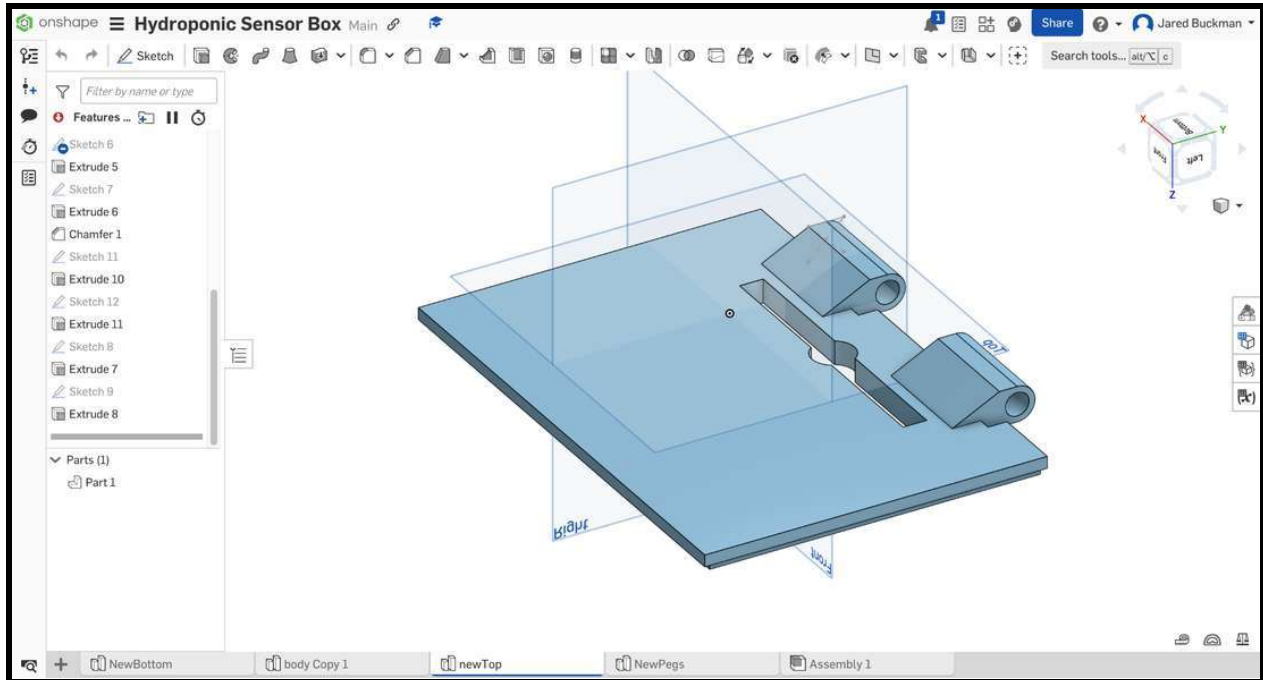
Side view:



Front view:







Communications

Notes on Meeting With Mrs. Borken:

High tunnels: season extenders- sides roll down and up (Waldingfield Farms)

Cool weather crops and water dependent crops are being hurt the most

Ideal and uses geothermal heat: spending too much money on importing foods- opposite issue (weather is too cold there)

How to optimize space (cucumbers) optimize design to think of the space

Hydroponic design (or not?)

Research aspects of a greenhouse

Spray design wastes a lot because it gets on the leaves and plants can't use it

Iceland: indoor growing is better than natural conditions

Warm place: i can take modeling from Iceland and apply it in an arid place like the Middle East

Design: need to get the water to the roots

Desalinization: take the salt out of the water to use for crops: heat the water to separate the salt from water, keep the water. Oil rich countries boarder in the ocean make it cheaper to obtain this water and use energy for this process

Cooling system maybe in Middle East (exhaust fan? Vent the hot air)

Issue is not the crop itself (but focus on one crop so you can scale it to the demand)

- Scientific research client: What problem is she trying to solve

- Hydroponics: you can control the temp of the water + add nutrients



Carly Borken

to me

12:46 PM (20 minutes ago)

☆ ↶ ⋮

Hi Mr. Buckman,

It has been a real treat working with your company, Growing Industries, this year. I am really excited about the work you are producing for my company. Here are some specific requests I have for our unit.

- I want the hydroponic system to make changes automatically when it senses things are off (like turn on a heater when the temp is not right or add nutrients if the levels are not optimal).
- I want to get a notification when the pH or nutrient level drops below optimal on an ap/email
- Tomatoes and lettuces are really important to me culturally, so I would love to have you grow those.

Thanks!

Heiress Carly Borken



Carly Borken | 11th grade Dean, Science Faculty, Upper School Sustainability Coordinator

35 Beachside Avenue | Westport, CT 06880

P: 203-349-7410

cborken@gfacademy.org | www.gfacademy.org








Nicholas S. Kaczmar

to me (last)

4:07 PM (20/20/2020, 11:00AM) (8 days ago)

☆ ↶ ⋮

Hi Jared,

You're doing a great job- super interesting and you're on the right track. The sensing/ IoT control approach you're taking is great- we've been working on several very similar projects here. We had a recent PhD student (who is now working as a Research Scientist at Meta) do a very similar project. He was growing lettuce in NFT, with similar sensors for to monitor reservoir pVEC and environmental variable such as temp/humidity/light intensity. He also included scales that were weighing the water reservoir, and each NFT channel. This was to monitor the water use of the plants, as well as monitor how much weight the plants were gaining in real time. He also had cameras that took photos of the plants every 15 minutes. This provided a solid data set for the entire growth cycle of the plant. We did several growth cycles, changing small environmental variables (temp or light), to see how it would impact plant growth. He then used this as a training data set for an AI model to create a "digital twin" that could predict plant growth for any change in environmental variables.

In regards to your questions- responses in RED

- I was planning on growing tomatoes to use them as an example of one crop that is harmed by climate change and using California as an example location. Do you have any knowledge or research on this topic you can share? Do you think tomatoes would grow best in my system?
 - Unfortunately your system is too small for tomatoes. Typically tomatoes are grown in rockwool substrate on drip- and are grown for ~9-12 months at a time. They are grown on high wires that are 4m high and grow the plants in a way that they typically get ~20-30 meters in length. (see attached photo) if you really want to grow tomatoes- then I would recommend looking at dwarf varieties such as "Tiny Tim" (<https://victoryseeds.com/products/tiny-tim-tomato?srsltid=AfmBOop6f8Mz7TMB8x6v8C8BpOK8RafE1yG021NafELea1H509>). These have a small compact growth form and would fit in that system. Please keep in mind that it would take ~3 months from seed to get fruit.
 - I would recommend that you start with leafy greens such as lettuce, as they are easy plants for beginners and have a relatively quick growth cycle (35 days from seed). You could also look at other leafy greens such as basil and kale. Keep in mind that the majority of feed lettuce is grown in California and Arizona, so there would still be a climate argument for you growing that crop.
 - Rave butterhead lettuce is the variety that I would recommend starting with- it's our "lab rat". https://www.igrowseeds.com/vegetables/lettuce/butterhead-lettuce-boston/ave-pelleted-lettuce-seed-2967JP11.html?utm_source=google&utm_medium=coop&utm_campaign=BL%20%20PMAX%20%20Vegetables&utm_keyword=Seed_source%3F&id=C0XGQAwu9BtG8ARuA9J2s4fC505vnuhthVedHq2hepl1m-aTR4C6bA9bA9bW59bGSRH4aafmv_EALw_wcB
- How would you begin growing your plants? I plan to use foam board to incubate the seeds until they sprout, and then transfer them to the NFT system.
 - I always start my seeds in 1" rockwool cubes. Water with a basic fertilizer solution, don't squeeze or change the form of the cubes. I would seed 10-20% more than you need in case of poor germination etc. Keep under a light for ~14 days, watering daily. Transplant at this point.
 - The holes in your system appear to be quite large. Did it come with netpots? If not, you will need netpots of appropriate size to hold your seedling in place.
- Do you have any suggestions on what else I can do with my project? The difference maker in my project is the automation side and the app that allows users to log progress and hopefully soon, upload photos to the app, but the more research I do, the more I see existing companies with similar products. Any suggestions on how to differentiate my project?
 - There are quite a bit of people who are building very similar systems to you. I'd look at the PiGrow community as it is an open source platform that is doing something very similar. I think adding a camera that takes photos at set durations would be helpful- if you're into Python- check out PlantCV. It is a software library to measure plant size from photos in real time on a Pi. It could be helpful in letting the user know the optimum time to harvest the plant. Also consider adding a temp/RH/h2o sensor to the unit for environmental monitoring. Water temp- isn't the most crucial aspect to look at- typically you need to chill the water more than heat it (but I rarely do either). If you are looking for a heater for your reservoir- I would use a basic aquarium heater controlled by a relay with the Arduino. Monitoring of EC is pretty important as well as it will tell the user when more nutrients are required.

Something very important that you are missing is lighting. I'm not the biggest fan of that particular NFT system as you need to light each level individually or else the NFT above it shades the plants out. So you will need to get lights for each level. There are a ton of different options out there- but it really depends on your budget. Let me know what you have to work with and I can provide you with a few recommendations.

Hope this helps. You're doing an awesome job- keep up the good work! Let me know if you're ever in theta- I'd be happy to give you a tour of our facilities and projects.

Nicholas Kaczmar
 Research Support Specialist II
 School of Integrative Plant Science
 Horticulture Section
 172 Kottrell Plant Lab
 College of Agriculture and Life Sciences
 Cornell University
 315-540-7341
 nsk1@cornell.edu | nsk1@cornell.edu
<http://www.cals.cornell.edu/>

Codebase 1.1 (Water Temperature Sensor)

```
import os
import glob
import time

#these tow lines mount the device:
os.system('modprobe w1-gpio')
os.system('modprobe w1-therm')

base_dir = '/sys/bus/w1/devices/'
device_path = glob.glob(base_dir + '28*')[0] #get file path of sensor
rom = device_path.split('/')[-1] #get rom name

def read_temp_raw():
    with open(device_path + '/w1_slave','r') as f:
        valid, temp = f.readlines()
    return valid, temp

def read_temp():
    valid, temp = read_temp_raw()

    while 'YES' not in valid:
        time.sleep(0.2)
        valid, temp = read_temp_raw()

    pos = temp.index('t=')
    if pos != -1:
        #read the temperature .
        temp_string = temp[pos+2:]
        temp_c = float(temp_string)/1000.0
        temp_f = temp_c * (9.0 / 5.0) + 32.0
        return temp_c, temp_f

print(' ROM: ' + rom)

with open("TemperatureReadings.txt", "w") as file:
    file.write("Temp(C) Temp(F)")
    file.write("\n")
    for i in range(3):
        c, f = read_temp()
        print('C={:,.3f} F={:,.3f}'.format(c, f))
        file.write('C={:,.3f} F={:,.3f}'.format(c, f))
        file.write("\n")
        time.sleep(1)
    file.close()
```

Codebase 2.1 (pH Sensor)

```
#define SensorPin A0          // the pH meter Analog output is connected with the Arduino's
Analog
unsigned long int avgValue; //Store the average value of the sensor feedback
float b;
int buf[10],temp;
void setup()
{
    pinMode(13,OUTPUT);
    Serial.begin(9600);
    Serial.println("Ready");    //Test the serial monitor
}

void loop()
{
    for(int i=0;i<10;i++)        //Get 10 sample value from the sensor for smooth the value
    {
        buf[i]=analogRead(SensorPin);
        delay(10);
    }
    for(int i=0;i<9;i++)        //sort the analog from small to large
    {
        for(int j=i+1;j<10;j++)
        {
            if(buf[i]>buf[j])
            {
                temp=buf[i];
                buf[i]=buf[j];
                buf[j]=temp;
            }
        }
    }
    avgValue=0;
    for(int i=2;i<8;i++)        //take the average value of 6 center sample
        avgValue+=buf[i];
    float pHValue=(float)avgValue*5.0/1024/6; //convert the analog into millivolt
    pHValue=3.5*pHValue;        //convert the millivolt into pH value
    Serial.print("    pH:");
    Serial.print(pHValue,2);
    Serial.println(" ");
    digitalWrite(13, HIGH);
    delay(800);
    digitalWrite(13, LOW);
}
```

Codebase 2.2 (Improvements to pH Sensor Code)

```
#define SensorPin A0 //The pH meter Analog output is connected with the Arduino's Analog
unsigned long int avgValue; //Store the average value of the sensor feedback
float b = -2.65;
float m = 0.0391;
int buf[10],temp;

void setup()
{
  pinMode(13,OUTPUT);
  Serial.begin(9600);
  Serial.println("Ready"); //Test the serial monitor
}

void loop()
{
  for(int i=0;i<10;i++) //Get 10 sample values from the sensor to smooth the value
  {
    buf[i]=analogRead(SensorPin);
    delay(20);
  }
  for(int i=0;i<9;i++) //sort the analog from small to large
  {
    for(int j=i+1;j<10;j++)
    {
      if(buf[i]>buf[j])
      {
        temp=buf[i];
        buf[i]=buf[j];
        buf[j]=temp;
      }
    }
  }
}

avgValue=0;
for(int i=2;i<8;i++) //take the average value of 6 center sample
  avgValue+=buf[i];

float voltage=(float)avgValue*3.3/1024/6; //convert the analog into millivolt
float pHValue=(float)avgValue/6; //take average of 6 samples
float calibratedPH = m*pHValue + b; //convert the analog data into pH
value (in millivots)

Serial.print("Voltage:"); //print voltage heading
Serial.print(voltage, 2); //print voltage data
Serial.print(" pH:"); //print pH heading same line
Serial.print(calibratedPH,2); //print pH data
Serial.println(" ");
digitalWrite(13, HIGH);
delay(800);
digitalWrite(13, LOW);
}
```

Codebase 3.1 (Water Level Sensor)

```
//Define the sensor pin
const int sensorPin = 7; //Digital pin connected to the sensor's signal (SIG) pin

void setup() {
  // Initialize the sensor pin as an input
  pinMode(sensorPin, INPUT);

  // Start serial communication for debugging
  Serial.begin(9600);
  Serial.println("Ready:");
}

void loop() {
  // Read the sensor value
  int sensorState = digitalRead(sensorPin);

  // Check if water is detected
  if (sensorState == HIGH) {
    Serial.println("Water detected");
  } else {
    Serial.println("Water level low. Add more water");
  }

  // Delay for stability
  delay(2000); // 2000 milliseconds
}
```

Codebase 4.1 (Joint pH and Water Level Code on Arduino)

```
// pH Sensor Definitions
#define SensorPin A0 // The pH meter Analog output is connected to Arduino's Analog A0
unsigned long int avgValue; // Store the average value of the sensor feedback
float b = -2.65; // Calibration offset
float m = 0.0391; // Calibration slope
int buf[10], temp;

// Water Level Sensor Definitions
const int waterSensorPin = 7; // Digital pin connected to the water sensors signal (SIG) pin

void setup() {
  // Setup pH sensor
  pinMode(13, OUTPUT);
  Serial.begin(9600);
  Serial.println("System Ready!"); // Initialization message

  // Setup water level sensor
  pinMode(waterSensorPin, INPUT);
}

void loop() {
  for (int i = 0; i < 10; i++) { // Get 10 sample values from the pH sensor
    buf[i] = analogRead(SensorPin);
    delay(15);
  }

  // Sort the analog values from small to large
  for (int i = 0; i < 9; i++) {
    for (int j = i + 1; j < 10; j++) {
      if (buf[i] > buf[j]) {
        temp = buf[i];
        buf[i] = buf[j];
        buf[j] = temp;
      }
    }
  }

  avgValue = 0;
  for (int i = 2; i < 8; i++) { // Take the average value of the 6 center samples
    avgValue += buf[i];
  }

  float voltage = (float)avgValue * 3.3 / 1024 / 6; // Convert the analog value to
millivolts
  float pHValue = (float)avgValue / 6; // Average of 6 samples
  float calibratedPH = m * pHValue + b; // Convert the analog data to pH value

  // Print pH sensor readings
  Serial.print("Voltage: ");
  Serial.print(voltage, 2);
  Serial.print(" V pH: ");
```

```
Serial.print(calibratedPH, 2);  
Serial.println();  
  
int sensorState = digitalRead(waterSensorPin);  
  
// Check if water is detected  
if (sensorState == HIGH) {  
    Serial.println("Water detected.");  
} else {  
    Serial.println("Water level low. Add more water.");  
}
```

Codebase 5.1 (Accessing the Arduino Code From Raspberry Pi)

```
arduino_port = "/dev/ttyACM0" # port name

import serial
import csv
import time

# Configure the serial connection
arduino_port = "/dev/ttyACM0"
baud_rate = 9600
csv_file = "data.csv"

try:
    # Open serial port
    ser = serial.Serial(arduino_port, baud_rate, timeout=1)
    print("Connected to Arduino on", arduino_port)

    # Open CSV file for writing
    with open(csv_file, mode='w', newline='') as file:
        writer = csv.writer(file)
        writer.writerow(["Time", "Temperature", "Humidity"]) # Write header row

    # Continuously read from Arduino
    while True:
        line = ser.readline().decode('utf-8').strip() # Read and decode data
        if line:
            print("Data received:", line)
            data = line.split(",") # Split CSV string into list
            writer.writerow(data) # Write to CSV
            file.flush() # Ensure data is written to file
except serial.SerialException as e:
    print(f"Error connecting to Arduino: {e}")
except KeyboardInterrupt:
    print("Exiting program.")
finally:
    if 'ser' in locals() and ser.is_open:
        ser.close()
    print("Serial connection closed.")
```

Codebase 6.1 (LCD Display Panel)

```
import smbus
import time

# I2C address of the LCD
I2C_ADDR = 0x27
LCD_WIDTH = 16 # Maximum characters per line

# LCD commands
LCD_CHR = 1 # Sending data
LCD_CMD = 0 # Sending command
LCD_LINE_1 = 0x80 # Address for 1st line
LCD_BACKLIGHT = 0x08 # Backlight on

ENABLE = 0b00000100 # Enable bit
E_DELAY = 0.0005

# Initialize I2C bus
bus = smbus.SMBus(1)

def lcd_init():
    """Initialize the LCD display."""
    lcd_byte(0x33, LCD_CMD) # Initialize
    lcd_byte(0x32, LCD_CMD) # Set to 4-bit mode
    lcd_byte(0x06, LCD_CMD) # Move cursor right
    lcd_byte(0x0C, LCD_CMD) # Turn cursor off
    lcd_byte(0x28, LCD_CMD) # 2 line display
    lcd_byte(0x01, LCD_CMD) # Clear display
    time.sleep(E_DELAY)

def lcd_byte(bits, mode):
    """Send a byte to the LCD."""
    bits_high = mode | (bits & 0xF0) | LCD_BACKLIGHT
    bits_low = mode | ((bits << 4) & 0xF0) | LCD_BACKLIGHT

    # Send high bits
    bus.write_byte(I2C_ADDR, bits_high)
    lcd_toggle_enable(bits_high)

    # Send low bits
    bus.write_byte(I2C_ADDR, bits_low)
    lcd_toggle_enable(bits_low)

def lcd_toggle_enable(bits):
    """Toggle the enable pin."""
    time.sleep(E_DELAY)
    bus.write_byte(I2C_ADDR, (bits | ENABLE))
    time.sleep(E_DELAY)
    bus.write_byte(I2C_ADDR, (bits & ~ENABLE))
    time.sleep(E_DELAY)

def lcd_string(message, line):
```

```

    """Send a string to the LCD."""
    message = message.ljust(LCD_WIDTH, " ")
    lcd_byte(line, LCD_CMD)
    for i in range(LCD_WIDTH):
        lcd_byte(ord(message[i]), LCD_CHR)

def main():
    """Main program to display 'Hello, World!'."""
    lcd_init()
    lcd_string("Hello, World!", LCD_LINE_1)
    time.sleep(3)

if __name__ == "__main__":
    main()

```

Codebase 7.1 (Display Sensor Data on LCD)

```
# /usr/bin/env python3
import os
import glob
import time
import serial
import csv
import smbus

# --- Water Temperature Sensor Setup ---
os.system('modprobe w1-gpio')
os.system('modprobe w1-therm')

base_dir = '/sys/bus/w1/devices/'
device_path = glob.glob(base_dir + '28*')[0] # Get file path of sensor
rom = device_path.split('/')[-1] # Get ROM name

def read_temp_raw():
    with open(device_path + '/w1_slave', 'r') as f:
        valid, temp = f.readlines()
    return valid, temp

def read_temp():
    valid, temp = read_temp_raw()
    while 'YES' not in valid:
        time.sleep(0.5)
        valid, temp = read_temp_raw()

    pos = temp.index('t=')
    if pos != -1:
        temp_string = temp[pos + 2:]
        temp_c = format(round(float(temp_string) / 1000.0, 2), '.2f') # Round to 2 decimals
        temp_f = format(round(float(temp_c) * (9.0 / 5.0) + 32.0, 2), '.2f') # Round to 2
decimals
        return temp_c, temp_f

# --- LCD Setup ---
I2C_ADDR = 0x27
LCD_WIDTH = 16
LCD_CHR = 1
LCD_CMD = 0
LCD_LINE_1 = 0x80
LCD_LINE_2 = 0xC0
LCD_BACKLIGHT = 0x08
ENABLE = 0b00000100

E_DELAY = 0.005

bus = smbus.SMBus(1)

def lcd_init():
    lcd_byte(0x33, LCD_CMD)
```

```

    lcd_byte(0x32, LCD_CMD)
    lcd_byte(0x06, LCD_CMD)
    lcd_byte(0x0C, LCD_CMD)
    lcd_byte(0x28, LCD_CMD)
    lcd_byte(0x01, LCD_CMD)
    time.sleep(E_DELAY)

def lcd_byte(bits, mode):
    bits_high = mode | (bits & 0xF0) | LCD_BACKLIGHT
    bits_low = mode | ((bits << 4) & 0xF0) | LCD_BACKLIGHT
    bus.write_byte(I2C_ADDR, bits_high)
    lcd_toggle_enable(bits_high)
    bus.write_byte(I2C_ADDR, bits_low)
    lcd_toggle_enable(bits_low)

def lcd_toggle_enable(bits):
    time.sleep(E_DELAY)
    bus.write_byte(I2C_ADDR, (bits | ENABLE))
    time.sleep(E_DELAY)
    bus.write_byte(I2C_ADDR, (bits & ~ENABLE))
    time.sleep(E_DELAY)

def lcd_string(message, line):
    message = message.ljust(LCD_WIDTH, " ")
    lcd_byte(line, LCD_CMD)
    for i in range(LCD_WIDTH):
        lcd_byte(ord(message[i]), LCD_CHR)

def lcd_scroll_string(message, line):
    """Scroll a long message across the LCD line."""
    if len(message) <= LCD_WIDTH:
        lcd_string(message, line)
    else:
        for i in range(len(message) - LCD_WIDTH + 1):
            lcd_string(message[i:i+LCD_WIDTH], line)
            time.sleep(0.2)

# --- Arduino Setup ---
arduino_port = "/dev/ttyACM0"
baud_rate = 9600

# --- Combined Data Writing ---
output_file = "CombinedData.csv"

try:
    ser = serial.Serial(arduino_port, baud_rate, timeout=1)
    print("Connected to Arduino on", arduino_port)

    lcd_init()
    lcd_string("Initializing...", LCD_LINE_1)

    with open(output_file, mode='w', newline='') as file:

```

```

writer = csv.writer(file)
writer.writerow(["Time", "Temperature(C)", "Temperature(F)", "Voltage", "pH", "Water
Level"])

print(f'ROM: {rom}')
time.sleep(3)

while True:
    temp_c, temp_f = read_temp()
    timestamp = time.strftime("%Y-%m-%d %H:%M:%S")

    arduino_data = ser.readline().decode('utf-8').strip()
    if arduino_data:
        print(f'{timestamp} {temp_c}C, {temp_f}F, {arduino_data}')
        writer.writerow([timestamp, temp_c, temp_f] + arduino_data.split(','))
        file.flush()

        # Display temperature on line 1
        lcd_string(f"T: {temp_c}C {temp_f}F", LCD_LINE_1)

        # Scroll Arduino data on line 2
        lcd_scroll_string(arduino_data, LCD_LINE_2)
        time.sleep(60)

except serial.SerialException as e:
    print(f"Error connecting to Arduino: {e}")
except KeyboardInterrupt:
    print("Exiting program.")
finally:
    if 'ser' in locals() and ser.is_open:
        ser.close()
    print("Serial connection closed.")

```

Codebase 8.1 (Installing Crontab on the Raspberry Pi)

```
nano /home/pi/start_displaySensorData.sh

#!/bin/bash
echo "Starting displaySensorData.py at $(date)" >> /home/pi/displaySensorData.log
python3 /home/pi/displaySensorData.py &

nano /home/pi/stop_displaySensorData.sh

#!/bin/bash
echo "Stopping displaySensorData.py at $(date)" >> /home/pi/displaySensorData.log
pkill -f /home/pi/displaySensorData.py

chmod +x /home/pi/start_displaySensorData.sh

chmod +x /home/pi/stop_displaySensorData.sh

crontab -e

0 6 * * * /home/pi/start_displaySensorData.sh
0 21 * * * /home/pi/stop_displaySensorData.sh

crontab -l

ps aux | grep displaySensorData.py
grep CRON /var/log/syslog

@reboot /bin/sleep 30; /usr/bin/python /home/pi/displaySensorData.py
```

Codebase 9.1 (Complete Python Code)

```
# /usr/bin/env python3
import os
import glob
import time
import serial
import csv
import smbus
import firebase_admin
from firebase_admin import credentials, db

# --- Firebase Setup ---
cred = credentials.Certificate("/home/pi/firebase/hydro-app-fb074-default-rtdb-export.json")
# Update with your path
firebase_admin.initialize_app(cred, {
    "databaseURL": "https://hydro-app-fb074-default-rtdb.firebaseio.com" # Update with your
    database URL
})
ref = db.reference("sensor_data")

# --- Water Temperature Sensor Setup ---
os.system('modprobe w1-gpio')
os.system('modprobe w1-therm')

base_dir = '/sys/bus/w1/devices/'
device_path = glob.glob(base_dir + '28*')[0] # Get file path of sensor
rom = device_path.split('/')[-1] # Get ROM name

def read_temp_raw():
    with open(device_path + 'w1_slave', 'r') as f:
        valid, temp = f.readlines()
    return valid, temp

def read_temp():
    valid, temp = read_temp_raw()
    while 'YES' not in valid:
        time.sleep(0.5)
        valid, temp = read_temp_raw()

    pos = temp.index('t=')
    if pos != -1:
        temp_string = temp[pos + 2:]
        temp_c = format(round(float(temp_string) / 1000.0, 2), '.2f') # Round to 2 decimals
        temp_f = format(round(float(temp_c) * (9.0 / 5.0) + 32.0, 2), '.2f') # Round to 2
        decimals
    return temp_c, temp_f
    return None, None

# --- LCD Setup ---
I2C_ADDR = 0x27
LCD_WIDTH = 16
LCD_CHR = 1
```

```

LCD_CMD = 0
LCD_LINE_1 = 0x80
LCD_LINE_2 = 0xC0
LCD_BACKLIGHT = 0x08
ENABLE = 0b00000100

E_DELAY = 0.001

bus = smbus.SMBus(1)

def lcd_init():
    lcd_byte(0x33, LCD_CMD)
    lcd_byte(0x32, LCD_CMD)
    lcd_byte(0x06, LCD_CMD)
    lcd_byte(0x0C, LCD_CMD)
    lcd_byte(0x28, LCD_CMD)
    lcd_byte(0x01, LCD_CMD)
    time.sleep(E_DELAY)

def lcd_byte(bits, mode):
    bits_high = mode | (bits & 0xF0) | LCD_BACKLIGHT
    bits_low = mode | ((bits << 4) & 0xF0) | LCD_BACKLIGHT
    bus.write_byte(I2C_ADDR, bits_high)
    lcd_toggle_enable(bits_high)
    bus.write_byte(I2C_ADDR, bits_low)
    lcd_toggle_enable(bits_low)

def lcd_toggle_enable(bits):
    time.sleep(E_DELAY)
    bus.write_byte(I2C_ADDR, (bits | ENABLE))
    time.sleep(E_DELAY)
    bus.write_byte(I2C_ADDR, (bits & ~ENABLE))
    time.sleep(E_DELAY)

def lcd_string(message, line):
    message = message.ljust(LCD_WIDTH, " ")
    lcd_byte(line, LCD_CMD)
    for i in range(LCD_WIDTH):
        lcd_byte(ord(message[i]), LCD_CHR)

def lcd_scroll_string(message, line):
    """Scroll a long message across the LCD line."""
    if len(message) <= LCD_WIDTH:
        lcd_string(message, line)
    else:
        for i in range(len(message) - LCD_WIDTH + 1):
            lcd_string(message[i:i+LCD_WIDTH], line)
            time.sleep(0.2)

# --- Arduino Setup ---
arduino_port = "/dev/ttyACM0"
baud_rate = 9600

```

```

# --- Combined Data Writing ---
output_file = "CombinedData.csv"

def get_sensor_data():
    """Reads temperature from DS18B20 and sensor values from Arduino."""
    temp_c, temp_f = read_temp()
    timestamp = time.strftime("%Y-%m-%d %H:%M:%S")

    # Read Arduino data (voltage, pH, water level)
    arduino_data = ser.readline().decode('utf-8').strip()

    if arduino_data:
        try:
            voltage, ph, water_level = arduino_data.split(',')
            return {
                "timestamp": timestamp,
                "temperature_C": float(temp_c),
                "temperature_F": float(temp_f),
                "voltage": float(voltage),
                "pH": float(ph),
                "water_level": float(water_level)
            }
        except ValueError:
            print("Error parsing Arduino data:", arduino_data)
            return None
    return None

try:
    ser = serial.Serial(arduino_port, baud_rate, timeout=1)
    print("Connected to Arduino on", arduino_port)

    lcd_init()
    lcd_string("Initializing...", LCD_LINE_1)

    with open(output_file, mode='w', newline='') as file:
        writer = csv.writer(file)
        writer.writerow(["Time", "Temperature(C)", "Temperature(F)", "Voltage", "pH", "Water
Level"])

    print(f'ROM: {rom}')
    time.sleep(3)

    while True:
        sensor_data = get_sensor_data()
        if sensor_data:
            # Log to console
            print(f'{sensor_data["timestamp"]} {sensor_data["temperature_C"]}C,
{sensor_data["temperature_F"]}F, '
                f'V: {sensor_data["voltage"]} pH: {sensor_data["pH"]} WL:
{sensor_data["water_level"]}')
```

```

# Write to CSV
writer.writerow([
    sensor_data["timestamp"],
    sensor_data["temperature_C"],
    sensor_data["temperature_F"],
    sensor_data["voltage"],
    sensor_data["pH"],
    sensor_data["water_level"]
])
file.flush()

# Send to Firebase
ref.push(sensor_data)
print("Sent to Firebase:", sensor_data)

# Display temperature on Line 1
lcd_string(f"T: {sensor_data['temperature_C']}C
{sensor_data['temperature_F']}F", LCD_LINE_1)

# Scroll Arduino data on Line 2
arduino_data_string = f"V: {sensor_data['voltage']} pH: {sensor_data['pH']}
WL: {sensor_data['water_level']}"
start_time = time.time()
while (time.time() - start_time) < 30:
    lcd_scroll_string(arduino_data_string, LCD_LINE_2)

time.sleep(30) # Adjust this to allow sync

except serial.SerialException as e:
    print(f"Error connecting to Arduino: {e}")
except KeyboardInterrupt:
    print("Exiting program.")
finally:
    if 'ser' in locals() and ser.is_open:
        ser.close()
    print("Serial connection closed.")

```


List of Abbreviations

ADC: Analog to Digital Converter
AI: Artificial Intelligence
CAD: Computer-Aided Design
DWC: Deep Water Culture
EC: Electrical Conductivity
GPIO: General Purpose Input/Output
HTTP: Hypertext Transfer Protocol
IDE: Integrated Development Environment
IoT: Internet of Things
LCD: Liquid Crystal Display
NFT: Nutrient Film Technique
pH: Potential of Hydrogen
SSH: Secure Shell
UART: Universal Asynchronous Receiver-Transmitter

References

- "24 Top-Performing Companies in Hydroponics Farming." 2017. Inven.ai. 2017.
<https://www.inven.ai/company-lists/top-24-hydroponics-farming-companies>.
- Action Against Hunger. 2022. "8 Crops Endangered by Climate Change." Action against Hunger. January 14, 2022.
<https://www.actionagainsthunger.org/story/8-crops-endangered-climate-change/>.
- AeroFarms. n.d. "About AeroFarms · the Vertical Farming, Elevated Flavor Company."
AeroFarms. <https://www.aerofarms.com/about-us/>.
- Agriculture Academy. n.d. "What Is Hydroponics and How Does It Work?" Wwww.youtube.com.
<https://www.youtube.com/watch?v=VOBrGBF9IQM>.
- "APP." 2025. Plant Factory. 2025.
https://plantfactory.com.au/pages/app?srsId=AfmBOorG_-Asa6eSvFeJlwFqR9P9vECTuYbf9iLaM8rofJFFtPUANnrZ.
- ArduinoGetStarted.com. 2025. "Arduino File.write() | Arduino Reference." Arduino Getting Started. February 23, 2025.
<https://arduinogetstarted.com/reference/library/arduino-file.write>.
- "Automated Hydroponic System Build – Projects | Kyle Gabriel." n.d.
<https://kylegabriel.com/projects/2020/06/automated-hydroponic-system-build.html>.
- Bialecki Farms. 2018. "Kratky Hydroponic High Tunnel." YouTube. March 2, 2018.
<https://www.youtube.com/watch?v=3tjqvqxEJmE>.
- "Blocked." 2025. Medium.com. 2025.
<https://medium.com/@sandalidissanayake123/create-connectivity-between-raspberry-pi-and-firebase-cla82edb3b68>.
- Cairns, Rebecca. 2024. "This 'GigaFarm' in the Desert Could Produce 3 Million Kilograms of Food." CNN. March 14, 2024.
<https://www.cnn.com/world/dubai-gigafarm-biggest-vertical-farm-climate-hnk-spc-int/index.html>.
- "Climate Change Is Making It Harder to Grow the Potatoes Traditionally Used for French Fries» Yale Climate Connections." 2021. Yale Climate Connections. Yale Climate Connections. April 9, 2021.
<https://yaleclimateconnections.org/2021/04/climate-change-is-making-it-harder-to-grow-potatoes-french-fries/>.

- “Climate Change: How a Thirst for Soy Puts Latin America in the Eye of the Storm.” n.d. The Nature Conservancy.
<https://www.nature.org/en-us/about-us/where-we-work/latin-america/stories-in-latin-america/climate-change-thirst-soy-puts-latin-america-eye-storm/>.
- “Climate Profile: Saudi Arabia.” 2024. Atlantic Council. May 31, 2024.
<https://www.atlanticcouncil.org/programs/middle-east-programs/rafik-hariri-center-for-the-middle-east/empowerme/macromena/climate-profile-saudi-arabia/>.
- Davy Wybiral. 2018. “Calibrating an Analog PH Sensor.” YouTube. November 17, 2018.
<https://www.youtube.com/watch?v=9ElbTbh80gA>.
- Everest Fernandez. 2013. “How to Prepare and Pre-Soak Rockwool Cubes for Hydroponics—the Right Way!” YouTube. April 1, 2013.
<https://www.youtube.com/watch?v=luwxbCT1QCY>.
- Farm Plastic Supply. 2022. “High Tunnel vs. Greenhouse – Pros and Cons of Each.” Farm Plastic Supply. March 15, 2022.
https://farmplasticsupply.com/blog/greenhouse-coverings/high-tunnel-vs-greenhouse-pros-and-cons-of-each?srsltid=AfmBOoqhH3UesjjDOPQA0cpqa8FunP7tL5U__946HzrPHTWnOKq9_9ma.
- “Figure of the Week: Climate Change and African Agriculture.” n.d. Brookings.
<https://www.brookings.edu/articles/figure-of-the-week-climate-change-and-african-agriculture/>.
- Fromaget, Patrick. 2018. “How to Schedule a Task on a Raspberry Pi?” RaspberryTips. July 29, 2018. <https://raspberrytips.com/schedule-task-raspberry-pi/>.
- “Global Crop Yields Projected to Drop as Temperatures Rise, New Study Finds.” n.d. Alliance for Science.
<https://allianceforscience.cornell.edu/blog/2020/11/global-crop-yields-projected-to-drop-as-temperatures-rise-new-study-finds/>.
- “Grove – PH Sensor Kit (E-201C-Blue) – Seed Wiki.” n.d. Wiki.seeedstudio.com.
<https://wiki.seeedstudio.com/Grove-PH-Sensor-kit/>.
- Hoidal, Natalie. 2020. “Small-Scale Hydroponics.” Extension.umn.edu. 2020.
<https://extension.umn.edu/how/small-scale-hydroponics>.
- Hossain, Mosabber. 2021. “As Rising ‘Heat Shocks’ Ruin Rice Crops, Bangladesh Faces Hunger Risk.” Reuters, May 28, 2021.
<https://www.reuters.com/article/world/as-rising-heat-shocks-ruin-rice-crops-bangladesh-faces-hunger-risk-idUSKCN2D90NC/>.

"How to Use Rockwool Cubes for Growing, Seed Starting, and Cutting Propagation." 2018. Trees.com. June 3, 2018.
<https://www.trees.com/gardening-and-landscaping/rockwool-cubes>.

InFarm. n.d. "Infarm." Wwww.infarm.com. <https://www.infarm.com/>.

International Rescue Committee. 2023. "10 Countries at Risk of Climate Disaster ." Wwww.rescue.org. March 20, 2023.
<https://www.rescue.org/article/10-countries-risk-climate-disaster>.

Izzy Analytics. 2019. "Raspberry Pi DS18B20 Sensor." YouTube. July 22, 2019.
<https://www.youtube.com/watch?v=X48OoO8r2Vo>.

Judith. 2019. "PH in Hydroponics: How to Maintain the PH Levels of Hydroponic Systems." Blog.jencoi.com. August 30, 2019.
<https://blog.jencoi.com/ph-in-hydroponics-how-to-maintain-the-ph-levels-of-hydroponic-systems>.

Juniper, Tony. 2020. "How Climate Change Has Altered Cocoa Farming in Ivory Coast." Trellis. Trellis. January 25, 2020.
<https://trellis.net/article/how-climate-change-has-altered-cocoa-farming-ivory-coast/>.

Kratsch, Heidi. 2015. "Hydroponics: A Brief Guide to Growing Food without Soil." Extension | University of Nevada, Reno. 2015.
<https://extension.unr.edu/publication.aspx?PubID=2756>.

"LCD Scrolling Program - Raspberry Pi Forums." 2018. Raspberrypi.com. 2018.
<https://forums.raspberrypi.com/viewtopic.php?t=220239>.

M Heidenreich. 2020. "How to Use the LCD1602 I2C Display with Raspberry Pi (Python Tutorial with Multi-Threading)." YouTube. November 25, 2020.
<https://www.youtube.com/watch?v=DHbLBTRpTWM>.

Morrison, Rose. 2023. "11 Ways Farmers Are Adapting to Climate Change." Earth.org. October 26, 2023.
<https://earth.org/11-ways-farmers-are-adapting-to-the-unpredictability-of-climate-change/>

National Geographic . n.d. "5 Ways Climate Change Will Affect You: Crop Changes." Nationalgeographic.com.
<https://www.nationalgeographic.com/climate-change/how-to-live-with-it/crops.html>.

"Non-Contact Water/Liquid Level Sensor SKU: CQRSENYW001 - CQRobot-Wiki." 2025. Cqrobot.wiki. 2025.

- http://www.cqrobot.wiki/index.php/Non-Contact_Water/Liquid_Level_Sensor_SKU:_CQRSENYW001.
- Ogasa, Nikk. 2021. "Rice Feeds Half the World. Climate Change's Droughts and Floods Put It at Risk." Science News. September 24, 2021.
<https://www.sciencenews.org/article/rice-agriculture-feeds-world-climate-change-drought-flood-risk>.
- OklahomaGardening. 2025. "Hydroponics for the Home or Classroom." Youtu.be. 2025.
<https://youtu.be/oP1zEn-e4xI>.
- Omoyo, Nyandiko Nicodemus, Jacob Wakhungu, and Silvery Oteng'i. 2015. "Effects of Climate Variability on Maize Yield in the Arid and Semi Arid Lands of Lower Eastern Kenya." Agriculture & Food Security 4 (1). <https://doi.org/10.1186/s40066-015-0028-2>.
- Pankajakshan, Praveen. 2023. "How Climate Smart Agriculture Can Help Us Tackle Global Food Insecurity." Earth.org. May 8, 2023. <https://earth.org/climate-smart-agriculture/>.
- Pearson, Charlie. 2024. "Do Hydroponic Plants Need Sunlight?" Proponics. January 9, 2024.
<https://proponics.co.uk/blogs/lighting/do-hydroponic-plants-need-sunlight>.
- Rajaseger, Ganapathy, Kit Lun Chan, Kay Yee Tan, Shan Ramasamy, Mar Cho Khin, Anburaj Amaladoss, and Patel Kadamb Haribhai. 2023. "Hydroponics: Current Trends in Sustainable Crop Production." Bioinformation 19 (9): 925–38.
<https://doi.org/10.6026/97320630019925>.
- Ramirez, Ruzell. 2020. "How to Start Programs Automatically on the Raspberry Pi." Circuit Basics. November 16, 2020.
<https://www.circuitbasics.com/starting-programs-automatically-using-cron-on-a-raspberry-pi/>.
- "Raspberry Pi DS18B20 Temperature Sensor Tutorial." 2016. YouTube Video. YouTube.
<https://www.youtube.com/watch?v=aEnSO-Jy2vE>.
- "Send Data to Firebase Using Raspberry Pi." n.d. Wwww.youtube.com.
https://www.youtube.com/watch?v=_cdQOBsMt-Y.
- Singh, Hardeep, and Bruce Dunn. 2017. "Electrical Conductivity and PH Guide for Hydroponics – Oklahoma State University." Extension.okstate.edu. April 2017.
<https://extension.okstate.edu/fact-sheets/electrical-conductivity-and-ph-guide-for-hydroponics.html>.
- US EPA. 2015. "Ecosystem Effects of Ozone Pollution | US EPA." US EPA. June 11, 2015.
<https://www.epa.gov/ground-level-ozone-pollution/ecosystem-effects-ozone-pollution>.

- US EPA, OAR. 2022. "Climate Change Impacts on Agriculture and Food Supply." Wwww.epa.gov. October 19, 2022.
<https://www.epa.gov/climateimpacts/climate-change-impacts-agriculture-and-food-supply#10foot>.
- USDA. 2024. "USDA ERS - Farming and Farm Income." Usda.gov. September 5, 2024.
<https://www.ers.usda.gov/data-products/ag-and-food-statistics-charting-the-essentials/farming-and-farm-income/>.
- "What Is the Ideal Water Temperature for Hydroponics? | Growee." 2022. Growee Smart Hydroponics. December 21, 2022.
<https://getgrowee.com/ideal-water-temperature-for-hydroponics/>.
- "What Plants Can Be Grown in Water?" n.d. Miracleagro.com.
<https://miracleagro.com/en-us/indoor-gardening/what-plants-can-be-grown-in-water.html>.
2025. AppHarvest - Fight the Food Fight. 2025.
<https://appharvest.wpenginepowered.com/sustainability/>.